Spring Security

实战干货

码农小胖哥 著



felord.cn 出品

版权声明

本教程版权归作者所有，不得用于有偿商业场景。

个人介绍

我是 **码农小胖哥**，一名软件工程师，同时也是一名热衷于技术分享、拥抱开源的技术博主。

我的技术栈主要是Java、Python、Golang和Kotlin。欢迎各位同行进行交流分享。

✓我的个人技术分享都集中在我的个人博客：https://felord.cn

✓业余运营有自己的公众号：



✓也在微信视频号上进行一些创作：



✓我在其它平台的专栏：

开发环境

- Spring Security 5.x
- Spring Boot 2.x
- Java 8+

准备工作

1. 前言

web应用达到生产需要就必须有安全控制。java web领域经常提及的两大开源框架主要有两种选择Spring Security和Apache Shiro 。所以学习这两种框架也是java开发者提高水平的必经之路。从今天开始连续一段时间内，研究一下Spring Security。如果想学习的同学可以关注一下公众号：`Felordcn` 或者通过`https://felord.cn`来及时获取相关的干货。

2. Spring Security 和Apache Shiro

相对于Apache Shiro，Spring Security提供了更多的诸如`LDAP`、`OAuth2.0`、`ACL`、`Kerberos`、`SAML`、`SSO`、`OpenID`等诸多的安全认证、鉴权协议，可以按需引用。对认证/鉴权更加灵活，粒度更细。可以结合你自己的业务场景进行更加合理的定制化开发。在最新的Spring Security 5.x中更是提供了响应式应用（reactive application）提供了安全控制支持。从语言上来讲，支持使用kotlin、groovy进行开发。

Spring Security因为是利用了Spring IOC 和AOP的特性而无法脱离Spring独立存在。而Apache Shiro可以独立存在。但是Java Web领域Spring可以说是事实上的J2EE规范。使用Java技术栈很少能脱离Spring。也因为功能强大Spring Security被认为非常重，这是不对的。认真学习之后会发现其实也就是那么回事。两种框架都是非常优秀的安全框架，根据实际需要做技术选型。如果你要学习这两种安全框架就必须熟悉一下一些相对专业的概念。

3. 认证/鉴权

这两个概念英文分别为`authentication`/`authorization`。是不是特别容易混淆。无论你选择Apache Shiro 或者 Spring Security 都需要熟悉这两个概念。其实简单来说认证（authentication）就是为了证明`你是谁`，比如你输入账号密码证明你是用户名为`Felordcn`的用户。而授权（authorization）是通过认证后的用户所绑定的角色等凭证来证明`你可以做什么`。打一个现实中的例子。十一长假大家远行都要乘坐交通工具，现在坐车实名制，也就是说你坐车需要两件东西：`身份证`和`车票`。身份证是为了证明你确实是你，这就是`authentication`；而车票是为了证明你张三确实买了票可以上车，这就是`authorization`。这个例子从另一方面也证明了。如果只有认证没有授权，认证就没有意义。如果没有认证，授权就无法赋予真正的可信任的用户。两者是同时存在的。

4. 过滤器链

对于servlet web应用来说，想要通用的安全控制最好莫过于使用`Servlet Filter`。过滤器责任链（关于责任链可以通过**https://www.felord.cn/chainpattern.html** 来了解）来组成一系列的过滤策略，不同的条件的请求进入不同的过滤器进行各自的处理逻辑。我们可以对这些`Filter`进行排列组合以满足我们的实际业务需要。

5. RBAC模型

RBAC 是基于角色的访问控制（Role-Based Access Control ）的简称。在 RBAC 中，权限与角色相关联，用户通过成为适当角色的成员而得到这些角色的权限。这就极大地简化了权限的管理。这样管理都是层级相互依赖的，权限赋予给角色，而把角色又赋予用户，这样的权限设计很清楚，管理起来很方便。当你拥有某个角色以后，你自然继承了该角色的所有功能。对你的一些操作限制不需要直接与你进行沟通，只需要操作你拥有的角色。比如你在公司既是一个java程序员又是一个前端程序员，那么你不但要当sqlboy还要当页面仔。如果有一天经理说了前端负责测试工作，好了你又承担了测试任务。

6. 其他一些概念

比如其它一些常见的安全策略、攻击方式。比如 反向代理、网关、壁垒机这种偏运维的知识；CSRF（Cross-site request forgery）跨站请求伪造 、XSS（跨站脚本攻击）也需要了解一些。对于一些上面提到的什么OAuth2.0之类的协议也最好研究一下。当然这些不是必须的。

总结

本文粗略的简述了Spring Security 和Apache Shiro的一些异同。以及学习它们的一些前置条件。如果你不满足这些条件学习起来可能比较吃力。所以本文的作用是为你学习预热，做一些准备工作，避免新入门的同学陷入迷途。也希望大家多多支持，多多关注码农小胖哥。

Spring Security中的用户信息

1. 前言

前一篇介绍了 `Spring Security` 入门的基础准备。从今天开始我们来一步步窥探它是如何工作的。我们又该如何驾驭它。请多多关注公众号： `Felordcn` 。本篇将通过 `Spring Boot 2.x` 来讲解 `Spring Security` 中的用户主体`UserDetails`。以及从中找点乐子。

2. Spring Boot 集成 Spring Security

这个简直老生常谈了。不过为了照顾大多数还是说一下。集成 `Spring Security` 只需要引入其对应的 `Starter` 组件。 `Spring Security` 不仅仅能保护`Servlet Web` 应用，也可以保护`Reactive Web`应用，本文我们讲前者。我们只需要在 `Spring Security` 项目引入以下依赖即可：

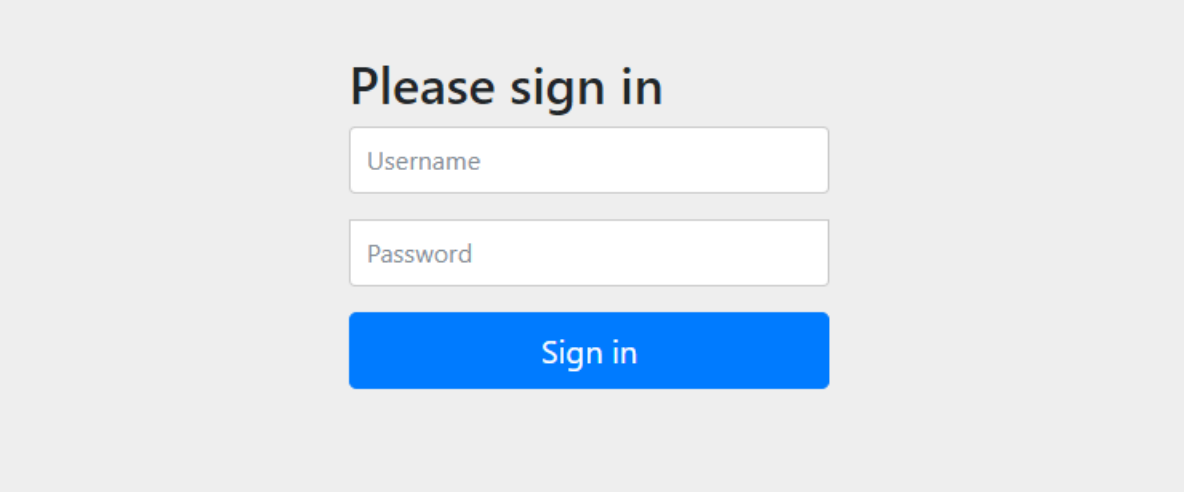
```
    <dependencies>
        <!--  actuator 指标监控  非必须 -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-actuator</artifactId>
        </dependency>
        <!--  spring security starter 必须  -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <!-- spring mvc  servlet web  必须  -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <!--   lombok 插件 非必须       -->
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
```

```
            <optional>true</optional>
        </dependency>
        <!-- 测试   -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.security</groupId>
            <artifactId>spring-security-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

```

3. UserDetailsServiceAutoConfiguration

启动项目，访问`Actuator`端点`http://localhost:8080/actuator`会跳转到一个登录页面`http://localhost:8080/login`如下：



要求你输入用户名`Username`（默认值为user）和密码`Password`。密码在springboot控制台会打印出类似`Using generated security password: e1f163be-ad18-4be1-977c-88a6bcee0d37`的字样，后面的长串就是密码，当然这不是生产可用的。如果你足够细心会从控制台打印日志发现该随机密码是由`UserDetailsServiceAutoConfiguration`配置类生成的，我们就从它开始顺藤摸瓜来一探究竟。

3.1 UserDetailsService

`UserDetailsService`接口。该接口只提供了一个方法：

```
    UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException;
```

该方法很容易理解：**通过用户名来加载用户**。这个方法主要用于从系统数据中查询并加载具体的用户到Spring Security中。

3.2 UserDetails

从上面`UserDetailsService`可以知道最终交给Spring Security的是`UserDetails`。该接口是提供用户信息的核心接口。该接口实现仅仅存储用户的信息。后续会将该接口提供的用户信息封装到认证对象`Authentication`中去。`UserDetails`默认提供了：

- 用户的权限集，默认需要添加`ROLE_`前缀
- 用户的加密后的密码，不加密会使用`{noop}`前缀
- 应用内唯一的用户名
- 账户是否过期
- 账户是否锁定
- 凭证是否过期
- 用户是否可用

如果以上的信息满足不了你使用，你可以自行实现扩展以存储更多的用户信息。比如用户的邮箱、手机号等等。通常我们使用其实现类：

```
org.springframework.security.core.userdetails.User
```

该类内置一个建造器`UserBuilder`会很方便地帮助我们构建`UserDetails`对象，后面我们会用到它。

3.3 UserDetailsServiceAutoConfiguration

`UserDetailsServiceAutoConfiguration`全限定名为：

```
org.springframework.boot.autoconfigure.security.servlet.UserDetailsServiceAutoConfiguration
```

源码如下：

```
@Configuration
@ConditionalOnClass(AuthenticationManager.class)
@ConditionalOnBean(ObjectPostProcessor.class)
@ConditionalOnMissingBean({ AuthenticationManager.class, AuthenticationProvider.class, UserDetailsService.class })
public class UserDetailsServiceAutoConfiguration {

    private static final String NOOP_PASSWORD_PREFIX = "{noop}";

    private static final Pattern PASSWORD_ALGORITHM_PATTERN = Pattern.compile("^\\{.+}.*$");

    private static final Log logger = LogFactory.getLog(UserDetailsServiceAutoConfiguration.class);

    @Bean
    @ConditionalOnMissingBean(
            type = "org.springframework.security.oauth2.client.registration.ClientRegistrationRepository")
    @Lazy
    public InMemoryUserDetailsManager inMemoryUserDetailsManager(SecurityProperties properties,
            ObjectProvider<PasswordEncoder> passwordEncoder) {
```

```
19          SecurityProperties.User user = properties.getUser();
20          List<String> roles = user.getRoles();
21          return new InMemoryUserDetailsManager(
22
    User.withUsername(user.getName()).password(getOrDeducePassword(user,
    passwordEncoder.getIfAvailable()))
23                          .roles(StringUtils.toStringArray(roles)).build());
24      }
25
26      private String getOrDeducePassword(SecurityProperties.User user,
    PasswordEncoder encoder) {
27          String password = user.getPassword();
28          if (user.isPasswordGenerated()) {
29              logger.info(String.format("%n%nUsing generated security password:
    %s%n", user.getPassword()));
30          }
31          if (encoder != null ||
    PASSWORD_ALGORITHM_PATTERN.matcher(password).matches()) {
32              return password;
33          }
34          return NOOP_PASSWORD_PREFIX + password;
35      }
36
37  }
38
```

我们来简单解读一下该类，从 `@Conditional` 系列注解我们知道该类在类路径下存在 `AuthenticationManager` 、在Spring 容器中存在Bean `ObjectPostProcessor` 并且不存在Bean `AuthenticationManager` , `AuthenticationProvider` , `UserDetailsService` 的情况下生效。**千万不要纠结这些类干嘛用的!** 该类只初始化了一个 `UserDetailsManager` 类型的Bean。 `UserDetailsManager` 类型负责对安全用户实体抽象 `UserDetails` 的增删查改操作。同时还继承了 `UserDetailsService` 接口。

明白了上面这些让我们把目光再回到 `UserDetailsServiceAutoConfiguration` 上来。该类初始化了一个名为 `InMemoryUserDetailsManager` 的内存用户管理器。该管理器通过配置注入了一个默认的 `UserDetails` 存在内存中，就是我们上面用的那个 `user` ，每次启动 `user` 都是动态生成的。**那么问题来了如果我们定义自己的 `UserDetailsManager` Bean是不是就可以实现我们需要的用户管理逻辑呢?**

3.4 自定义UserDetailsManager

我们来自定义一个 `UserDetailsManager` 来看看能不能达到自定义用户管理的效果。首先我们针对 `UserDetailsManager` 的所有方法进行一个代理的实现，我们依然将用户存在内存中，区别就是这是我们自定义的:

```
1   package cn.felord.spring.security;
2
3   import org.springframework.security.access.AccessDeniedException;
4   import org.springframework.security.core.Authentication;
5   import org.springframework.security.core.context.SecurityContextHolder;
6   import org.springframework.security.core.userdetails.UserDetails;
7   import org.springframework.security.core.userdetails.UsernameNotFoundException;
8
9   import java.util.HashMap;
10  import java.util.Map;
11
```

```java
/**
 * 代理 {@link org.springframework.security.provisioning.UserDetailsManager} 所有功能
 *
 * @author Felordcn
 */
public class UserDetailsRepository {

    private Map<String, UserDetails> users = new HashMap<>();


    public void createUser(UserDetails user) {
        users.putIfAbsent(user.getUsername(), user);
    }


    public void updateUser(UserDetails user) {
        users.put(user.getUsername(), user);
    }


    public void deleteUser(String username) {
        users.remove(username);
    }


    public void changePassword(String oldPassword, String newPassword) {
        Authentication currentUser = SecurityContextHolder.getContext()
                .getAuthentication();

        if (currentUser == null) {
            // This would indicate bad coding somewhere
            throw new AccessDeniedException(
                    "Can't change password as no Authentication object found in context "
                            + "for current user.");
        }

        String username = currentUser.getName();

        UserDetails user = users.get(username);


        if (user == null) {
            throw new IllegalStateException("Current user doesn't exist in database.");
        }

        // todo copy InMemoryUserDetailsManager  自行实现具体的更新密码逻辑
    }


    public boolean userExists(String username) {

        return users.containsKey(username);
    }


```

```
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        return users.get(username);
    }
}
```

该类负责具体对 `UserDetails` 的增删改查操作。我们将其注入Spring 容器：

```
    @Bean
    public UserDetailsRepository userDetailsRepository() {
        UserDetailsRepository userDetailsRepository = new UserDetailsRepository();

        // 为了让我们的登录能够运行 这里我们初始化一个用户Felordcn 密码采用明文 当你在密码12345上使用了前缀{noop} 意味着你的密码不使用加密，authorities 一定不能为空 这代表用户的角色权限集合
        UserDetails felordcn = User.withUsername("Felordcn").password("{noop}12345").authorities(AuthorityUtils.NO_AUTHORITIES).build();
        userDetailsRepository.createUser(felordcn);
        return userDetailsRepository;
    }
```

为了方便测试 我们也内置一个名称为 `Felordcn` 密码为 `12345` 的 `UserDetails` 用户，密码采用明文 当你在密码 `12345` 上使用了前缀 `{noop}` 意味着你的密码不使用加密，这里我们并没有指定密码加密方式你可以使用 `PasswordEncoder` 来指定一种加密方式。通常推荐使用 `Bcrypt` 作为加密方式。默认Spring Security使用的也是此方式。authorities 一定不能为 `null` 这代表用户的角色权限集合。接下来我们实现一个 `UserDetailsManager` 并注入Spring 容器：

```
    @Bean
    public UserDetailsManager userDetailsManager(UserDetailsRepository userDetailsRepository) {
        return new UserDetailsManager() {
            @Override
            public void createUser(UserDetails user) {
                userDetailsRepository.createUser(user);
            }

            @Override
            public void updateUser(UserDetails user) {
                userDetailsRepository.updateUser(user);
            }

            @Override
            public void deleteUser(String username) {
                userDetailsRepository.deleteUser(username);
            }

            @Override
            public void changePassword(String oldPassword, String newPassword) {
                userDetailsRepository.changePassword(oldPassword, newPassword);
            }

            @Override
            public boolean userExists(String username) {
                return userDetailsRepository.userExists(username);
            }

            @Override
```

```
30              public UserDetails loadUserByUsername(String username) throws
    UsernameNotFoundException {
31                  return userDetailsRepository.loadUserByUsername(username);
32              }
33          };
34      }
```

这样实际执行委托给了 `UserDetailsRepository` 来做。我们重复 `章节3.` 的动作进入登陆页面分别输入 `Felordcn` 和 `12345` 成功进入。

3.5 数据库管理用户

经过以上的配置，相信聪明的你已经知道如何使用数据库来管理用户了 。只需要将 `UserDetailsRepository` 中的 `users` 属性替代为抽象的Dao接口就行了，无论你使用 `Jpa` 还是 `Mybatis` 来实现。

4. 总结

今天我们对Spring Security 中的用户信息 `UserDetails` 相关进行的一些解读。并自定义了用户信息处理服务。相信你已经对在Spring Security中如何加载用户信息，如何扩展用户信息有所掌握了。后面我们会由浅入深慢慢解读Spring Security。相关代码已经上传git仓库,关注公众号 `Felordcn` 后回复 `ss01` 获取demo源码。 后续也可以及时获取更多相关干货教程。

如何保护用户的密码

1. 前言

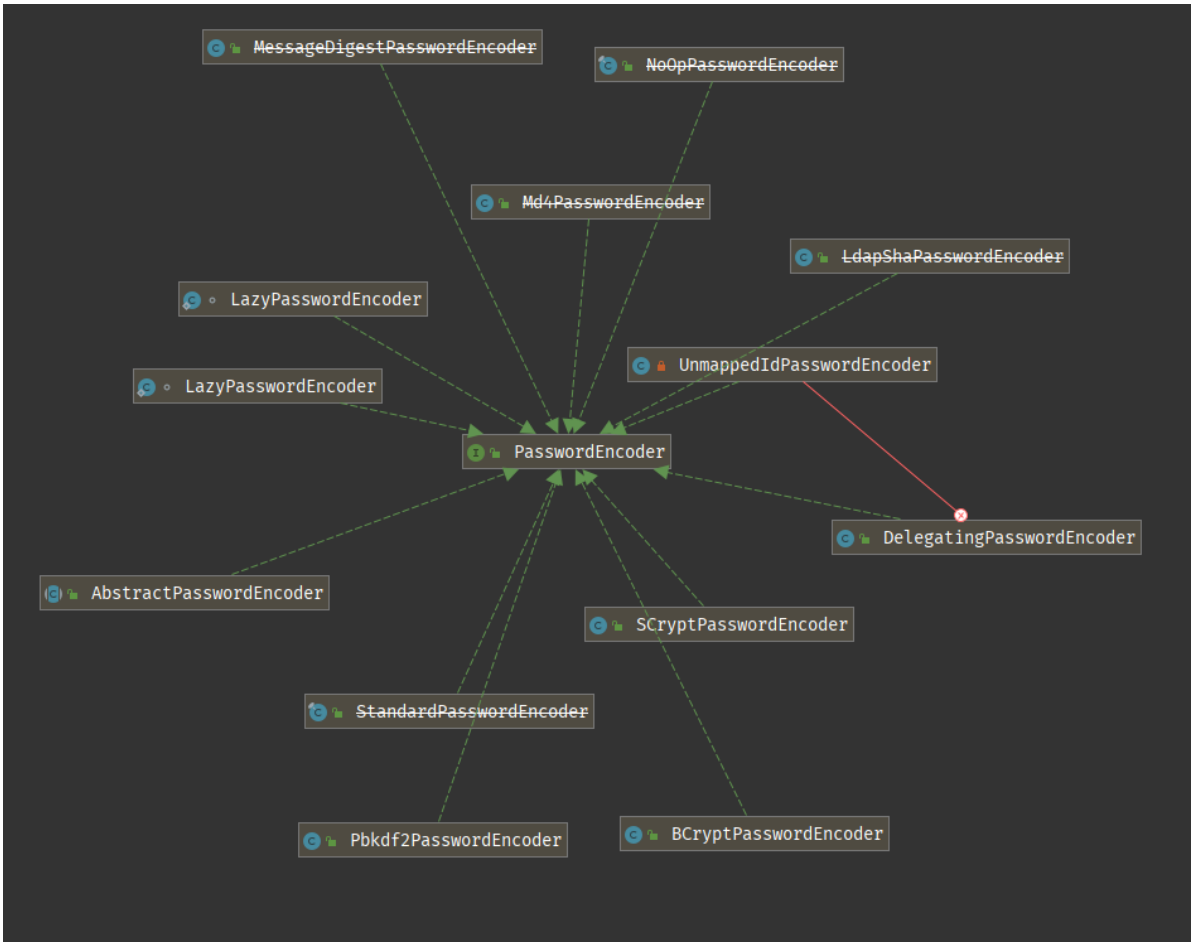
上一文（https://www.felord.cn） 我们对Spring Security中的重要用户信息主体 `UserDetails` 进行了探讨。中间例子我们使用了明文密码，规则是通过对密码明文添加 `{noop}` 前缀。那么本节将对 Spring Security 中的密码编码进行一些探讨。

2. 不推荐使用md5

首先 `md5` 不是加密算法，是哈希摘要。以前通常使用其作为密码哈希来保护密码。由于彩虹表的出现， `md5` 和 `sha1` 之类的摘要算法都已经不安全了。如果有不相信的同学 可以到一些解密网站 如 cmd5 网站尝试解密 你会发现 `md5` 和 `sha1` 是真的非常容易被破解。

3. Spring Security中的密码算法

上一文（https://www.felord.cn） 我们提到了 `InMemoryUserDetailsManager` 初始化Bean 需要传输一个 `ObjectProvider<PasswordEncoder>` 参数。这里的 `PasswordEncoder` 就是我们对密码进行编码的工具接口。该接口只有两个功能： 一个是匹配验证。另一个是密码编码。



上图就是Spring Security 提供的 `org.springframework.security.crypto.password.PasswordEncoder` 一些实现，有的已经过时。其中我们注意到一个叫**委托密码编码器的实现** 。

3.1 委托密码编码器 DelegatingPasswordEncoder

什么是委托（Delegate）？ 就是甲方交给乙方的活。乙方呢手里又很多的渠道，但是乙方光想赚差价又不想干活。所以乙方根据一些规则又把活委托给了别人，让别人来干。这里的乙方就是 `DelegatingPasswordEncoder` 。该类维护了以下清单：

- `final String idForEncode` 通过id来匹配编码器，该id不能是 `{}` 包括的。`DelegatingPasswordEncoder` 初始化传入，用来提供默认的密码编码器。
- `final PasswordEncoder passwordEncoderForEncode` 通过上面 `idForEncode` 所匹配到的 `PasswordEncoder` **用来对密码进行编码**。
- `final Map<String, PasswordEncoder> idToPasswordEncoder` 用来维护多个 `idForEncode` 与具体 `PasswordEncoder` 的映射关系。 `DelegatingPasswordEncoder` 初始化时装载进去，会在初始化时进行一些规则校验。
- `PasswordEncoder defaultPasswordEncoderForMatches = new UnmappedIdPasswordEncoder()` 默认的密码匹配器，上面的 `Map` 中都不存在就用它来执行 `matches` 方法进行匹配验证。这是一个内部类实现。

`DelegatingPasswordEncoder` 编码方法：

```
1       @Override
2       public String encode(CharSequence rawPassword) {
3           return PREFIX + this.idForEncode + SUFFIX +
    this.passwordEncoderForEncode.encode(rawPassword);
4       }
5
```

从上面源码可以看出来通过`DelegatingPasswordEncoder`编码后的密码是遵循一定的规则的，遵循`{idForEncode}encodePassword`。也就是前缀`{}`包含了编码的方式再拼接上该方式编码后的密码串。

`DelegatingPasswordEncoder`密码匹配方法：

```
1       @Override
2       public boolean matches(CharSequence rawPassword, String
    prefixEncodedPassword) {
3           if (rawPassword == null && prefixEncodedPassword == null) {
4               return true;
5           }
6           String id = extractId(prefixEncodedPassword);
7           PasswordEncoder delegate = this.idToPasswordEncoder.get(id);
8           if (delegate == null) {
9               return this.defaultPasswordEncoderForMatches
10                  .matches(rawPassword, prefixEncodedPassword);
11          }
12          String encodedPassword = extractEncodedPassword(prefixEncodedPassword);
13          return delegate.matches(rawPassword, encodedPassword);
14      }
```

密码匹配通过传入原始密码和遵循`{idForEncode}encodePassword`规则的密码编码串。通过获取编码方式id（`idForEncode`）来从`DelegatingPasswordEncoder`中的映射集合`idToPasswordEncoder`中获取具体的`PasswordEncoder`进行匹配校验。找不到就使用`UnmappedIdPasswordEncoder`。

这就是`DelegatingPasswordEncoder`的工作流程。那么`DelegatingPasswordEncoder`在哪里实例化呢?

3.2 密码器静态工厂PasswordEncoderFactories

从名字上就看得出来这是个工厂啊，专门制造`PasswordEncoder`。而且还是个静态工厂只提供了初始化`DelegatingPasswordEncoder`的方法：

```
1       @SuppressWarnings("deprecation")
2       public static PasswordEncoder createDelegatingPasswordEncoder() {
3           String encodingId = "bcrypt";
4           Map<String, PasswordEncoder> encoders = new HashMap<>();
5           encoders.put(encodingId, new BCryptPasswordEncoder());
6           encoders.put("ldap", new
    org.springframework.security.crypto.password.LdapShaPasswordEncoder());
7           encoders.put("MD4", new
    org.springframework.security.crypto.password.Md4PasswordEncoder());
8           encoders.put("MD5", new
    org.springframework.security.crypto.password.MessageDigestPasswordEncoder("MD5"))
    ;
9           encoders.put("noop",
    org.springframework.security.crypto.password.NoOpPasswordEncoder.getInstance());
10          encoders.put("pbkdf2", new Pbkdf2PasswordEncoder());
11          encoders.put("scrypt", new SCryptPasswordEncoder());
12          encoders.put("SHA-1", new
    org.springframework.security.crypto.password.MessageDigestPasswordEncoder("SHA-
    1"));
13          encoders.put("SHA-256", new
    org.springframework.security.crypto.password.MessageDigestPasswordEncoder("SHA-
    256"));
```

```
14          encoders.put("sha256", new
    org.springframework.security.crypto.password.StandardPasswordEncoder());
15
16          return new DelegatingPasswordEncoder(encodingId, encoders);
17      }
```

从上面可以非常具体地看出来 `DelegatingPasswordEncoder` 提供的密码编码方式。默认采用了**bcrypt**进行编码。我们可终于明白了为什么上一文中我们使用 `{noop}12345` 能和我们前台输入的 `12345` 匹配上。**这么搞有什么好处呢？这可以实现一个场景，如果有一天我们对密码编码规则进行替换或者轮转。现有的用户不会受到影响。** 那么Spring Security 是如何配置密码编码器 `PasswordEncoder` 呢?

4. Spring Security 加载 PasswordEncoder 的规则

我们在Spring Security配置适配器 `WebSecurityConfigurerAdapter` （该类我以后的文章会仔细分析可通过 `https://felord.cn` 来及时获取相关信息）找到了引用 `PasswordEncoderFactories` 的地方，一个内部 `PasswordEncoder` 实现 `LazyPasswordEncoder` 。从源码上看该类是懒加载的只有用到了才去实例化。在该类的内部方法中发现了 `PasswordEncoder` 的规则。

```
1           // 获取最终干活的PasswordEncoder
2           private PasswordEncoder getPasswordEncoder() {
3               if (this.passwordEncoder != null) {
4                   return this.passwordEncoder;
5               }
6               PasswordEncoder passwordEncoder =
    getBeanOrNull(PasswordEncoder.class);
7               if (passwordEncoder == null) {
8                   passwordEncoder =
    PasswordEncoderFactories.createDelegatingPasswordEncoder();
9               }
10              this.passwordEncoder = passwordEncoder;
11              return passwordEncoder;
12          }
13          // 从Spring IoC容器中获取Bean 有可能获取不到
14          private <T> T getBeanOrNull(Class<T> type) {
15              try {
16                  return this.applicationContext.getBean(type);
17              } catch(NoSuchBeanDefinitionException notFound) {
18                  return null;
19              }
20          }
```

上面的两个方法总结：如果能从从Spring IoC容器中获取 `PasswordEncoder` 的Bean就用该Bean作为编码器，没有就使用 `DelegatingPasswordEncoder` 。默认是 `bcrypt` 方式。文中多次提到该算法。而且还是Spring Security默认的。那么它到底是什么呢?

5. bcrypt 编码算法

这里简单提一下 `bcrypt` ， `bcrypt` 使用的是布鲁斯·施内尔在1993年发布的 `Blowfish` 加密算法。 `bcrypt` 算法将 `salt` 随机并混入最终加密后的密码，验证时也无需单独提供之前的 `salt` ，从而无需单独处理 `salt` 问题。加密后的格式一般为：

```
1       $2a$10$/bTVvqqlH9UiE0ZJZ7N2Me3RIgUCdgMheyTgV0B4cMCSokPa.6oCa
```

其中：`$`是分割符，无意义;`2a`是`bcrypt`加密版本号；`10`是`cost`的值；而后的前`22`位是`salt`值；再然后的字符串就是密码的密文了。

5.1 bcrypt 特点

- `bcrypt`有个特点就是非常慢。这大大提高了使用彩虹表进行破解的难度。也就是说该类型的密码暗文拥有让破解者无法忍受的时间成本。同时对于开发者来说也需要注意该时长是否能超出系统忍受范围内。通常是`MD5`的数千倍。
- 同样的密码每次使用`bcrypt`编码，密码暗文都是不一样的。 也就是说你有两个网站如果都使用了`bcrypt` 它们的暗文是不一样的，这不会因为一个网站泄露密码暗文而使另一个网站也泄露密码暗文。

所以从`bcrypt`的特点上来看，其安全强度还是非常有保证的。

6. 总结

今天我们对Spring Security中的密码编码进行分析。发现了默认情况下使用`bcrypt`进行编码。而密码验证匹配则通过密码暗文前缀中的加密方式id控制。你也可以向Spring IoC容器注入一个`PasswordEncoder`类型的Bean 来达到自定义的目的。我们还对`bcrypt`算法进行一些简单了解，对其特点进行了总结。后面我们会Spring Security进行进一步学习。关于上一篇文章的demo我也已经替换成了数据库管理用户。相关的代码你可以通过关注我公众号: `Felordcn` 回复 `ss02` 获取。

Spring Boot 中的 Spring Security 自动配置初探

1. 前言

我们在前几篇对 Spring Security 的用户信息管理机制，密码机制进行了探讨。我们发现 **Spring Security Starter**相关的 `Servlet` 自动配置都在 `spring-boot-autoconfigure-2.1.9.RELEASE` （当前 Spring Boot 版本为 `2.1.9.RELEASE` ） 模块的路径 `org.springframework.boot.autoconfigure.security.servlet` 之下。其实官方提供的Starter组件的自动配置你都能在 `spring-boot-autoconfigure-2.1.9.RELEASE` 下找到。今天我们进一步来解密 **Spring Security** 在 **Spring Boot** 的配置和使用。

> 对于学习Spring Security的最好方式莫过于从应用入手，然后逆着推导。

2. Spring Boot 下 Spring Security 的自动配置

我们可以通过 `org.springframework.boot.autoconfigure.security.servlet` 路径下找到 **Spring Security** 关于 `Servlet` 的自动配置类。我们来大致了解一下。

2.1 SecurityAutoConfiguration

```
1    package org.springframework.boot.autoconfigure.security.servlet;
2
3    import org.springframework.boot.autoconfigure.EnableAutoConfiguration;
4    import org.springframework.boot.autoconfigure.condition.ConditionalOnClass;
```

```
5   import
    org.springframework.boot.autoconfigure.condition.ConditionalOnMissingBean;
6   import
    org.springframework.boot.autoconfigure.security.SecurityDataConfiguration;
7   import org.springframework.boot.autoconfigure.security.SecurityProperties;
8   import
    org.springframework.boot.context.properties.EnableConfigurationProperties;
9   import org.springframework.context.ApplicationEventPublisher;
10  import org.springframework.context.annotation.Bean;
11  import org.springframework.context.annotation.Configuration;
12  import org.springframework.context.annotation.Import;
13  import
    org.springframework.security.authentication.AuthenticationEventPublisher;
14  import
    org.springframework.security.authentication.DefaultAuthenticationEventPublisher;
15
16  /**
17   * {@link EnableAutoConfiguration Auto-configuration} for Spring Security.
18   *
19   * @author Dave Syer
20   * @author Andy Wilkinson
21   * @author Madhura Bhave
22   * @since 1.0.0
23   */
24  @Configuration
25  @ConditionalOnClass(DefaultAuthenticationEventPublisher.class)
26  @EnableConfigurationProperties(SecurityProperties.class)
27  @Import({ SpringBootWebSecurityConfiguration.class,
    WebSecurityEnablerConfiguration.class,
28          SecurityDataConfiguration.class })
29  public class SecurityAutoConfiguration {
30
31      @Bean
32      @ConditionalOnMissingBean(AuthenticationEventPublisher.class)
33      public DefaultAuthenticationEventPublisher
    authenticationEventPublisher(ApplicationEventPublisher publisher) {
34          return new DefaultAuthenticationEventPublisher(publisher);
35      }
36
37  }
```

`SecurityAutoConfiguration` 顾名思义安全配置类。该类引入（`@import`）了 `SpringBootWebSecurityConfiguration`、`WebSecurityEnablerConfiguration` 和 `SecurityDataConfiguration` 三个配置类。 让这三个模块的类生效。是一个复合配置，是 Spring Security 自动配置最重要的一个类之一。 Spring Boot 自动配置经常使用这种方式以达到灵活配置的目的，这也是我们研究 Spring Security 自动配置的一个重要入口 同时 `SecurityAutoConfiguration` 还将 `DefaultAuthenticationEventPublisher` 作为默认的 `AuthenticationEventPublisher` 注入 **Spring IoC** 容器。如果你熟悉 Spring 中的事件机制你就会知道该类是一个 Spring 事件发布器。该类内置了一个 `HashMap<String, Constructor<? extends AbstractAuthenticationEvent>>` 维护了认证异常处理和对应异常事件处理逻辑的映射关系，比如账户过期异常 `AccountExpiredException` 对应认证过期事件 `AuthenticationFailureExpiredEvent` ，也就是说发生不同认证的异常使用不同处理策略。

2.2 SpringBootWebSecurityConfiguration

```
@Configuration
@ConditionalOnClass(WebSecurityConfigurerAdapter.class)
@ConditionalOnMissingBean(WebSecurityConfigurerAdapter.class)
@ConditionalOnWebApplication(type = Type.SERVLET)
public class SpringBootWebSecurityConfiguration {

  @Configuration
  @Order(SecurityProperties.BASIC_AUTH_ORDER)
  static class DefaultConfigurerAdapter extends WebSecurityConfigurerAdapter {

  }

}
```

这个类是Spring Security 对 Spring Boot Servlet Web 应用的默认配置。核心在于 `WebSecurityConfigurerAdapter` 适配器。从 `@ConditionalOnMissingBean(WebSecurityConfigurerAdapter.class)` 我们就能看出 `WebSecurityConfigurerAdapter` 是安全配置的核心。 默认情况下 `DefaultConfigurerAdapter` 将以 `SecurityProperties.BASIC_AUTH_ORDER` （`-5`） 的顺序注入 Spring IoC 容器，这是个空实现。如果我们需要个性化可以通过继承 `WebSecurityConfigurerAdapter` 来实现。我们会在以后的博文重点介绍该类。

2.3 WebSecurityEnablerConfiguration

```
@Configuration
@ConditionalOnBean(WebSecurityConfigurerAdapter.class)
@ConditionalOnMissingBean(name = BeanIds.SPRING_SECURITY_FILTER_CHAIN)
@ConditionalOnWebApplication(type = ConditionalOnWebApplication.Type.SERVLET)
@EnableWebSecurity
public class WebSecurityEnablerConfiguration {

}
```

该配置类会在 `SpringBootWebSecurityConfiguration` 注入 Spring IoC 容器后启用 `@EnableWebSecurity` 注解。也就是说 `WebSecurityEnablerConfiguration` 目的仅仅就是在某些条件下激活 `@EnableWebSecurity` 注解。那么这个注解都有什么呢?

3. @EnableWebSecurity 注解

```
@Retention(value = java.lang.annotation.RetentionPolicy.RUNTIME)
@Target(value = { java.lang.annotation.ElementType.TYPE })
@Documented
@Import({ WebSecurityConfiguration.class,
      SpringWebMvcImportSelector.class,
      OAuth2ImportSelector.class })
@EnableGlobalAuthentication
@Configuration
public @interface EnableWebSecurity {

  /**
   * Controls debugging support for Spring Security. Default is false.
```

```
13          * @return if true, enables debug support with Spring Security
14          */
15         boolean debug() default false;
16     }
```

`@Enable*` 这类注解都是带配置导入的注解。通过导入一些配置来启用一些特定功能。`@EnableWebSecurity` 导入了 `WebSecurityConfiguration` 、 `SpringWebMvcImportSelector` 、 `OAuth2ImportSelector` 以及启用了 `@EnableGlobalAuthentication` 注解。

3.1 WebSecurityConfiguration

该配置类 `WebSecurityConfiguration` 使用一个 `WebSecurity` 对象基于用户指定的或者默认的安全配置，你可以通过继承 `WebSecurityConfigurerAdapter` 或者实现 `WebSecurityConfigurer` 来定制 `WebSecurity` 创建一个 `FilterChainProxy` Bean来对用户请求进行安全过滤。这个 `FilterChainProxy` 的名称就是 `WebSecurityEnablerConfiguration` 上的 `BeanIds.SPRING_SECURITY_FILTER_CHAIN` 也就是 `springSecurityFilterChain` ,它是一个Filter，最终会被作为Servlet过滤器链中的一个Filter应用到Servlet容器中。安全处理的策略主要是过滤器的调用顺序。 `WebSecurityConfiguration` 最终会通过 `@EnableWebSecurity` 应用到系统。

源码分析：

```
1   package org.springframework.security.config.annotation.web.configuration;
2
3   import java.util.Collections;
4   import java.util.List;
5   import java.util.Map;
6
7   import javax.servlet.Filter;
8
9   import org.springframework.beans.factory.BeanClassLoaderAware;
10  import org.springframework.beans.factory.annotation.Autowired;
11  import org.springframework.beans.factory.annotation.Value;
12  import
    org.springframework.beans.factory.config.ConfigurableListableBeanFactory;
13  import org.springframework.context.annotation.Bean;
14  import org.springframework.context.annotation.Configuration;
15  import org.springframework.context.annotation.DependsOn;
16  import org.springframework.context.annotation.ImportAware;
17  import org.springframework.core.OrderComparator;
18  import org.springframework.core.Ordered;
19  import org.springframework.core.annotation.AnnotationAttributes;
20  import org.springframework.core.annotation.AnnotationUtils;
21  import org.springframework.core.annotation.Order;
22  import org.springframework.core.type.AnnotationMetadata;
23  import
    org.springframework.security.access.expression.SecurityExpressionHandler;
24  import org.springframework.security.config.annotation.ObjectPostProcessor;
25  import org.springframework.security.config.annotation.SecurityConfigurer;
26  import
    org.springframework.security.config.annotation.web.WebSecurityConfigurer;
27  import
    org.springframework.security.config.annotation.web.builders.WebSecurity;
28  import org.springframework.security.context.DelegatingApplicationListener;
29  import org.springframework.security.web.FilterChainProxy;
30  import org.springframework.security.web.FilterInvocation;
```

```
31    import
      org.springframework.security.web.access.WebInvocationPrivilegeEvaluator;
32    import
      org.springframework.security.web.context.AbstractSecurityWebApplicationInitializ
      er;
33
34
35    /**
36     * Spring Web Security 的配置类 :
37     *  1. 使用一个 WebSecurity 对象基于安全配置创建一个 FilterChainProxy 对象来对用户请求
      进行安全过滤。
38     *  2. 也会暴露诸如 安全SpEL表达式处理器 SecurityExpressionHandler 等一些类。
39     *
40     * @see EnableWebSecurity
41     * @see WebSecurity
42     *
43     * @author Rob Winch
44     * @author Keesun Baik
45     * @since 3.2
46     */
47    @Configuration
48    public class WebSecurityConfiguration implements ImportAware,
      BeanClassLoaderAware {
49        private WebSecurity webSecurity;
50       // 是否启用了调试模式，来自注解 @EnableWebSecurity 的属性 debug，缺省值 false
51        private Boolean debugEnabled;
52
53        private List<SecurityConfigurer<Filter, WebSecurity>>
      webSecurityConfigurers;
54
55        private ClassLoader beanClassLoader;
56
57        @Autowired(required = false)
58        private ObjectPostProcessor<Object> objectObjectPostProcessor;
59        /**
60         *
61         * 代理监听器 应该时监听 DefaultAuthenticationEventPublisher 的一些处理策略
62         */
63        @Bean
64        public static DelegatingApplicationListener delegatingApplicationListener()
      {
65            return new DelegatingApplicationListener();
66        }
67         /**
68          *
69          * 安全SpEL表达式处理器 SecurityExpressionHandler 缺省为一个
      DefaultWebSecurityExpressionHandler
70          */
71        @Bean
72        @DependsOn(AbstractSecurityWebApplicationInitializer.DEFAULT_FILTER_NAME)
73        public SecurityExpressionHandler<FilterInvocation>
      webSecurityExpressionHandler() {
74            return webSecurity.getExpressionHandler();
75        }
76
77        /**
78         *  Spring Security 核心过滤器  Spring Security Filter Chain  , Bean ID 为
      springSecurityFilterChain
```

```
79          * @return the {@link Filter} that represents the security filter chain
80          * @throws Exception
81          */
82         @Bean(name = AbstractSecurityWebApplicationInitializer.DEFAULT_FILTER_NAME)
83         public Filter springSecurityFilterChain() throws Exception {
84             boolean hasConfigurers = webSecurityConfigurers != null
85                     && !webSecurityConfigurers.isEmpty();
86             if (!hasConfigurers) {
87                 WebSecurityConfigurerAdapter adapter = objectObjectPostProcessor
88                         .postProcess(new WebSecurityConfigurerAdapter() {
89                         });
90                 webSecurity.apply(adapter);
91             }
92             return webSecurity.build();
93         }
94 
95         /**
96          *
97          * 用于模板 如JSP Freemarker 的一些页面标签按钮控制支持
98          * Creates the {@link WebInvocationPrivilegeEvaluator} that is necessary for
    the JSP
99          * tag support.
100         * @return the {@link WebInvocationPrivilegeEvaluator}
101         * @throws Exception
102         */
103        @Bean
104        @DependsOn(AbstractSecurityWebApplicationInitializer.DEFAULT_FILTER_NAME)
105        public WebInvocationPrivilegeEvaluator privilegeEvaluator() throws Exception
    {
106            return webSecurity.getPrivilegeEvaluator();
107        }
108 
109        /**
110         *
111         * 用于创建web configuration的SecurityConfigurer实例，
112         * 注意该参数通过@Value(...)方式注入，对应的bean
    autowiredWebSecurityConfigurersIgnoreParents
113         * 也在该类中定义
114         *
115         * @param objectPostProcessor the {@link ObjectPostProcessor} used to create
    a
116         * {@link WebSecurity} instance
117         * @param webSecurityConfigurers the
118         * {@code <SecurityConfigurer<FilterChainProxy, WebSecurityBuilder>}
    instances used to
119         * create the web configuration
120         * @throws Exception
121         */
122        @Autowired(required = false)
123        public void setFilterChainProxySecurityConfigurer(
124                ObjectPostProcessor<Object> objectPostProcessor,
125                @Value("#
    {@autowiredWebSecurityConfigurersIgnoreParents.getWebSecurityConfigurers()}")
    List<SecurityConfigurer<Filter, WebSecurity>> webSecurityConfigurers)
126                throws Exception {
127            webSecurity = objectPostProcessor
128                    .postProcess(new WebSecurity(objectPostProcessor));
129            if (debugEnabled != null) {
```

```
130             webSecurity.debug(debugEnabled);
131         }
132 
133         Collections.sort(webSecurityConfigurers,
    AnnotationAwareOrderComparator.INSTANCE);
134 
135         Integer previousOrder = null;
136         Object previousConfig = null;
137         for (SecurityConfigurer<Filter, WebSecurity> config :
    webSecurityConfigurers) {
138             Integer order = AnnotationAwareOrderComparator.lookupOrder(config);
139             if (previousOrder != null && previousOrder.equals(order)) {
140                 throw new IllegalStateException(
141                         "@Order on WebSecurityConfigurers must be unique. Order
    of "
142                                 + order + " was already used on " +
    previousConfig + ", so it cannot be used on "
143                                 + config + " too.");
144             }
145             previousOrder = order;
146             previousConfig = config;
147         }
148         for (SecurityConfigurer<Filter, WebSecurity> webSecurityConfigurer :
    webSecurityConfigurers) {
149             webSecurity.apply(webSecurityConfigurer);
150         }
151         this.webSecurityConfigurers = webSecurityConfigurers;
152     }
153      /**
154       * 从当前bean容器中获取所有的WebSecurityConfigurer bean。
155       * 这些WebSecurityConfigurer通常是由开发人员实现的配置类，并且继承自
    WebSecurityConfigurerAdapter
156       *
157       */
158     @Bean
159     public static AutowiredWebSecurityConfigurersIgnoreParents
    autowiredWebSecurityConfigurersIgnoreParents(
160             ConfigurableListableBeanFactory beanFactory) {
161         return new AutowiredWebSecurityConfigurersIgnoreParents(beanFactory);
162     }
163 
164     /**
165      * A custom verision of the Spring provided AnnotationAwareOrderComparator
    that uses
166      * {@link AnnotationUtils#findAnnotation(Class, Class)} to look on super
    class
167      * instances for the {@link Order} annotation.
168      *
169      * @author Rob Winch
170      * @since 3.2
171      */
172     private static class AnnotationAwareOrderComparator extends OrderComparator
    {
173         private static final AnnotationAwareOrderComparator INSTANCE = new
    AnnotationAwareOrderComparator();
174 
175         @Override
176         protected int getOrder(Object obj) {
```

```
177             return lookupOrder(obj);
178         }
179 
180         private static int lookupOrder(Object obj) {
181             if (obj instanceof Ordered) {
182                 return ((Ordered) obj).getOrder();
183             }
184             if (obj != null) {
185                 Class<?> clazz = (obj instanceof Class ? (Class<?>) obj :
    obj.getClass());
186                 Order order = AnnotationUtils.findAnnotation(clazz,
    Order.class);
187                 if (order != null) {
188                     return order.value();
189                 }
190             }
191             return Ordered.LOWEST_PRECEDENCE;
192         }
193     }
194 
195     /*
196      * 要是为了获取注解 @EnableWebSecurity 的属性 debugEnabled
197      *
198      * @see
    org.springframework.context.annotation.ImportAware#setImportMetadata(org.
199      * springframework.core.type.AnnotationMetadata)
200      */
201     public void setImportMetadata(AnnotationMetadata importMetadata) {
202         Map<String, Object> enableWebSecurityAttrMap = importMetadata
203                 .getAnnotationAttributes(EnableWebSecurity.class.getName());
204         AnnotationAttributes enableWebSecurityAttrs = AnnotationAttributes
205                 .fromMap(enableWebSecurityAttrMap);
206         debugEnabled = enableWebSecurityAttrs.getBoolean("debug");
207         if (webSecurity != null) {
208             webSecurity.debug(debugEnabled);
209         }
210     }
211 
212     /*
213      * (non-Javadoc)
214      *
215      * @see
216      *
    org.springframework.beans.factory.BeanClassLoaderAware#setBeanClassLoader(java.
217      * lang.ClassLoader)
218      */
219     public void setBeanClassLoader(ClassLoader classLoader) {
220         this.beanClassLoader = classLoader;
221     }
222   }
```

3.2 SpringWebMvcImportSelector

该类是为了对 Spring Mvc 进行支持的。一旦发现应用使用 Spring Mvc 的核心前置控制器 `DispatcherServlet` 就会引入 `WebMvcSecurityConfiguration` 。主要是为了适配 Spring Mvc 。

3.3 OAuth2ImportSelector

该类是为了对 `OAuth2.0` 开放授权协议进行支持。 `ClientRegistration` 如果被引用，具体点也就是 `spring-security-oauth2` 模块被启用（引入依赖jar）时。会启用 `OAuth2` 客户端配置 `OAuth2ClientConfiguration` 。

3.4 @EnableGlobalAuthentication

这个类主要引入了 `AuthenticationConfiguration` 目的主要为了构造 认证管理器 `AuthenticationManager` 。 `AuthenticationManager` 十分重要后面我们会进行专门的分析。

4. SecurityFilterAutoConfiguration

我们在 `org.springframework.boot.autoconfigure.security.servlet` 路径下还发现了一个配置类 `SecurityFilterAutoConfiguration` 。该类用于向Servlet容器注册一个名称为 `securityFilterChainRegistration` 的bean, 实现类是 `DelegatingFilterProxyRegistrationBean` 。该 bean 的目的是注册另外一个 `Servlet Filter Bean` 到 `Servlet` 容器,实现类为 `DelegatingFilterProxy` 。 `DelegatingFilterProxy` 其实是一个代理过滤器，它被 `Servlet` 容器用于处理请求时，会将任务委托给指定给自己另外一个Filter bean。对于 `SecurityFilterAutoConfiguration` ,来讲，这个被代理的Filter bean的名字为 `springSecurityFilterChain` , 也就是我们上面提到过的 Spring Security Web提供的用于请求安全处理的Filter bean，其实现类是 `FilterChainProxy` 。

相关的源码分析：

```
1   package org.springframework.boot.autoconfigure.security.servlet;
2
3   import java.util.EnumSet;
4   import java.util.stream.Collectors;
5
6   import javax.servlet.DispatcherType;
7
8   import org.springframework.boot.autoconfigure.AutoConfigureAfter;
9   import org.springframework.boot.autoconfigure.EnableAutoConfiguration;
10  import org.springframework.boot.autoconfigure.condition.ConditionalOnBean;
11  import org.springframework.boot.autoconfigure.condition.ConditionalOnClass;
12  import
    org.springframework.boot.autoconfigure.condition.ConditionalOnWebApplication;
13  import
    org.springframework.boot.autoconfigure.condition.ConditionalOnWebApplication.Type
    ;
14  import org.springframework.boot.autoconfigure.security.SecurityProperties;
15  import
    org.springframework.boot.context.properties.EnableConfigurationProperties;
16  import
    org.springframework.boot.web.servlet.DelegatingFilterProxyRegistrationBean;
17  import org.springframework.context.annotation.Bean;
18  import org.springframework.context.annotation.Configuration;
```

```java
  import org.springframework.security.config.annotation.web.configuration.WebSecurityConfiguration;
  import org.springframework.security.config.http.SessionCreationPolicy;
  import org.springframework.security.web.context.AbstractSecurityWebApplicationInitializer;

  @Configuration
  // 仅在 Servlet 环境下生效
  @ConditionalOnWebApplication(type = Type.SERVLET)
  // 确保安全属性配置信息被加载并以bean形式被注册到容器
  @EnableConfigurationProperties(SecurityProperties.class)
  // 仅在特定类存在于 classpath 上时才生效
  @ConditionalOnClass({ AbstractSecurityWebApplicationInitializer.class,
        SessionCreationPolicy.class })
  // 指定该配置类在  SecurityAutoConfiguration 配置类应用之后应用
  @AutoConfigureAfter(SecurityAutoConfiguration.class)
  public class SecurityFilterAutoConfiguration {

      // 要注册到 Servlet 容器的 DelegatingFilterProxy Filter的
      // 目标代理Filter bean的名称 : springSecurityFilterChain
    private static final String DEFAULT_FILTER_NAME =
            AbstractSecurityWebApplicationInitializer.DEFAULT_FILTER_NAME;


      // 定义一个 bean securityFilterChainRegistration,
      // 该 bean 的目的是注册另外一个 bean 到 Servlet 容器 : 实现类为 DelegatingFilterProxy 的一个 Servlet Filter
      // 该 DelegatingFilterProxy Filter 其实是一个代理过滤器，它被 Servlet 容器用于匹配特定URL模式的请求，
      // 而它会将任务委托给指定给自己的名字为 springSecurityFilterChain 的 Filter, 也就是 Spring Security Web
      // 提供的用于请求安全处理的一个 Filter bean，其实现类是 FilterChainProxy
      // (可以将 1 个 FilterChainProxy 理解为 1 HttpFirewall + n SecurityFilterChain)
    @Bean
    @ConditionalOnBean(name = DEFAULT_FILTER_NAME)
    public DelegatingFilterProxyRegistrationBean securityFilterChainRegistration(
            SecurityProperties securityProperties) {
        DelegatingFilterProxyRegistrationBean registration = new DelegatingFilterProxyRegistrationBean(
                DEFAULT_FILTER_NAME);
        registration.setOrder(securityProperties.getFilter().getOrder());
        registration.setDispatcherTypes(getDispatcherTypes(securityProperties));
        return registration;
    }

    private EnumSet<DispatcherType> getDispatcherTypes(
            SecurityProperties securityProperties) {
        if (securityProperties.getFilter().getDispatcherTypes() == null) {
            return null;
        }
        return securityProperties.getFilter().getDispatcherTypes().stream()
                .map((type) -> DispatcherType.valueOf(type.name())).collect(Collectors
                        .collectingAndThen(Collectors.toSet(), EnumSet::copyOf));
    }
```

```
67
68      }
69
```

5. 总结

本文主要对 Spring Security 在 Spring Boot 中的自动配置一些机制进行了粗略的讲解。为什么没有细讲。因为从学习出发有些东西不是我们必须要深入了解的，但是又要知道一点点相关的知识。我们先宏观上有个大致的了解就行。**所以在阅读本文一定不要钻牛角尖。粗略知道配置策略、加载策略和一些关键类的作用即可。在你对 Spring Security 有了进一步学习之后，回头认真来看这些配置类会有更深层的思考。** 从另一个方面该文也给你阅读 Spring 源码提供了一些思路，学会这些才是最重要的。

URI中的 Ant 风格

1. 前言

我们经常在读到一些文章会遇到`uri`支持`Ant`风格，而且这个东西在 Spring MVC 和 Spring Security 中经常被提及。这到底是什么呢？今天我们来学习了解一下。这对我们学习 Spring MVC 和 Spring Security 十分必要。

2. Ant 风格

说白了`Ant`风格就是一种路径匹配表达式。主要用来对`uri`的匹配。其实跟正则表达式作用是一样的，只不过正则表达式适用面更加宽泛，`Ant`仅仅用于路径匹配。

3. Ant 通配符

`Ant`中的通配符有三种：

- `?` 匹配任何单字符
- `*` 匹配0或者任意数量的 **字符**
- `**` 匹配0或者更多的 **目录**

这里注意了单个`*`是在一个目录内进行匹配。 而`**`是可以匹配多个目录，一定不要迷糊。

3.1 Ant 通配符示例

通配符	示例	说明
`?`	`/ant/p?ttern`	匹配项目根路径下 `/ant/pattern` 和 `/ant/pXttern`,**但是不包括** `/ant/pttern`
`*`	`/ant/*.html`	匹配项目根路径下所有在 `ant` 路径下的 `.html` 文件
`*`	`/ant/*/path`	`/ant/path`、`/ant/a/path`、`/ant/bxx/path` 都匹配，不匹配 `/ant/axx/bxx/path`
`**`	`/ant/**/path`	`/ant/path`、`/ant/a/path`、`/ant/bxx/path`、`/ant/axx/bxx/path` 都匹配

3.2 最长匹配原则

从 3.1 可以看出 `*` 和 `**` 是有冲突的情况存在的。为了解决这种冲突就规定了最长匹配原则(has more characters)。 一旦一个 `uri` 同时符合两个 `Ant` 匹配那么走匹配规则字符最多的。为什么走最长？因为字符越长信息越多就越具体。比如 `/ant/a/path` 同时满足 `/**/path` 和 `/ant/*/path` 那么走 `/ant/*/path`

4. Spring MVC 和 Spring Security 中的 Ant 风格

接下来我们来看看 Spring MVC 和 Spring Security 下的 `Ant` 风格。

4.1 Spring MVC 中的 Ant 风格

这里也提一下在 Spring MVC 中 我们在控制器中写如下接口：

```
        /**
         * ant style test.
         *
         * @return the string
         */
        @GetMapping("/?ant")
        public String ant() {

            return "ant";
        }
```

你使用任意合法 `uri` 字符替代 `?` 发现都可以匹配，比如 `/bant` 。 还有Spring MVC 的一些 过滤器注册、格式化器注册都用到了 `Ant` 风格。

4.2 Spring Security 中的 Ant 风格

在 Spring Security 中 `WebSecurityConfigurerAdapter` 中的你可以通过如下配置进行路由权限访问控制：

```
    public class SecurityConfig extends WebSecurityConfigurerAdapter {

        @Autowired
        public void configureGlobal(AuthenticationManagerBuilder
authenticationManagerBuilder) throws Exception {
```

```
5
    authenticationManagerBuilder.inMemoryAuthentication().withUser("admin").password(
    "admin").roles("USER");
6           }
7
8           @Override
9           protected void configure(HttpSecurity http) throws Exception {
10              http.authorizeRequests()
11                      //放行静态资源 首页
12                      .antMatchers("/index.html","/static/**").permitAll()
13                      .anyRequest().authenticated();
14          }
15      }
16
```

上面 Spring Security 的配置中在 `antMatchers` 方法中通过 `Ant` 通配符来控制了资源的访问权限。这是接口权限控制的重要组成部分。

5. 总结

`Ant` 风格整体东西不多,也很好理解。 很多关于`uri` 的配置、路由匹配、处理都用到了 `Ant` 风格 。对于 Web 开发人员来说是必须掌握的技能之一。敬请关注公众号：**码农小胖哥** 和个人博客： https://felord.cn

自定义配置类WebSecurityConfigurerAdapter

1. 前言

今天我们要进一步的的学习如何自定义配置 Spring Security 我们已经多次提到了 `WebSecurityConfigurerAdapter` ，而且我们知道 Spring Boot 中的自动配置实际上是通过自动配置包下的 `SecurityAutoConfiguration` 总配置类上导入的 Spring Boot Web 安全配置类 `SpringBootWebSecurityConfiguration` 来配置的。所以我们就拿它开刀。如果还是一头雾水建议通过 https://felord.cn 查看 Spring Security **实战** 。

2. 自定义 Spring Boot Web 安全配置类

我们使用我们最擅长的 `Ctrl + C` 、 `Ctrl + V` 抄源码中的 `SpringBootWebSecurityConfiguration` ，命名为我们自定义的 `CustomSpringBootWebSecurityConfiguration` ：

```
1       @Configuration
2       @ConditionalOnClass(WebSecurityConfigurerAdapter.class)
3       @ConditionalOnWebApplication(type = ConditionalOnWebApplication.Type.SERVLET)
4       public class CustomSpringBootWebSecurityConfiguration {
5
6           @Configuration
7           @Order(SecurityProperties.BASIC_AUTH_ORDER)
8           static class DefaultConfigurerAdapter extends WebSecurityConfigurerAdapter
    {
```

```
9               @Override
10              protected void configure(AuthenticationManagerBuilder auth) throws
    Exception {
11                  super.configure(auth);
12              }
13
14              @Override
15              public void configure(WebSecurity web) throws Exception {
16                  super.configure(web);
17              }
18
19              @Override
20              protected void configure(HttpSecurity http) throws Exception {
21                  super.configure(http);
22              }
23          }
24      }
```

相信已经有人注意到了上面 `DefaultConfigurerAdapter` 中我覆写（`@Override`）了三个方法，我们一般会通过自定义配置这三个方法来自定义我们的安全访问策略。

2.1 认证管理器配置方法

`void configure(AuthenticationManagerBuilder auth)` 用来配置认证管理器 `AuthenticationManager`。说白了就是所有 `UserDetails` 相关的它都管，包含 `PasswordEncoder` 密码机。如果你不清楚可以通过 **Spring Security 中的 UserDetail** 进行了解。本文对 `AuthenticationManager` 不做具体分析讲解，后面会有专门的文章来讲这个东西。 可通过 **Spring Security 实战系列** 进行学习。

2.2 核心过滤器配置方法

`void configure(WebSecurity web)` 用来配置 `WebSecurity`。而 `WebSecurity` 是基于 `Servlet Filter` 用来配置 `springSecurityFilterChain`。而 `springSecurityFilterChain` 又被委托给了 **Spring Security 核心过滤器** Bean `DelegatingFilterProxy`。 相关逻辑你可以在 `WebSecurityConfiguration` 中找到。我们一般不会过多来自定义 `WebSecurity`，使用较多的使其 `ignoring()` 方法用来忽略 **Spring Security** 对静态资源的控制。

2.3 安全过滤器链配置方法

`void configure(HttpSecurity http)` 这个是我们使用最多的，用来配置 `HttpSecurity`。`HttpSecurity` 用于构建一个安全过滤器链 `SecurityFilterChain`。 `SecurityFilterChain` 最终被注入**核心过滤器**。 `HttpSecurity` 有许多我们需要的配置。我们可以通过它来进行自定义安全访问策略。所以我们单独开一章来讲解这个东西。

3. HttpSecurity 配置

`HttpSecurity` 是后面几篇文章的重点，我们将实际操作它来实现一些实用功能。所以本文要着重介绍它。

3.1 默认配置

```
    protected void configure(HttpSecurity http) throws Exception {
        logger.debug("Using default configure(HttpSecurity). If subclassed this will potentially override subclass configure(HttpSecurity).");

        http
            .authorizeRequests()
                .anyRequest().authenticated()
                .and()
            .formLogin().and()
            .httpBasic();
    }
```

上面是 Spring Security 在 Spring Boot 中的默认配置。通过以上的配置，你的应用具备了一下的功能：

- 所有的请求访问都需要被授权。
- 使用 `form` 表单进行登陆(默认路径为 `/login`)，也就是前几篇我们见到的登录页。
- 防止 `CSRF` 攻击、 `XSS` 攻击。
- 启用 `HTTP Basic` 认证

3.2 常用方法解读

`HttpSecurity` 使用了 `builder` 的构建方式来灵活制定访问策略。最早基于 `XML` 标签对 `HttpSecurity` 进行配置。现在大部分使用 `javaConfig` 方式。常用的方法解读如下：

方法	说明
openidLogin()	用于基于 OpenId 的验证
headers()	将安全标头添加到响应,比如说简单的 XSS 保护
cors()	配置跨域资源共享（ CORS ）
sessionManagement()	允许配置会话管理
portMapper()	允许配置一个PortMapper(HttpSecurity#(getSharedObject(class)))，其他提供SecurityConfigurer的对象使用 PortMapper 从 HTTP 重定向到 HTTPS 或者从 HTTPS 重定向到 HTTP。默认情况下，Spring Security使用一个PortMapperImpl映射 HTTP 端口8080到 HTTPS 端口8443，HTTP 端口80到 HTTPS 端口443
jee()	配置基于容器的预认证。 在这种情况下，认证由Servlet容器管理
x509()	配置基于x509的认证
rememberMe	允许配置“记住我”的验证
authorizeRequests()	允许基于使用HttpServletRequest限制访问
requestCache()	允许配置请求缓存
exceptionHandling()	允许配置错误处理
securityContext()	在HttpServletRequests之间的SecurityContextHolder上设置SecurityContext的管理。 当使用WebSecurityConfigurerAdapter时，这将自动应用
servletApi()	将HttpServletRequest方法与在其上找到的值集成到SecurityContext中。 当使用WebSecurityConfigurerAdapter时，这将自动应用
csrf()	添加 CSRF 支持，使用WebSecurityConfigurerAdapter时，默认启用
logout()	添加退出登录支持。当使用WebSecurityConfigurerAdapter时，这将自动应用。默认情况是，访问URL”/ logout”，使HTTP Session无效来清除用户，清除已配置的任何#rememberMe()身份验证，清除SecurityContextHolder，然后重定向到”/login?success”
anonymous()	允许配置匿名用户的表示方法。 当与WebSecurityConfigurerAdapter结合使用时，这将自动应用。 默认情况下，匿名用户将使用org.springframework.security.authentication.AnonymousAuthenticationToken表示，并包含角色 “ROLE_ANONYMOUS”
formLogin()	指定支持基于表单的身份验证。如果未指定FormLoginConfigurer#loginPage(String)，则将生成默认登录页面
oauth2Login()	根据外部OAuth 2.0或OpenID Connect 1.0提供程序配置身份验证
requiresChannel()	配置通道安全。为了使该配置有用，必须提供至少一个到所需信道的映射
httpBasic()	配置 Http Basic 验证
addFilterBefore()	在指定的Filter类之前添加过滤器
addFilterAt()	在指定的Filter类的位置添加过滤器
addFilterAfter()	在指定的Filter类的之后添加过滤器
and()	连接以上策略的连接器，用来组合安全策略。实际上就是"而且"的意思

4. 总结

到今天为止，我们已经由浅入深学习了很多关于 Spring Security 的知识。已经具有开始自定义来实现一些实用的功能了，在后面的文章开始我们将结合实际开发场景进行一些实战操作。敬请关注公众号：**码农小胖哥** 以第一时间获取相关教程。

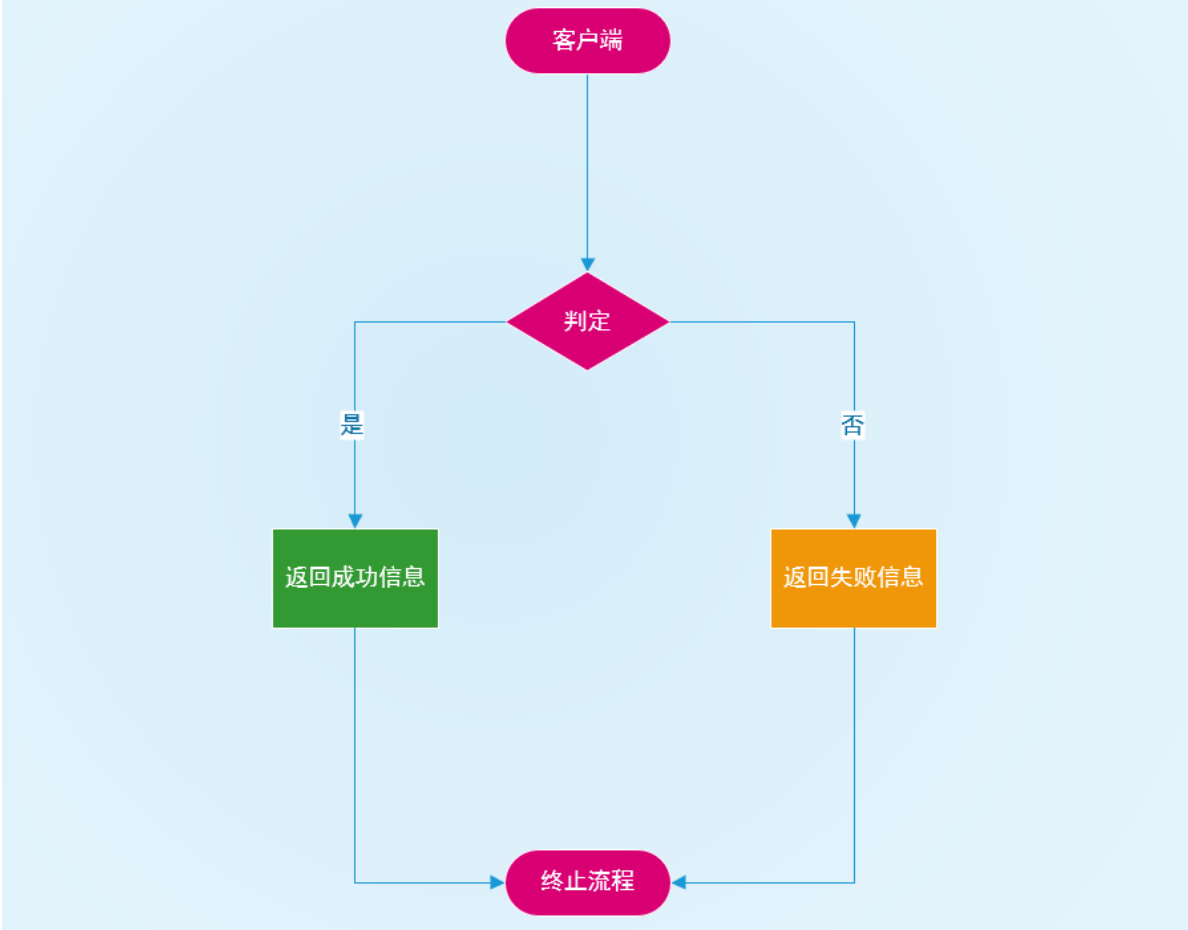
实现自定义登录

1. 前言

前面的关于 Spring Security 相关的文章只是一个预热。为了接下来更好的实战，如果你错过了请从 Spring Security **实战系列** 开始。安全访问的第一步就是认证（`Authentication`），认证的第一步就是登录。今天我们要通过对 Spring Security 的自定义，来设计一个可扩展，可伸缩的 form 登录功能。

2. form 登录的流程

下面是 form 登录的基本流程：



只要是 form 登录基本都能转化为上面的流程。接下来我们看看 Spring Security 是如何处理的。

3. Spring Security 中的登录

昨天 Spring Security 实战干货：自定义配置类入口WebSecurityConfigurerAdapter 中已经讲到了我们通常的自定义访问控制主要是通过 `HttpSecurity` 来构建的。默认它提供了三种登录方式：

- `formLogin()` 普通表单登录
- `oauth2Login()` 基于 `OAuth2.0` 认证/授权协议
- `openidLogin()` 基于 `OpenID` 身份认证规范

以上三种方式统统是 `AbstractAuthenticationFilterConfigurer` 实现的，

4. HttpSecurity 中的 form 表单登录

启用表单登录通过两种方式一种是通过 `HttpSecurity` 的 `apply(C configurer)` 方法自己构造一个 `AbstractAuthenticationFilterConfigurer` 的实现，这种是比较高级的玩法。 另一种是我们常见的使用 `HttpSecurity` 的 `formLogin()` 方法来自定义 `FormLoginConfigurer` 。我们先搞一下比较常规的第二种。

4.1 FormLoginConfigurer

该类是 form 表单登录的配置类。它提供了一些我们常用的配置方法：

- `loginPage(String loginPage)` ：登录 **页面**而并不是接口，对于前后分离模式需要我们进行改造 默认为 `/login` 。
- `loginProcessingUrl(String loginProcessingUrl)` 实际表单向后台提交用户信息的 `Action` ，再由过滤器 `UsernamePasswordAuthenticationFilter` 拦截处理，该 `Action` 其实不会处理任何逻辑。
- `usernameParameter(String usernameParameter)` 用来自定义用户参数名，默认 `username` 。
- `passwordParameter(String passwordParameter)` 用来自定义用户密码名，默认 `password`
- `failureUrl(String authenticationFailureUrl)` 登录失败后会重定向到此路径， 一般前后分离不会使用它。
- `failureForwardUrl(String forwardUrl)` 登录失败会转发到此， 一般前后分离用到它。 可定义一个 `Controller` （控制器）来处理返回值,但是要注意 `RequestMethod` 。
- `defaultSuccessUrl(String defaultSuccessUrl, boolean alwaysUse)` 默认登陆成功后跳转到此，如果 `alwaysUse` 为 `true` 只要进行认证流程而且成功，会一直跳转到此。一般推荐默认值 `false`
- `successForwardUrl(String forwardUrl)` 效果等同于上面 `defaultSuccessUrl` 的 `alwaysUse` 为 `true` 但是要注意 `RequestMethod` 。
- `successHandler(AuthenticationSuccessHandler successHandler)` 自定义认证成功处理器，可替代上面所有的 `success` 方式
- `failureHandler(AuthenticationFailureHandler authenticationFailureHandler)` 自定义失败处理器，可替代上面所有的 `failure` 方式
- `permitAll(boolean permitAll)` form 表单登录是否放开

知道了这些我们就能来搞个定制化的登录了。

5. Spring Security 聚合登录 实战

接下来是我们最激动人心的实战登录操作。 有疑问的可认真阅读 Spring 实战 的一系列预热文章。

5.1 简单需求

我们的接口访问都要通过认证，登陆错误后返回错误信息（json），成功后前台可以获取到对应数据库用户信息（json）（**实战中记得脱敏**）。

我们定义处理成功失败的控制器：

```
@RestController
@RequestMapping("/login")
public class LoginController {
    @Resource
    private SysUserService sysUserService;

    /**
     * 登录失败返回 401 以及提示信息.
     *
     * @return the rest
     */
    @PostMapping("/failure")
    public Rest loginFailure() {

        return RestBody.failure(HttpStatus.UNAUTHORIZED.value(), "登录失败了，老哥");
    }

    /**
     * 登录成功后拿到个人信息.
     *
     * @return the rest
     */
    @PostMapping("/success")
    public Rest loginSuccess() {
          // 登录成功后用户的认证信息 UserDetails会存在 安全上下文寄存器 SecurityContextHolder 中
          User principal = (User) SecurityContextHolder.getContext().getAuthentication().getPrincipal();
          String username = principal.getUsername();
          SysUser sysUser = sysUserService.queryByUsername(username);
          // 脱敏
          sysUser.setEncodePassword("[PROTECT]");
          return RestBody.okData(sysUser,"登录成功");
    }
}
```

然后 我们自定义配置覆写 `void configure(HttpSecurity http)` 方法进行如下配置（这里需要禁用crsf）：

```
@Configuration
@ConditionalOnClass(WebSecurityConfigurerAdapter.class)
@ConditionalOnWebApplication(type = ConditionalOnWebApplication.Type.SERVLET)
public class CustomSpringBootWebSecurityConfiguration {

    @Configuration
    @Order(SecurityProperties.BASIC_AUTH_ORDER)
    static class DefaultConfigurerAdapter extends WebSecurityConfigurerAdapter {
        @Override
```

```
10              protected void configure(AuthenticationManagerBuilder auth) throws
    Exception {
11                  super.configure(auth);
12              }
13
14              @Override
15              public void configure(WebSecurity web) throws Exception {
16                  super.configure(web);
17              }
18
19              @Override
20              protected void configure(HttpSecurity http) throws Exception {
21                  http.csrf().disable()
22                          .cors()
23                          .and()
24                          .authorizeRequests().anyRequest().authenticated()
25                          .and()
26                          .formLogin()
27                          .loginProcessingUrl("/process")
28                          .successForwardUrl("/login/success").
29                          failureForwardUrl("/login/failure");
30
31              }
32          }
33      }
```

使用 Postman 或者其它工具进行 Post 方式的表单提交 `http://localhost:8080/process?username=Felordcn&password=12345` 会返回用户信息：

```
1       {
2           "httpStatus": 200,
3           "data": {
4               "userId": 1,
5               "username": "Felordcn",
6               "encodePassword": "[PROTECT]",
7               "age": 18
8           },
9           "msg": "登录成功",
10          "identifier": ""
11      }
```

把密码修改为其它值再次请求认证失败后：

```
1       {
2           "httpStatus": 401,
3           "data": null,
4           "msg": "登录失败了，老哥",
5           "identifier": "-9999"
6       }
```

6. 多种登录方式并存的实现

就这么完了了么？现在登录的花样繁多。常规的就有短信、邮箱、扫码，第三方是以后我要讲的不在今天范围之内。 如何应对想法多的产品经理？ 我们来搞一个可扩展各种姿势的登录方式。我们在上面 **2. form 登录的流程** 中的 **用户** 和 **判定** 之间增加一个适配器来适配即可。 我们知道这个所谓的 **判定**就是 `UsernamePasswordAuthenticationFilter` 。

我们只需要保证 `uri` 为上面配置的 `/process` 并且能够通过 `getParameter(String name)` 获取用户名和密码即可 。

我突然觉得可以模仿 `DelegatingPasswordEncoder` 的搞法， 维护一个注册表执行不同的处理策略。当然我们要实现一个 `GenericFilterBean` 在 `UsernamePasswordAuthenticationFilter` 之前执行。同时制定登录的策略。

6.1 登录方式定义

定义登录方式枚举 ``。

```

    public enum LoginTypeEnum {

        /**
         * 原始登录方式.
         */
        FORM,
        /**
         * Json 提交.
         */
        JSON,
        /**
         * 验证码.
         */
        CAPTCHA

    }
```

6.2 定义前置处理器接口

定义前置处理器接口用来处理接收的各种特色的登录参数 并处理具体的逻辑。这个借口其实有点随意，重要的是你要学会思路。我实现了一个 默认的 `form' 表单登录 和 通过` RequestBody `放入` json` 的两种方式，篇幅限制这里就不展示了。具体的 **DEMO** 参见底部。

```
    public interface LoginPostProcessor {



        /**
         * 获取 登录类型
         *
         * @return the type
         */
        LoginTypeEnum getLoginTypeEnum();

        /**
         * 获取用户名
         *
         * @param request the request
         * @return the string
```

```
         */
        String obtainUsername(ServletRequest request);

        /**
         * 获取密码
         *
         * @param request the request
         * @return the string
         */
        String obtainPassword(ServletRequest request);

    }
```

6.3 实现登录前置处理过滤器

该过滤器维护了 `LoginPostProcessor` 映射表。 通过前端来判定登录方式进行策略上的预处理，最终还是会交给 `UsernamePasswordAuthenticationFilter` 。通过 `HttpSecurity` 的 `addFilterBefore(preLoginFilter, UsernamePasswordAuthenticationFilter.class)` 方法进行前置。

```
  package cn.felord.spring.security.filter;

  import cn.felord.spring.security.enumation.LoginTypeEnum;
  import org.springframework.security.web.util.matcher.AntPathRequestMatcher;
  import org.springframework.security.web.util.matcher.RequestMatcher;
  import org.springframework.util.Assert;
  import org.springframework.util.CollectionUtils;
  import org.springframework.web.filter.GenericFilterBean;

  import javax.servlet.FilterChain;
  import javax.servlet.ServletException;
  import javax.servlet.ServletRequest;
  import javax.servlet.ServletResponse;
  import javax.servlet.http.HttpServletRequest;
  import java.io.IOException;
  import java.util.Collection;
  import java.util.HashMap;
  import java.util.Map;

  import static
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFi
lter.SPRING_SECURITY_FORM_PASSWORD_KEY;
  import static
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFi
lter.SPRING_SECURITY_FORM_USERNAME_KEY;

  /**
   * 预登录控制器
   *
   * @author Felordcn
   * @since 16 :21 2019/10/17
   */
  public class PreLoginFilter extends GenericFilterBean {


      private static final String LOGIN_TYPE_KEY = "login_type";

```

```
34
35      private RequestMatcher requiresAuthenticationRequestMatcher;
36      private Map<LoginTypeEnum, LoginPostProcessor> processors = new HashMap<>
    ();
37
38
39      public PreLoginFilter(String loginProcessingUrl,
    Collection<LoginPostProcessor> loginPostProcessors) {
40          Assert.notNull(loginProcessingUrl, "loginProcessingUrl must not be
    null");
41          requiresAuthenticationRequestMatcher = new
    AntPathRequestMatcher(loginProcessingUrl, "POST");
42          LoginPostProcessor loginPostProcessor = defaultLoginPostProcessor();
43          processors.put(loginPostProcessor.getLoginTypeEnum(),
    loginPostProcessor);
44
45          if (!CollectionUtils.isEmpty(loginPostProcessors)) {
46              loginPostProcessors.forEach(element ->
    processors.put(element.getLoginTypeEnum(), element));
47          }
48
49      }
50
51
52      private LoginTypeEnum getTypeFromReq(ServletRequest request) {
53          String parameter = request.getParameter(LOGIN_TYPE_KEY);
54
55          int i = Integer.parseInt(parameter);
56          LoginTypeEnum[] values = LoginTypeEnum.values();
57          return values[i];
58      }
59
60
61      /**
62       * 默认还是Form .
63       *
64       * @return the login post processor
65       */
66      private LoginPostProcessor defaultLoginPostProcessor() {
67          return new LoginPostProcessor() {
68
69
70              @Override
71              public LoginTypeEnum getLoginTypeEnum() {
72
73                  return LoginTypeEnum.FORM;
74              }
75
76              @Override
77              public String obtainUsername(ServletRequest request) {
78                  return
    request.getParameter(SPRING_SECURITY_FORM_USERNAME_KEY);
79              }
80
81              @Override
82              public String obtainPassword(ServletRequest request) {
83                  return
    request.getParameter(SPRING_SECURITY_FORM_PASSWORD_KEY);
```

```
                }
            };
        }


        @Override
        public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain) throws IOException, ServletException {
            ParameterRequestWrapper parameterRequestWrapper = new ParameterRequestWrapper((HttpServletRequest) request);
            if (requiresAuthenticationRequestMatcher.matches((HttpServletRequest) request)) {

                LoginTypeEnum typeFromReq = getTypeFromReq(request);

                LoginPostProcessor loginPostProcessor = processors.get(typeFromReq);


                String username = loginPostProcessor.obtainUsername(request);

                String password = loginPostProcessor.obtainPassword(request);



 parameterRequestWrapper.setAttribute(SPRING_SECURITY_FORM_USERNAME_KEY, username);

 parameterRequestWrapper.setAttribute(SPRING_SECURITY_FORM_PASSWORD_KEY, password);

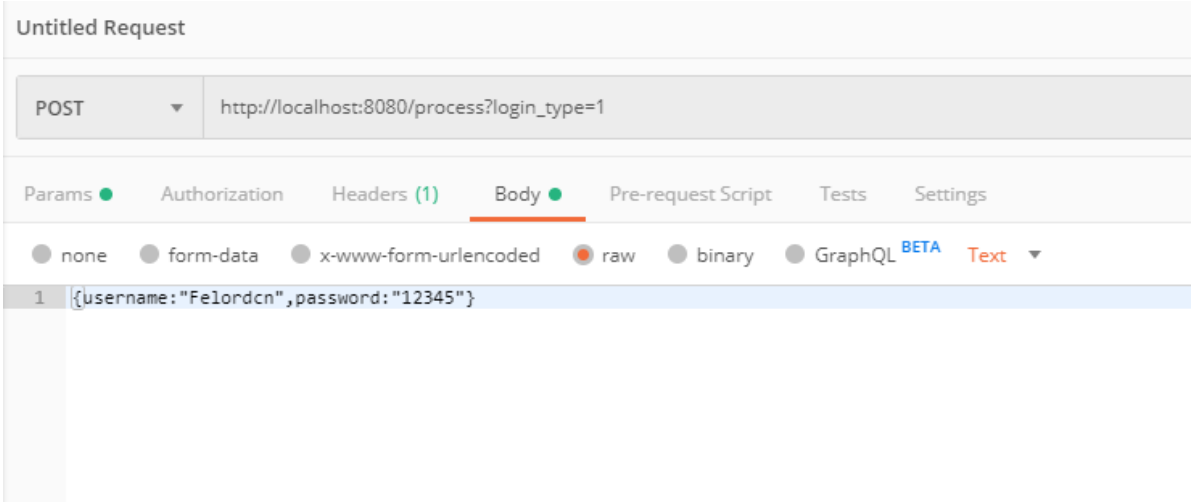
            }

            chain.doFilter(parameterRequestWrapper, response);


        }
    }
```

6.4 验证

通过 `POST` 表单提交方式 `http://localhost:8080/process?username=Felordcn&password=12345&login_type=0` 可以请求成功。或者以下列方式也可以提交成功：



更多的方式 只需要实现接口 `LoginPostProcessor` 注入 `PreLoginFilter`

7. 总结

今天我们通过各种技术的运用实现了从简单登录到可动态扩展的多种方式并存的实战运用，其实我们还间接实现了前后端分离的接口登录方式。相信对你来说会有不小的收获 。本次 **代码DEMO可通过关注公众号：码农小胖哥 回复 `ss03` 获取**，后面会更加精彩。

图解认证过滤器 UsernamePasswordAuthenticationFilter

1. 前言

欢迎阅读Spring Security 实战干货系列文章，在集成**Spring Security**安全框架的时候我们最先处理的可能就是根据我们项目的实际需要来定制注册登录了，尤其是**Http**登录认证。根据以前的相关文章介绍，**Http**登录认证由过滤器 `UsernamePasswordAuthenticationFilter` 进行处理。我们只有把这个过滤器搞清楚才能做一些定制化。今天我们就简单分析它的源码和工作流程。

2. UsernamePasswordAuthenticationFilter 源码分析

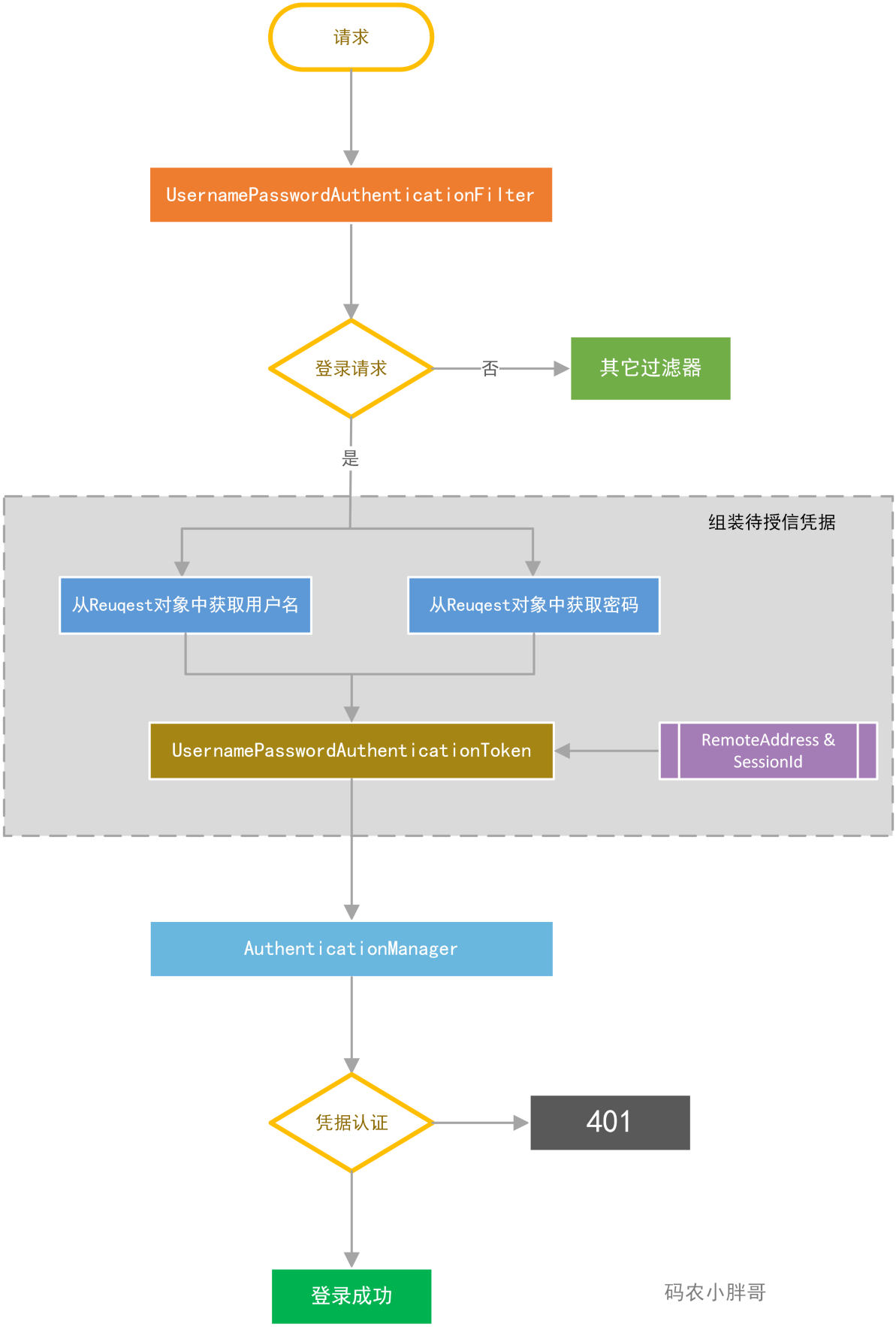
`UsernamePasswordAuthenticationFilter` 继承于 `AbstractAuthenticationProcessingFilter` （另文分析）。它的作用是拦截登录请求并获取账号和密码，然后把账号密码封装到认证凭据 `UsernamePasswordAuthenticationToken` 中，然后把凭据交给特定配置的 `AuthenticationManager` 去作认证。源码分析如下：

```
1  public class UsernamePasswordAuthenticationFilter extends
2        AbstractAuthenticationProcessingFilter {
3     // 默认取账户名、密码的key
4     public static final String SPRING_SECURITY_FORM_USERNAME_KEY = "username";
5     public static final String SPRING_SECURITY_FORM_PASSWORD_KEY = "password";
6     // 可以通过对应的set方法修改
7     private String usernameParameter = SPRING_SECURITY_FORM_USERNAME_KEY;
```

```
8       private String passwordParameter = SPRING_SECURITY_FORM_PASSWORD_KEY;
9       // 默认只支持 POST 请求
10      private boolean postOnly = true;
11
12    //  初始化一个用户密码 认证过滤器  默认的登录uri 是 /login 请求方式是POST
13    public UsernamePasswordAuthenticationFilter() {
14       super(new AntPathRequestMatcher("/login", "POST"));
15    }
16
17     // 实现其父类 AbstractAuthenticationProcessingFilter 提供的钩子方法 用去尝试认证
18    public Authentication attemptAuthentication(HttpServletRequest request,
19          HttpServletResponse response) throws AuthenticationException {
20        // 判断请求方式是否是POST
21       if (postOnly && !request.getMethod().equals("POST")) {
22          throw new AuthenticationServiceException(
23                "Authentication method not supported: " + request.getMethod());
24       }
25
26        // 先去 HttpServletRequest 对象中获取账号名、密码
27       String username = obtainUsername(request);
28       String password = obtainPassword(request);
29
30       if (username == null) {
31          username = "";
32       }
33
34       if (password == null) {
35          password = "";
36       }
37
38       username = username.trim();
39
40        // 然后把账号名、密码封装到 一个认证Token对象中，这是就是一个通行证，但是这时的状态时不可信的，一旦通过认证就变为可信的
41       UsernamePasswordAuthenticationToken authRequest = new UsernamePasswordAuthenticationToken(
42             username, password);
43
44       // 会将 HttpServletRequest 中的一些细节 request.getRemoteAddr() request.getSession 存入的到Token中
45       setDetails(request, authRequest);
46
47        // 然后 使用 父类中的 AuthenticationManager 对Token 进行认证
48       return this.getAuthenticationManager().authenticate(authRequest);
49    }
50    // 获取密码 很重要 如果你想改变获取密码的方式要么在此处重写，要么通过自定义一个前置的过滤器保证能此处能get到
51    @Nullable
52    protected String obtainPassword(HttpServletRequest request) {
53       return request.getParameter(passwordParameter);
54    }
55
56       // 获取账户很重要 如果你想改变获取密码的方式要么在此处重写，要么通过自定义一个前置的过滤器保证能此处能get到
57    @Nullable
58    protected String obtainUsername(HttpServletRequest request) {
59       return request.getParameter(usernameParameter);
60    }
```

```
61
62      // 参见上面对应的说明为凭据设置一些请求细节
63      protected void setDetails(HttpServletRequest request,
64              UsernamePasswordAuthenticationToken authRequest) {
65          authRequest.setDetails(authenticationDetailsSource.buildDetails(request));
66      }
67
68      // 设置账户参数的key
69      public void setUsernameParameter(String usernameParameter) {
70          Assert.hasText(usernameParameter, "Username parameter must not be empty or
    null");
71          this.usernameParameter = usernameParameter;
72      }
73
74      // 设置密码参数的key
75      public void setPasswordParameter(String passwordParameter) {
76          Assert.hasText(passwordParameter, "Password parameter must not be empty or
    null");
77          this.passwordParameter = passwordParameter;
78      }
79
80      // 认证的请求方式是只支持POST请求
81      public void setPostOnly(boolean postOnly) {
82          this.postOnly = postOnly;
83      }
84
85      public final String getUsernameParameter() {
86          return usernameParameter;
87      }
88
89      public final String getPasswordParameter() {
90          return passwordParameter;
91      }
92  }
```

为了加强对流程的理解，我特意画了一张图来对这个流程进行清晰的说明：



码农小胖哥

3. 我们可以定制什么

根据上面的流程，我们理解了`UsernamePasswordAuthenticationFilter`工作流程后可以做这些事情：

- 定制我们的登录请求URI和请求方式。
- 登录请求参数的格式定制化，比如可以使用JSON格式提交甚至几种并存。
- 将用户名和密码封装入凭据`UsernamePasswordAuthenticationToken`，定制业务场景需要的特殊凭据。

4. 我们会有什么疑问

`AuthenticationManager`从哪儿来，它又是什么，它是如何对凭据进行认证的，认证成功的后续细节是什么，认证失败的后续细节是什么。不要走开，持续关注：**码农小胖哥** 为你揭晓这个答案。

图解认证管理器AuthenticationManager

1. 前言

我们**上一篇**介绍了`UsernamePasswordAuthenticationFilter`的工作流程，留下了一个小小的伏笔，作为一个**Servlet Filter**应该存在一个`doFilter`实现方法，而它却没有，其实它的父类`AbstractAuthenticationProcessingFilter`提供了具体的实现。稍后我们会根据这个实现引出今天的主角`AuthenticationManager`，来继续介绍用户的认证过程。

2. AbstractAuthenticationProcessingFilter

`AbstractAuthenticationProcessingFilter`作为`UsernamePasswordAuthenticationFilter`的父类，实现了认证过滤器的处理逻辑。我们来看看它的核心方法`doFilter`的实现：

```
1   public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
2         throws IOException, ServletException {
3
4      HttpServletRequest request = (HttpServletRequest) req;
5      HttpServletResponse response = (HttpServletResponse) res;
6
7       // 先通过请求的uri来判断是否需要认证,比如默认的/login
8      if (!requiresAuthentication(request, response)) {
9         chain.doFilter(request, response);
10
11        return;
12     }
13
14     if (logger.isDebugEnabled()) {
15        logger.debug("Request is to process authentication");
16     }
17
18     Authentication authResult;
19
20     try {
21         // 接着就是执行子类钩子方法attemptAuthentication来获取认证结果对象Authentication
    , 这个对象不能是空 否则直接返回
22        authResult = attemptAuthentication(request, response);
23        if (authResult == null) {
24           // return immediately as subclass has indicated that it hasn't completed
```

```
            // authentication
            return;
        }
         // 处理session 策略，这里默认没有任何策略
        sessionStrategy.onAuthentication(authResult, request, response);
    }
    catch (InternalAuthenticationServiceException failed) {
        logger.error(
              "An internal error occurred while trying to authenticate the user.",
              failed);
         // 如果遇到异常 就会交给认证失败处理器 AuthenticationFailureHandler 来处理
        unsuccessfulAuthentication(request, response, failed);

        return;
    }
    catch (AuthenticationException failed) {
        // Authentication failed
        unsuccessfulAuthentication(request, response, failed);

        return;
    }

    //  认证成功后继续其它过滤器链 并最终交给认证成功处理器 AuthenticationSuccessHandler 处理
    if (continueChainBeforeSuccessfulAuthentication) {
        chain.doFilter(request, response);
    }

    successfulAuthentication(request, response, chain, authResult);
}
```

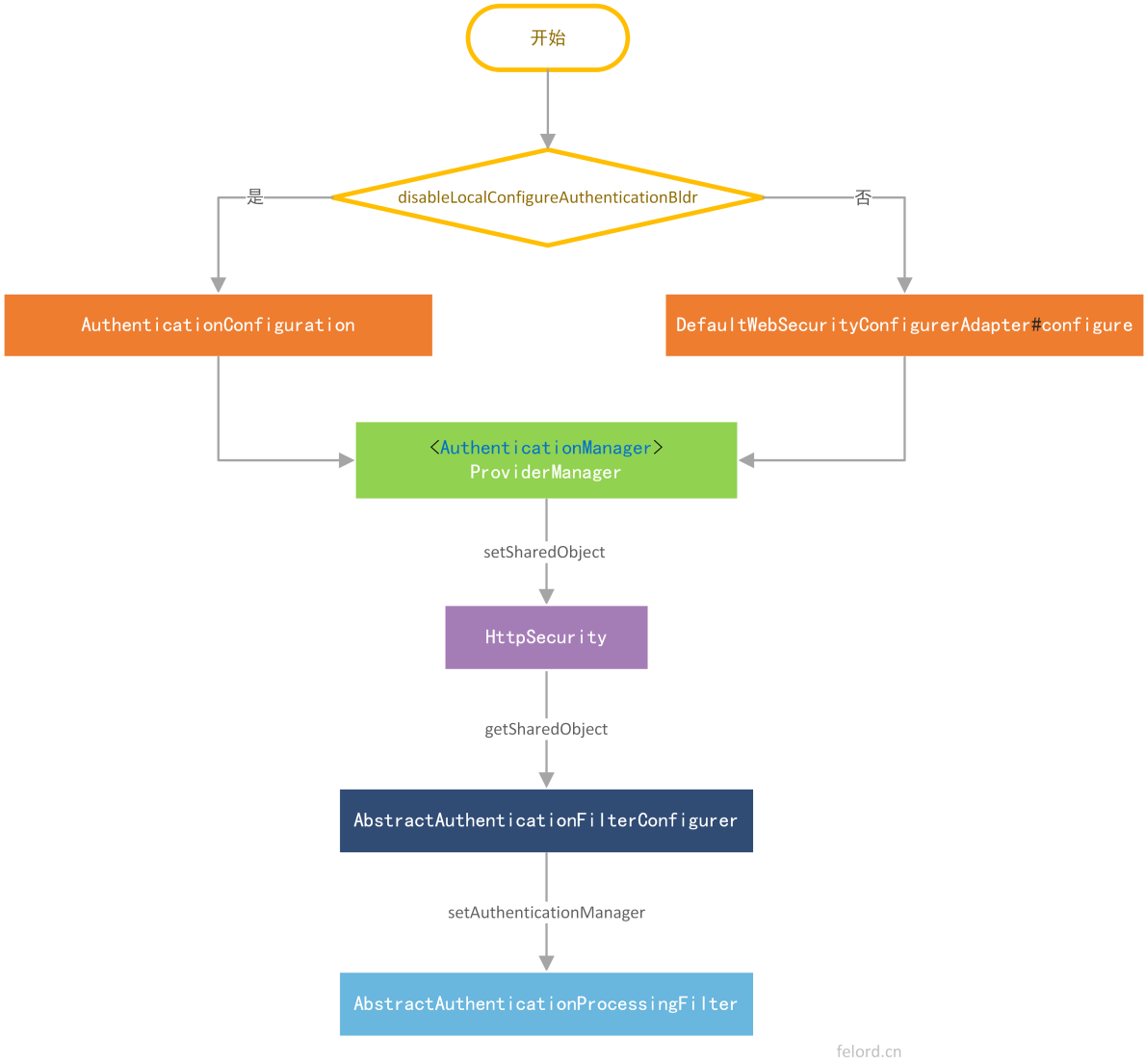
大部分逻辑这里是清晰的，关键在于`attemptAuthentication`方法，这个我们已经在上一文分析了是通过`AuthenticationManager`的`authenticate`方法进行认证逻辑的处理，接下来我们将重点分析这个接口来帮助我们了解Spring Seucirty的认证过程。

3. AuthenticationManager

`AuthenticationManager`这个接口方法非常奇特，入参和返回值的类型都是`Authentication`。该接口的作用是对用户的未授信凭据进行认证，认证通过则返回授信状态的凭据，否则将抛出认证异常`AuthenticationException`。

3.1 AuthenticationManager的初始化流程

那么`AbstractAuthenticationProcessingFilter`中的`AuthenticationManager`是在哪里配置的呢？ 看过Spring Security 实战干货系列应该知道`WebSecurityConfigurerAdapter`中的`void configure(AuthenticationManagerBuilder auth)`是配置`AuthenticationManager`的地方， 我根据源码总结了一下`AuthenticationManager`的初始化流程，相信可以帮助你去阅读相关的源码：



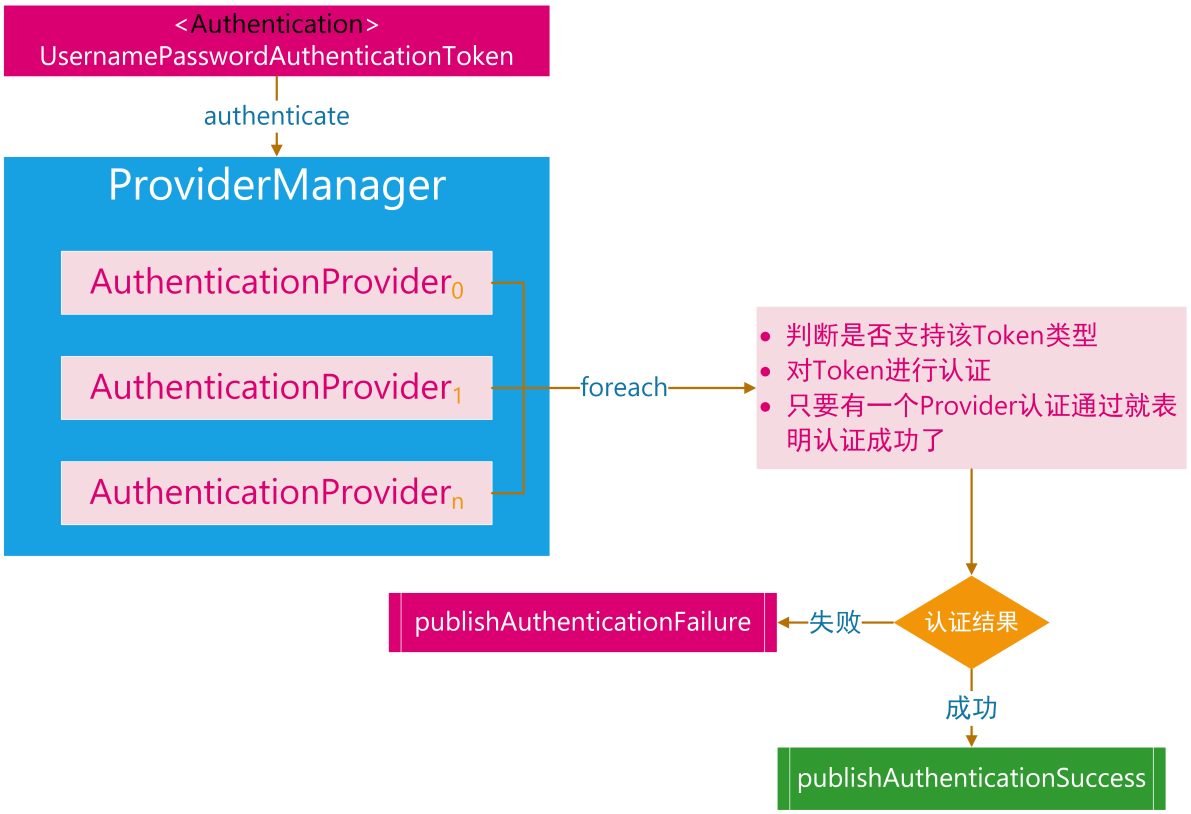
需要注意的是如果我们使用自定义配置一定不能按照类似下面的错误示范：

```
@Override
protected void configure(AuthenticationManagerBuilder auth) throws Exception {
    DaoAuthenticationProvider daoAuthenticationProvider = new
DaoAuthenticationProvider();

 daoAuthenticationProvider.setUserDetailsService(weChatSecurityConfigProperties.ge
tUserDetailsService());
    daoAuthenticationProvider.setPasswordEncoder(multiPasswordEncoder());
    auth.authenticationProvider(daoAuthenticationProvider);
    // 调用 super 将导致不生效 所以下面语句不要写
    super.configure(auth);
}
```

3.2 AuthenticationManager的认证过程

`AuthenticationManager` 的实现 `ProviderManager` 管理了众多的 `AuthenticationProvider` 。每一个 `AuthenticationProvider` 都只支持特定类型的 `Authentication` ，然后是对适配到的 `Authentication` 进行认证，只要有一个 `AuthenticationProvider` 认证成功，那么就认为认证成功，所有的都没有通过才认为是认证失败。认证成功后的 `Authentication` 就变成授信凭据，并触发认证成功的事件。认证失败的就抛出异常触发认证失败的事件。



从这里我们可以看出认证管理器 `AuthenticationManager` 针对特定的 `Authentication` 提供了特定的认证功能，我们可以借此来实现多种认证并存。

AuthenticationManager的初始化细节

1. 前言

今天有个同学告诉我，在Security Learning项目的一个分支中出现了一个问题，登录不兼容了，出现了No Provider异常。还有这事？我赶紧跑了一遍还真是，看来我大意了，不过最终找到了原因，问题就出在 `AuthenticationManager` 的初始化上。自定义了一个 `UseDetailService` 和 `AuthenticationProvider` 之后 `AuthenticationManager` 的默认初始化出问题了。

虽然在 Spring Security 实战干货：图解认证管理器AuthenticationManager 一文中对 `AuthenticationManager` 的流程进行了分析，但是还是不够深入，以至于出现了问题。今天就把这个坑补了。

2. AuthenticationManager的初始化

关于 `AuthenticationManager` 的初始化，流程部分请看这一篇 **文章**，里面有流程图。在流程图中我们提到了 `AuthenticationManager` 的默认初始化是由 `AuthenticationConfiguration` 完成的，但是只是一笔带过，具体的细节没有搞清楚。现在就搞定它。

AuthenticationConfiguration

`AuthenticationConfiguration` 初始化 `AuthenticationManager` 的核心方法就是下面这个方法：

```
public AuthenticationManager getAuthenticationManager() throws Exception {
    // 先判断 AuthenticationManager 是否初始化
   if (this.authenticationManagerInitialized) {
       // 如果已经初始化 那么直接返回初始化的
      return this.authenticationManager;
   }
    // 否则就去 Spring IoC 中获取其构建类
   AuthenticationManagerBuilder authBuilder =
this.applicationContext.getBean(AuthenticationManagerBuilder.class);
    // 如果不是第一次构建  好像是每次总要通过Builder来进行构建
   if (this.buildingAuthenticationManager.getAndSet(true)) {
       // 返回 一个委托的AuthenticationManager
      return new AuthenticationManagerDelegator(authBuilder);
   }
   // 如果是第一次通过Builder构建 将全局的认证配置整合到Builder中  那么以后就不用再整合全局的
配置了
   for (GlobalAuthenticationConfigurerAdapter config : globalAuthConfigurers) {
      authBuilder.apply(config);
   }
   // 构建AuthenticationManager
   authenticationManager = authBuilder.build();
   // 如果构建结果为null
   if (authenticationManager == null) {
       // 再次尝试去Spring IoC 获取懒加载的 AuthenticationManager  Bean
      authenticationManager = getAuthenticationManagerBean();
   }
   // 修改初始化状态
   this.authenticationManagerInitialized = true;
   return authenticationManager;
}
```

根据上面的注释，`AuthenticationManager` 的初始化流程是清楚的。但是又引出来了两个问题，我将另起两个章节来分析这两个问题。

AuthenticationManagerBuilder

> 第一个问题是 `AuthenticationManagerBuilder` 是如何注入Spring IoC的?

`AuthenticationManagerBuilder` 注入的过程也是在 `AuthenticationConfiguration` 中完成的，注入的是其内部的一个静态类 `DefaultPasswordEncoderAuthenticationManagerBuilder`，这个类和Spring Security的主配置类 `WebSecurityConfigurerAdapter` 的一个内部类同名，这两个类几乎逻辑相同，没有什么特别的。具体使用哪个由 `WebSecurityConfigurerAdapter.disableLocalConfigureAuthenticationBldr` 决定。

> 其参数 `ObjectPostProcessor<T>` 抽空会讲它的作用。

GlobalAuthenticationConfigurerAdapter

> 另一个问题是 `GlobalAuthenticationConfigurerAdapter` 从哪儿来?

`AuthenticationConfiguration` 包含下面自动注入 `GlobalAuthenticationConfigurerAdapter` 的方法:

```
@Autowired(required = false)
public void setGlobalAuthenticationConfigurers(
      List<GlobalAuthenticationConfigurerAdapter> configurers) {
   configurers.sort(AnnotationAwareOrderComparator.INSTANCE);
   this.globalAuthConfigurers = configurers;
}
```

该方法会根据它们各自的 `Order` 进行排序。该排序的意义在于 `AuthenticationManagerBuilder` 在执行构建 `AuthenticationManager` 时会按照排序的先后执行 `GlobalAuthenticationConfigurerAdapter` 的 `configure` 方法。

全局认证配置

第一个为 `EnableGlobalAuthenticationAutowiredConfigurer` ,它目前除了打印一下初始化信息没有什么实际作用。

认证处理器初始化注入

第二个为 `InitializeAuthenticationProviderBeanManagerConfigurer` ，核心方法为其内部类的实现:

```
@Override
public void configure(AuthenticationManagerBuilder auth) {
     //
     // 如果存在 AuthenticationProvider 已经注入 或者 已经有AuthenticationManager被代理

   if (auth.isConfigured()) {
      return;
   }

  // 尝试从Spring IoC获取 AuthenticationProvider
   AuthenticationProvider authenticationProvider = getBeanOrNull(
         AuthenticationProvider.class);
    // 获取不到就中断
   if (authenticationProvider == null) {
      return;
   }
    // 获取得到就配置到AuthenticationManagerBuilder中，最终会配置到 AuthenticationManager中
   auth.authenticationProvider(authenticationProvider);
}
```

这里的 `getBeanOrNull` 方法是有误区的，如果不仔细看的话，核心代码如下:

```
String[] userDetailsBeanNames =
InitializeUserDetailsBeanManagerConfigurer.this.context
      .getBeanNamesForType(type);
// Spring IoC 不能同时存在多个type相关类型的Bean 否则无法注入
if (userDetailsBeanNames.length != 1) {
   return null;
}
```

如果 Spring IoC 容器中存在了多个 `AuthenticationProvider`，那么这些 `AuthenticationProvider` 就不会生效。

用户详情管理器初始化注入

第三个为 `InitializeUserDetailsBeanManagerConfigurer`，优先级低于上面。它的核心方法为：

```
public void configure(AuthenticationManagerBuilder auth) throws Exception {
   if (auth.isConfigured()) {
      return;
   }
    // 不能有多个 否则 就中断
   UserDetailsService userDetailsService = getBeanOrNull(
         UserDetailsService.class);
   if (userDetailsService == null) {
      return;
   }
    // 开始配置普通 密码认证器 DaoAuthenticationProvider
   PasswordEncoder passwordEncoder = getBeanOrNull(PasswordEncoder.class);
   UserDetailsPasswordService passwordManager =
getBeanOrNull(UserDetailsPasswordService.class);

   DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
   provider.setUserDetailsService(userDetailsService);
   if (passwordEncoder != null) {
      provider.setPasswordEncoder(passwordEncoder);
   }
   if (passwordManager != null) {
      provider.setUserDetailsPasswordService(passwordManager);
   }
   provider.afterPropertiesSet();

   auth.authenticationProvider(provider);
}
```

跟 `InitializeAuthenticationProviderBeanManagerConfigurer` 流程差不多，只不过这里主要处理的是 `UserDetailsService`、`DaoAuthenticationProvider`。当执行到上面这个方法时，如果 Spring IoC 容器中存在了多个 `UserDetailsService`，那么这些 `UserDetailsService` 就不会生效，影响 `DaoAuthenticationProvider` 的注入。

3. 真相大白

到此为什么在认证的时候找不到原因终于找到了，**原来我在使用Spring Security默认配置时(注意这个前提)**，向Spring IoC注入了多个 `UserDetailsService` 导致 `DaoAuthenticationProvider` 没有生效。也就是说在一套配置中如果你存在多个 `UserDetailsService` 的Spring Bean将会影响 `DaoAuthenticationProvider` 的注入。

> 但是我仍然需要注入多个 `AuthenticationProvider` 怎么办?

首先把你需要配置的 `AuthenticationProvider` 注入Spring IoC，然后在 `HttpSecurity` 中这么写：

```
protected void configure(HttpSecurity http) throws Exception {
    ApplicationContext context = http.getSharedObject(ApplicationContext.class);
    CaptchaAuthenticationProvider captchaAuthenticationProvider =
context.getBean("captchaAuthenticationProvider",
CaptchaAuthenticationProvider.class);
    http.authenticationProvider(captchaAuthenticationProvider);
    // 省略
    }
```

有几个 `AuthenticationProvider` 你就按照上面配置几个。

> 一般情况下一个 `UserDetailsService` 对应一个 `AuthenticationProvider` 。

4. 总结

这一篇对于需要多种认证方式并存的Spring Security配置非常重要，如果你在配置中不注意，很容易引发 `No Provider ……` 的异常。所以有很有必要学习一下。

通过本文我们对Spring Security认证管理器 `AuthenticationManager` 的初始化过程和认证过程进行了分析，如果你熟悉了 `AuthenticationManager` 的逻辑可以实现多种认证方式的并存等能力，实现很多有用的逻辑，这对集成Spring Security到项目中非常重要。多多关注：**码农小胖哥** 获取更多的原创干货。

Spring Security中的“分布式对象”

1. 前言

在上一篇我们对 `AuthenticationManager` 的初始化的细节进行了分析，其中里面有一段代码引起了不少同学的注意：

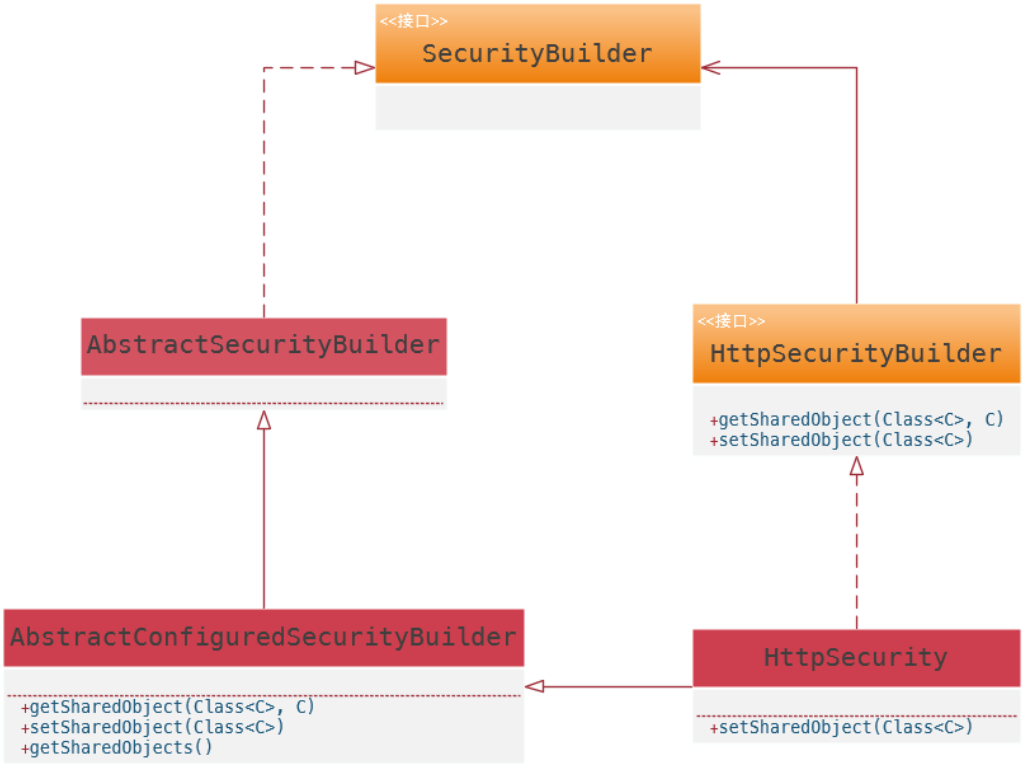
```
  ApplicationContext context = http.getSharedObject(ApplicationContext.class);
    CaptchaAuthenticationProvider captchaAuthenticationProvider =
context.getBean("captchaAuthenticationProvider",
CaptchaAuthenticationProvider.class);
```

上面直接从 `HttpSecurity` 对象中获取到Spring的应用上下文对象 `ApplicationContext` ，它是怎么做到的呢？SharedObject又是个什么概念？今天就来搞清楚这个问题。

2. SharedObject

> 在Spring Security中SharedObject既不是对象也不是接口，而是某一类“可共享”的对象的统称。

顾名思义，SharedObject的意思是可共享的对象。它的作用是如果一些对象你希望在不同的作用域配置中共享它们就把这些对象变成SharedObject，有点分布式对象的感觉。为了更加便于你理解，下面是相关的体系结构：



`AbstractConfiguredSecurityBuilder`或者`HttpSecurityBuilder`的实现类才具有操作SharedObject的能力。一种是注册SharedObject，另一种是获取SharedObject。

SharedObject的注册

SharedObject会以其`Class`类型为`Key`,实例为`Value`存储到一个`HashMap<Class<?>,Object>`中，具体可看`HttpSecurity`源码。它的注册分为两个部分，第一是`HttpSecurity`初始化的时候装配进去的。我们来看看：

```
public HttpSecurity(ObjectPostProcessor<Object> objectPostProcessor,
        AuthenticationManagerBuilder authenticationBuilder,
        Map<Class<?>, Object> sharedObjects) {
    super(objectPostProcessor);
    Assert.notNull(authenticationBuilder, message: "authenticationBuilder cannot be null");
    setSharedObject(AuthenticationManagerBuilder.class, authenticationBuilder);
    for (Map.Entry<Class<?>, Object> entry : sharedObjects
            .entrySet()) {
        setSharedObject((Class<Object>) entry.getKey(), entry.getValue());
    }
    ApplicationContext context = (ApplicationContext) sharedObjects
            .get(ApplicationContext.class);
    this.requestMatcherConfigurer = new RequestMatcherConfigurer(context);
}
```

我们熟知的AuthenticationManagerBuilder在这里被共享。

还有一部分是在所有的`HttpSecurityBuilder`对象初始化时注册的。它初始化和配置都是由`SecurityConfigurer`来完成的：

```
public interface SecurityConfigurer<O, B extends SecurityBuilder<O>> {

   void init(B builder) throws Exception;

   void configure(B builder) throws Exception;
}
```

上面两个方法分别用来初始化和配置`HttpSecurityBuilder`。比如我们熟知的`WebSecurityConfigurerAdapter`就是用来配置`HttpSecurity`的，在其`init`方法中我们可以找到相关的代码：

```
private Map<Class<?>, Object> createSharedObjects() {
   Map<Class<?>, Object> sharedObjects = new HashMap<>();
   sharedObjects.putAll(localConfigureAuthenticationBldr.getSharedObjects());
   sharedObjects.put(UserDetailsService.class, userDetailsService());
   sharedObjects.put(ApplicationContext.class, context);
   sharedObjects.put(ContentNegotiationStrategy.class,
contentNegotiationStrategy);
   sharedObjects.put(AuthenticationTrustResolver.class, trustResolver);
   return sharedObjects;
}
```

这也是我在文章开头可以获取到`ApplicationContext`的根本原因。

SharedObject的获取和使用

我们能获取到哪些被标记为SharedObject类呢？`SecurityConfigurer`有很多实现，这些实现都是用来配置一些特定的同认证授权相关的功能的。比如`OAuth2ClientConfigurer`用来配置OAuth2客户端的，它里面就将常用的一些对象设置为SharedObject：

```
public OAuth2ClientConfigurer<B>
clientRegistrationRepository(ClientRegistrationRepository
clientRegistrationRepository) {
   Assert.notNull(clientRegistrationRepository, "clientRegistrationRepository
cannot be null");
   this.getBuilder().setSharedObject(ClientRegistrationRepository.class,
clientRegistrationRepository);
   return this;
}
```

当你在`HttpSecurity`的配置中的其它地方需要用到`ClientRegistrationRepository`时，你可以直接通过`getSharedObject`获取，就像文章开头一样，而不用在去写一些获取方法了。

3. 总结

SharedObject是Spring Security提供的一个非常好用的功能，如果你在不同的地方需要对一个对象重复使用就可以将它注册为SharedObject，甚至直接注入Spring IoC像开头那样获取就可以了。这个特性能够简化配置，提高代码的可读性，也为Spring Security的DSL特性打下了基础。好了今天的分享就到这里，我是：**码农小胖哥**，多多关注，获取更多实用的编程干货。

从零手写一个验证码登录

1. 前言

前面关于Spring Security写了两篇文章，一篇是介绍 UsernamePasswordAuthenticationFilter，另一篇是介绍 AuthenticationManager。很多同学表示无法理解这两个东西有什么用，能解决哪些实际问题？所以今天就对这两篇理论进行实战运用，我们从零写一个短信验证码登录并适配到Spring Security体系中。如果你在阅读中有什么疑问可以回头看看这两篇文章，能解决很多疑惑。

> 当然你可以修改成邮箱或者其它通讯设备的验证码登录。

2. 验证码生命周期

> 验证码存在有效期，一般5分钟。 一般逻辑是用户输入手机号后去获取验证码，服务端对验证码进行缓存。在最大有效期内用户只能使用验证码验证成功一次（避免验证码浪费）；超过最大时间后失效。

验证码的缓存生命周期：

```
public interface CaptchaCacheStorage {

    /**
     * 验证码放入缓存.
     *
     * @param phone the phone
     * @return the string
     */
    String put(String phone);

    /**
     * 从缓存取验证码.
     *
     * @param phone the phone
     * @return the string
     */
    String get(String phone);

    /**
     * 验证码手动过期.
     *
     * @param phone the phone
     */
    void expire(String phone);
}
```

我们一般会借助于缓存中间件，比如Redis、 Ehcache、 Memcached等等来做这个事情。为了方便收看该教程的同学们所使用的不同的中间件。这里我结合 Spring Cache 特意抽象了验证码的缓存处理。

```
private static final String SMS_CAPTCHA_CACHE = "captcha";
@Bean
CaptchaCacheStorage captchaCacheStorage() {
    return new CaptchaCacheStorage() {

        @CachePut(cacheNames = SMS_CAPTCHA_CACHE, key = "#phone")
```

```
        @Override
        public String put(String phone) {
            return RandomUtil.randomNumbers(5);
        }

        @Cacheable(cacheNames = SMS_CAPTCHA_CACHE, key = "#phone")
        @Override
        public String get(String phone) {
            return null;
        }

        @CacheEvict(cacheNames = SMS_CAPTCHA_CACHE, key = "#phone")
        @Override
        public void expire(String phone) {

        }
    };
}
```

> 务必保证缓存的可靠性，这与用户的体验息息相关。

接着我们就来编写和业务无关的验证码服务了，验证码服务的核心功能有两个：**发送验证码**和**验证码校验**。其它的诸如统计、黑名单、历史记录可根据实际业务定制。这里只实现核心功能。

```
/**
 * 验证码服务.
 * 两个功能:  发送和校验.
 *
 * @param captchaCacheStorage the captcha cache storage
 * @return the captcha service
 */
@Bean
public CaptchaService captchaService(CaptchaCacheStorage captchaCacheStorage) {
    return new CaptchaService() {
        @Override
        public boolean sendCaptcha(String phone) {
            String existed = captchaCacheStorage.get(phone);
            if (StringUtils.hasText(existed)) {
                // 节约成本的话如果缓存中有当前手机可用的验证码 不再发新的验证码
                return true;
            }
            // 生成验证码并放入缓存
            String captchaCode = captchaCacheStorage.put(phone);
            log.info("captcha: {}", captchaCode);

            //todo 这里自行完善调用第三方短信服务发送验证码
            return true;
        }

        @Override
        public boolean verifyCaptcha(String phone, String code) {
            String cacheCode = captchaCacheStorage.get(phone);

            if (Objects.equals(cacheCode, code)) {
                // 验证通过手动过期
                captchaCacheStorage.expire(phone);
                return true;
```

```
            }
            return false;
        }
    };
}
```

接下来就可以根据`CaptchaService`编写短信发送接口`/captcha/{phone}`了。

```
@RestController
@RequestMapping("/captcha")
public class CaptchaController {

    @Resource
    CaptchaService captchaService;


    /**
     * 模拟手机号发送验证码.
     *
     * @param phone the mobile
     * @return the rest
     */
    @GetMapping("/{phone}")
    public Rest<?> captchaByMobile(@PathVariable String phone) {
        //todo 手机号 正则自行验证

        if (captchaService.sendCaptcha(phone)){
            return RestBody.ok("验证码发送成功");
        }
        return RestBody.failure(-999,"验证码发送失败");
    }

}
```

3. 集成到Spring Security

下面的教程就必须用到前两篇介绍的知识了。我们要实现验证码登录就必须定义一个**Servlet Filter**进行处理。它的作用这里再重复一下：

- 拦截短信登录接口。
- 获取登录参数并封装为`Authentication`凭据。
- 交给`AuthenticationManager`认证。

我们需要先定制`Authentication`和`AuthenticationManager`

3.1 验证码凭据

`Authentication`在我看来就是一个载体，在未得到认证之前它用来携带登录的关键参数，比如用户名和密码、验证码；在认证成功后它携带用户的信息和角色集。所以模仿`UsernamePasswordAuthenticationToken`来实现一个`CaptchaAuthenticationToken`，去掉不必要的功能，抄就完事儿了：

```
package cn.felord.spring.security.captcha;

import org.springframework.security.authentication.AbstractAuthenticationToken;
```

```java
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.SpringSecurityCoreVersion;

import java.util.Collection;

/**
 * 验证码认证凭据.
 * @author felord.cn
 */
public class CaptchaAuthenticationToken extends AbstractAuthenticationToken {

    private static final long serialVersionUID = SpringSecurityCoreVersion.SERIAL_VERSION_UID;

    private final Object principal;
    private String captcha;

    /**
     * 此构造函数用来初始化未授信凭据.
     *
     * @param principal   the principal
     * @param captcha the captcha
     * @see CaptchaAuthenticationToken#CaptchaAuthenticationToken(Object, String, Collection)
     */
    public CaptchaAuthenticationToken(Object principal, String captcha) {
        super(null);
        this.principal =  principal;
        this.captcha = captcha;
        setAuthenticated(false);
    }

    /**
     * 此构造函数用来初始化授信凭据.
     *
     * @param principal       the principal
     * @param captcha     the captcha
     * @param authorities the authorities
     * @see CaptchaAuthenticationToken#CaptchaAuthenticationToken(Object, String)
     */
    public CaptchaAuthenticationToken(Object principal, String captcha,
                                      Collection<? extends GrantedAuthority> authorities) {
        super(authorities);
        this.principal = principal;
        this.captcha = captcha;
        super.setAuthenticated(true); // must use super, as we override
    }

    public Object getCredentials() {
        return this.captcha;
    }

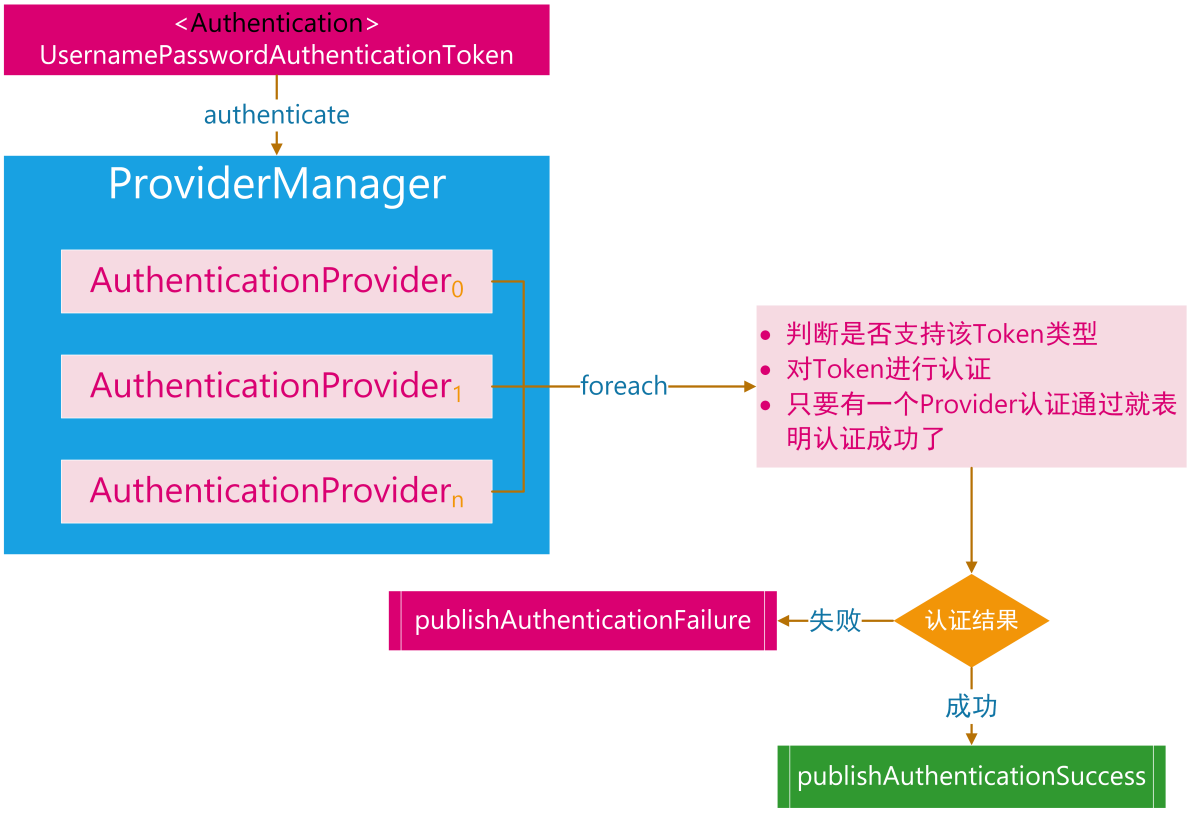
    public Object getPrincipal() {
        return this.principal;
    }

```

```
58      public void setAuthenticated(boolean isAuthenticated) throws
    IllegalArgumentException {
59          if (isAuthenticated) {
60              throw new IllegalArgumentException(
61                      "Cannot set this token to trusted - use constructor which
    takes a GrantedAuthority list instead");
62          }
63
64          super.setAuthenticated(false);
65      }
66
67      @Override
68      public void eraseCredentials() {
69          super.eraseCredentials();
70          captcha = null;
71      }
```

3.2 验证码认证管理器

我们还需要定制一个`AuthenticationManager`来对上面定义的凭据`CaptchaAuthenticationToken`进行认证处理。下面这张图有必要再拿出来看一下：



要定义`AuthenticationManager`只需要定义其实现`ProviderManager`。而`ProviderManager`又需要依赖`AuthenticationProvider`。所以我们要实现一个专门处理`CaptchaAuthenticationToken`的`AuthenticationProvider`。`AuthenticationProvider`的流程是：

1. 从`CaptchaAuthenticationToken`拿到手机号、验证码。
2. 利用手机号从数据库查询用户信息，并判断用户是否是有效用户，实际上就是实现`UserDetailsService`接口
3. 验证码校验。
4. 校验成功则封装授信的凭据。
5. 校验失败抛出认证异常。

根据这个流程实现如下：

```java
package cn.felord.spring.security.captcha;

import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.InitializingBean;
import org.springframework.context.MessageSource;
import org.springframework.context.MessageSourceAware;
import org.springframework.context.support.MessageSourceAccessor;
import org.springframework.security.authentication.AuthenticationProvider;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.SpringSecurityMessageSource;
import
org.springframework.security.core.authority.mapping.GrantedAuthoritiesMapper;
import
org.springframework.security.core.authority.mapping.NullAuthoritiesMapper;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.util.Assert;

import java.util.Collection;
import java.util.Objects;

/**
 * 验证码认证器.
 * @author felord.cn
 */
@Slf4j
public class CaptchaAuthenticationProvider implements AuthenticationProvider,
InitializingBean, MessageSourceAware {
    private final GrantedAuthoritiesMapper authoritiesMapper = new
NullAuthoritiesMapper();
    private final UserDetailsService userDetailsService;
    private final CaptchaService captchaService;
    private MessageSourceAccessor messages =
SpringSecurityMessageSource.getAccessor();

    /**
     * Instantiates a new Captcha authentication provider.
     *
     * @param userDetailsService the user details service
     * @param captchaService     the captcha service
     */
    public CaptchaAuthenticationProvider(UserDetailsService userDetailsService,
CaptchaService captchaService) {
        this.userDetailsService = userDetailsService;
        this.captchaService = captchaService;
    }

    @Override
    public Authentication authenticate(Authentication authentication) throws
AuthenticationException {
        Assert.isInstanceOf(CaptchaAuthenticationToken.class, authentication,
                () -> messages.getMessage(
                        "CaptchaAuthenticationProvider.onlySupports",
                        "Only CaptchaAuthenticationToken is supported"));
```

```
51
52          CaptchaAuthenticationToken unAuthenticationToken =
    (CaptchaAuthenticationToken) authentication;
53
54          String phone = unAuthenticationToken.getName();
55          String rawCode = (String) unAuthenticationToken.getCredentials();
56
57          UserDetails userDetails = userDetailsService.loadUserByUsername(phone);
58
59          // 此处省略对UserDetails 的可用性 是否过期  是否锁定 是否失效的检验  建议根据实际
    情况添加  或者在 UserDetailsService 的实现中处理
60          if (Objects.isNull(userDetails)) {
61              throw new BadCredentialsException("Bad credentials");
62          }
63
64          // 验证码校验
65          if (captchaService.verifyCaptcha(phone, rawCode)) {
66              return createSuccessAuthentication(authentication, userDetails);
67          } else {
68              throw new BadCredentialsException("captcha is not matched");
69          }
70
71      }
72
73      @Override
74      public boolean supports(Class<?> authentication) {
75          return
    CaptchaAuthenticationToken.class.isAssignableFrom(authentication);
76      }
77
78      @Override
79      public void afterPropertiesSet() throws Exception {
80          Assert.notNull(userDetailsService, "userDetailsService must not be
    null");
81          Assert.notNull(captchaService, "captchaService must not be null");
82      }
83
84      @Override
85      public void setMessageSource(MessageSource messageSource) {
86          this.messages = new MessageSourceAccessor(messageSource);
87      }
88
89      /**
90       * 认证成功将非授信凭据转为授信凭据.
91       * 封装用户信息  角色信息。
92       *
93       * @param authentication the authentication
94       * @param user           the user
95       * @return the authentication
96       */
97      protected Authentication createSuccessAuthentication(Authentication
    authentication, UserDetails user) {
98
99          Collection<? extends GrantedAuthority> authorities =
    authoritiesMapper.mapAuthorities(user.getAuthorities());
100         CaptchaAuthenticationToken authenticationToken = new
    CaptchaAuthenticationToken(user, null, authorities);
101         authenticationToken.setDetails(authentication.getDetails());
```

```
102
103             return authenticationToken;
104         }
105
106     }
```

然后就可以组装`ProviderManager`了：

```
1   ProviderManager providerManager = new
    ProviderManager(Collections.singletonList(captchaAuthenticationProvider));
```

经过3.1和3.2的准备，我们的准备工作就完成了。

3.3 验证码认证过滤器

定制好验证码凭据和验证码认证管理器后我们就可以定义验证码认证过滤器了。修改一下`UsernamePasswordAuthenticationFilter`就能满足需求：

```
1   package cn.felord.spring.security.captcha;
2
3   import org.springframework.lang.Nullable;
4   import
    org.springframework.security.authentication.AuthenticationServiceException;
5   import org.springframework.security.core.Authentication;
6   import org.springframework.security.core.AuthenticationException;
7   import
    org.springframework.security.web.authentication.AbstractAuthenticationProcessingF
    ilter;
8   import org.springframework.security.web.util.matcher.AntPathRequestMatcher;
9
10  import javax.servlet.http.HttpServletRequest;
11  import javax.servlet.http.HttpServletResponse;
12
13  public class CaptchaAuthenticationFilter extends
    AbstractAuthenticationProcessingFilter {
14
15
16      public static final String SPRING_SECURITY_FORM_PHONE_KEY = "phone";
17      public static final String SPRING_SECURITY_FORM_CAPTCHA_KEY = "captcha";
18
19
20      public CaptchaAuthenticationFilter() {
21          super(new AntPathRequestMatcher("/clogin", "POST"));
22      }
23
24      public Authentication attemptAuthentication(HttpServletRequest request,
25                                                  HttpServletResponse response)
    throws AuthenticationException {
26
27          if (!request.getMethod().equals("POST")) {
28              throw new AuthenticationServiceException(
29                      "Authentication method not supported: " +
    request.getMethod());
30          }
31
32          String phone = obtainPhone(request);
33          String captcha = obtainCaptcha(request);
34
```

```
        if (phone == null) {
            phone = "";
        }

        if (captcha == null) {
            captcha = "";
        }

        phone = phone.trim();

        CaptchaAuthenticationToken authRequest = new CaptchaAuthenticationToken(
                phone, captcha);

        // Allow subclasses to set the "details" property
        setDetails(request, authRequest);

        return this.getAuthenticationManager().authenticate(authRequest);
    }

    @Nullable
    protected String obtainCaptcha(HttpServletRequest request) {
        return request.getParameter(SPRING_SECURITY_FORM_CAPTCHA_KEY);
    }

    @Nullable
    protected String obtainPhone(HttpServletRequest request) {
        return request.getParameter(SPRING_SECURITY_FORM_PHONE_KEY);
    }

    protected void setDetails(HttpServletRequest request,
                              CaptchaAuthenticationToken authRequest) {

authRequest.setDetails(authenticationDetailsSource.buildDetails(request));
    }

}
```

这里我们指定了拦截验证码登陆的请求为：

```
POST /clogin?phone=手机号&captcha=验证码 HTTP/1.1
Host: localhost:8082
```

接下来就是配置了。

3.4 配置

我把所有的验证码认证的相关配置集中了起来，并加上了注释。

```
package cn.felord.spring.security.captcha;

import cn.hutool.core.util.RandomUtil;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.cache.annotation.CacheEvict;
import org.springframework.cache.annotation.CachePut;
import org.springframework.cache.annotation.Cacheable;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
```

```java
import org.springframework.security.authentication.ProviderManager;
import org.springframework.security.core.authority.AuthorityUtils;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetailsService;
import
org.springframework.security.web.authentication.AuthenticationFailureHandler;
import
org.springframework.security.web.authentication.AuthenticationSuccessHandler;
import org.springframework.util.StringUtils;

import java.util.Collections;
import java.util.Objects;

/**
 * 验证码认证配置.
 *
 * @author felord.cn
 * @since 13 :23
 */
@Slf4j
@Configuration
public class CaptchaAuthenticationConfiguration {
    private static final String SMS_CAPTCHA_CACHE = "captcha";

    /**
     * spring cache 管理验证码的生命周期.
     *
     * @return the captcha cache storage
     */
    @Bean
    CaptchaCacheStorage captchaCacheStorage() {
        return new CaptchaCacheStorage() {

            @CachePut(cacheNames = SMS_CAPTCHA_CACHE, key = "#phone")
            @Override
            public String put(String phone) {
                return RandomUtil.randomNumbers(5);
            }

            @Cacheable(cacheNames = SMS_CAPTCHA_CACHE, key = "#phone")
            @Override
            public String get(String phone) {
                return null;
            }

            @CacheEvict(cacheNames = SMS_CAPTCHA_CACHE, key = "#phone")
            @Override
            public void expire(String phone) {

            }
        };
    }

    /**
     * 验证码服务.
     * 两个功能： 发送和校验.
     *
     * @param captchaCacheStorage the captcha cache storage
```

```
67          * @return the captcha service
68          */
69         @Bean
70         public CaptchaService captchaService(CaptchaCacheStorage
    captchaCacheStorage) {
71             return new CaptchaService() {
72                 @Override
73                 public boolean sendCaptcha(String phone) {
74                     String existed = captchaCacheStorage.get(phone);
75                     if (StringUtils.hasText(existed)) {
76                         // 节约成本的话如果缓存存在可用的验证码 不再发新的验证码
77                         log.warn("captcha code 【 {} 】 is available now", existed);
78                         return false;
79                     }
80                     // 生成验证码并放入缓存
81                     String captchaCode = captchaCacheStorage.put(phone);
82                     log.info("captcha: {}", captchaCode);
83 
84                     //todo 这里自行完善调用第三方短信服务
85                     return true;
86                 }
87 
88                 @Override
89                 public boolean verifyCaptcha(String phone, String code) {
90                     String cacheCode = captchaCacheStorage.get(phone);
91 
92                     if (Objects.equals(cacheCode, code)) {
93                         // 验证通过手动过期
94                         captchaCacheStorage.expire(phone);
95                         return true;
96                     }
97                     return false;
98                 }
99             };
100        }
101 
102        /**
103         * 自行实现根据手机号查询可用的用户，这里简单举例.
104         * 注意该接口可能出现多态。所以最好加上注解@Qualifier
105         *
106         * @return the user details service
107         */
108        @Bean
109        @Qualifier("captchaUserDetailsService")
110        public UserDetailsService captchaUserDetailsService() {
111            // 验证码登陆后密码无意义了但是需要填充一下
112            return username -> User.withUsername(username).password("TEMP")
113                    //todo  这里权限 你需要自己注入
114                    .authorities(AuthorityUtils.createAuthorityList("ROLE_ADMIN",
    "ROLE_APP")).build();
115        }
116 
117        /**
118         * 验证码认证器.
119         *
120         * @param captchaService     the captcha service
121         * @param userDetailsService the user details service
122         * @return the captcha authentication provider
```

```
123         */
124        @Bean
125        public CaptchaAuthenticationProvider
    captchaAuthenticationProvider(CaptchaService captchaService,
126
    @Qualifier("captchaUserDetailsService")
127
    UserDetailsService userDetailsService) {
128            return new CaptchaAuthenticationProvider(userDetailsService,
    captchaService);
129        }
130
131
132        /**
133         * 验证码认证过滤器.
134         *
135         * @param authenticationSuccessHandler  the authentication success handler
136         * @param authenticationFailureHandler  the authentication failure handler
137         * @param captchaAuthenticationProvider the captcha authentication provider
138         * @return the captcha authentication filter
139         */
140        @Bean
141        public CaptchaAuthenticationFilter
    captchaAuthenticationFilter(AuthenticationSuccessHandler
    authenticationSuccessHandler,
142
    AuthenticationFailureHandler authenticationFailureHandler,
143
    CaptchaAuthenticationProvider captchaAuthenticationProvider) {
144            CaptchaAuthenticationFilter captchaAuthenticationFilter = new
    CaptchaAuthenticationFilter();
145            // 配置 authenticationManager
146            ProviderManager providerManager = new
    ProviderManager(Collections.singletonList(captchaAuthenticationProvider));
147            captchaAuthenticationFilter.setAuthenticationManager(providerManager);
148            // 成功处理器
149
     captchaAuthenticationFilter.setAuthenticationSuccessHandler(authenticationSucce
    ssHandler);
150            // 失败处理器
151
     captchaAuthenticationFilter.setAuthenticationFailureHandler(authenticationFailu
    reHandler);
152
153            return captchaAuthenticationFilter;
154        }
155    }
```

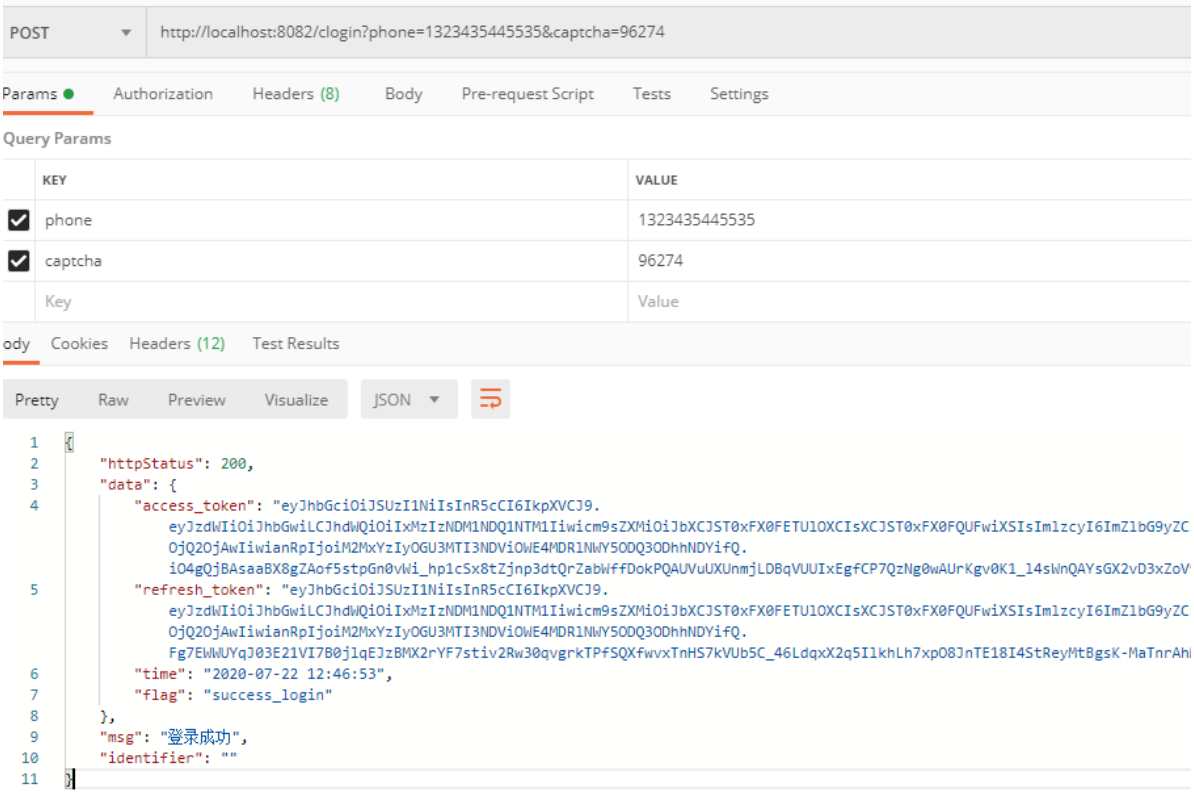
然而这并没有完，你需要将`CaptchaAuthenticationFilter`配置到整个Spring Security的过滤器链中，这种看了胖哥教程的同学应该非常熟悉了。

```
protected void configure(HttpSecurity http) throws Exception {
    http.csrf().disable() HttpSecurity
            .cors() CorsConfigurer<HttpSecurity>
            .and() HttpSecurity
            // session 生成策略用无状态策略
            .sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS) SessionManageme
            .and() HttpSecurity
            .exceptionHandling().accessDeniedHandler(new SimpleAccessDeniedHandler()).authenticationEn
            .and() HttpSecurity
            .authorizeRequests().anyRequest().authenticated().withObjectPostProcessor(filterSecurityIn
            .and() HttpSecurity
            .addFilterBefore(preLoginFilter, UsernamePasswordAuthenticationFilter.class)
            // 验证码登陆过滤器配置
            .addFilterBefore(captchaAuthenticationFilter,PreLoginFilter.class)
            // jwt 必须配置于 UsernamePasswordAuthenticationFilter 之前
            .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)
            // 登录  成功后返回jwt token  失败后返回 错误信息
```

1 给过滤器找一个合适的位置

> **请特别注意：**务必保证登录接口和验证码接口可以匿名访问，如果是动态权限可以给接口添加 `ROLE_ANONYMOUS` 角色。

大功告成，测试如下：



而且原先的登录方式不受影响。

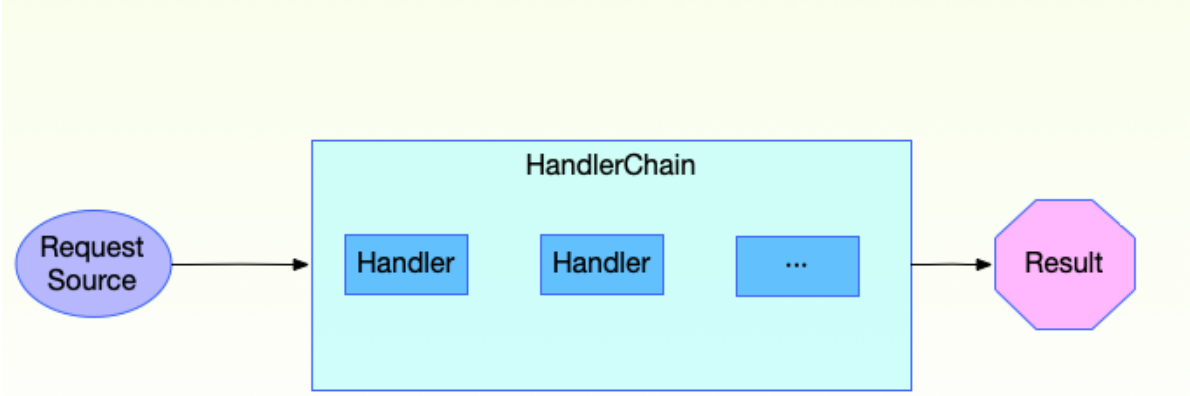
4. 总结

通过对 UsernamePasswordAuthenticationFilter 和 AuthenticationManager 的系统学习，我们了解了 Spring Security 认证的整个流程，本文是对这两篇的一个实际运用。相信看到这一篇后你就不会对前几篇的图解懵逼了，这也是理论到实践的一次尝试。DEMO 可以通过公众号：**码农小胖哥** 回复 captcha 获取，如果有用还请关注、点赞、转发给胖哥一个创作的动力。

Spring Security 中的内置 Filter

1. 前言

上一文我们使用 Spring Security 实现了各种登录聚合的场面。其中我们是通过在 `UsernamePasswordAuthenticationFilter` 之前一个自定义的过滤器实现的。我怎么知道自定义过滤器要加在 `UsernamePasswordAuthenticationFilter` 之前。我在这个系列开篇说了 Spring Security 权限控制的一个核心关键就是 **过滤器链** ，这些过滤器如下图进行过滤传递，甚至比这个更复杂！这只是一个最小单元。



Spring Security 内置了一些过滤器，他们各有各的本事。如果你掌握了这些过滤器，很多实际开发中的需求和问题都很容易解决。今天我们来见识一下这些内置的过滤器。

2. 内置过滤器初始化

在 Spring Security 初始化核心过滤器时 `HttpSecurity` 会通过将 Spring Security 内置的一些过滤器以 `FilterComparator` 提供的规则进行比较按照比较结果进行排序注册。

2.1 排序规则

`FilterComparator` 维护了一个顺序的注册表 `filterToOrder` 。

```
1       FilterComparator() {
2           Step order = new Step(INITIAL_ORDER, ORDER_STEP);
3           put(ChannelProcessingFilter.class, order.next());
4           put(ConcurrentSessionFilter.class, order.next());
5           put(WebAsyncManagerIntegrationFilter.class, order.next());
6           put(SecurityContextPersistenceFilter.class, order.next());
7           put(HeaderWriterFilter.class, order.next());
8           put(CorsFilter.class, order.next());
9           put(CsrfFilter.class, order.next());
10          put(LogoutFilter.class, order.next());
11          filterToOrder.put(
12
    "org.springframework.security.oauth2.client.web.OAuth2AuthorizationRequestRedirectFilter",
13                  order.next());
14          filterToOrder.put(
```

```
15
    "org.springframework.security.saml2.provider.service.servlet.filter.Saml2WebSsoAuthenticationRequestFilter",
16                  order.next());
17          put(X509AuthenticationFilter.class, order.next());
18          put(AbstractPreAuthenticatedProcessingFilter.class, order.next());
19
    filterToOrder.put("org.springframework.security.cas.web.CasAuthenticationFilter",
20                  order.next());
21          filterToOrder.put(
22
    "org.springframework.security.oauth2.client.web.OAuth2LoginAuthenticationFilter",
23                  order.next());
24          filterToOrder.put(
25
    "org.springframework.security.saml2.provider.service.servlet.filter.Saml2WebSsoAuthenticationFilter",
26                  order.next());
27          put(UsernamePasswordAuthenticationFilter.class, order.next());
28          put(ConcurrentSessionFilter.class, order.next());
29          filterToOrder.put(
30                  "org.springframework.security.openid.OpenIDAuthenticationFilter",
    order.next());
31          put(DefaultLoginPageGeneratingFilter.class, order.next());
32          put(DefaultLogoutPageGeneratingFilter.class, order.next());
33          put(ConcurrentSessionFilter.class, order.next());
34          put(DigestAuthenticationFilter.class, order.next());
35          filterToOrder.put(
36
    "org.springframework.security.oauth2.server.resource.web.BearerTokenAuthenticationFilter", order.next());
37          put(BasicAuthenticationFilter.class, order.next());
38          put(RequestCacheAwareFilter.class, order.next());
39          put(SecurityContextHolderAwareRequestFilter.class, order.next());
40          put(JaasApiIntegrationFilter.class, order.next());
41          put(RememberMeAuthenticationFilter.class, order.next());
42          put(AnonymousAuthenticationFilter.class, order.next());
43          filterToOrder.put(
44
    "org.springframework.security.oauth2.client.web.OAuth2AuthorizationCodeGrantFilter",
45                  order.next());
46          put(SessionManagementFilter.class, order.next());
47          put(ExceptionTranslationFilter.class, order.next());
48          put(FilterSecurityInterceptor.class, order.next());
49          put(SwitchUserFilter.class, order.next());
50      }
```

这些就是所有内置的过滤器。 他们是通过下面的方法获取自己的序号：

```
    private Integer getOrder(Class<?> clazz) {
        while (clazz != null) {
            Integer result = filterToOrder.get(clazz.getName());
            if (result != null) {
                return result;
            }
            clazz = clazz.getSuperclass();
        }
        return null;
    }
```

通过过滤器的类全限定名从注册表 `filterToOrder` 中获取自己的序号，如果没有直接获取到序号通过递归获取父类在注册表中的序号作为自己的序号，序号越小优先级越高。上面的过滤器并非全部会被初始化。有的需要额外引入一些功能包，有的看 `HttpSecurity` 的配置情况。 在上一篇**文章**中。我们禁用了 `CSRF` 功能，就意味着 `CsrfFilter` 不会被注册。

3. 内置过滤器讲解

接下来我们就对这些内置过滤器进行一个系统的认识。**我们将按照默认顺序进行讲解。**

3.1 ChannelProcessingFilter

`ChannelProcessingFilter` 通常是用来过滤哪些请求必须用 `https` 协议， 哪些请求必须用 `http` 协议， 哪些请求随便用哪个协议都行。它主要有两个属性：

- `ChannelDecisionManager` 用来判断请求是否符合既定的协议规则。它维护了一个 `ChannelProcessor` 列表 这些 `ChannelProcessor` 是具体用来执行 `ANY_CHANNEL` 策略 （任何通道都可以），`REQUIRES_SECURE_CHANNEL` 策略 （只能通过 `https` 通道），`REQUIRES_INSECURE_CHANNEL` 策略 （只能通过 `http` 通道）。
- `FilterInvocationSecurityMetadataSource` 用来存储 url 与 对应的 `ANY_CHANNEL` 、`REQUIRES_SECURE_CHANNEL` 、 `REQUIRES_INSECURE_CHANNEL` 的映射关系。

`ChannelProcessingFilter` 通过 `HttpScurity#requiresChannel()` 等相关方法引入其配置对象 `ChannelSecurityConfigurer` 来进行配置。

3.2 ConcurrentSessionFilter

`ConcurrentSessionFilter` 主要用来判断 `session` 是否过期以及更新最新的访问时间。其流程为：

1. `session` 检测，如果不存在直接放行去执行下一个过滤器。存在则进行下一步。
2. 根据 `sessionid` 从 `SessionRegistry` 中获取 `SessionInformation` ，从 `SessionInformation` 中获取 `session` 是否过期；没有过期则更新 `SessionInformation` 中的访问日期；
   如果过期，则执行 `doLogout()` 方法，这个方法会将 `session` 无效，并将 `SecurityContext` 中的 `Authentication` 中的权限置空，同时在 `SecurityContenxtHoloder` 中清除 `SecurityContext` 然后查看是否有跳转的 `expiredUrl` ，如果有就跳转，没有就输出提示信息。

`ConcurrentSessionFilter` 通过 `SessionManagementConfigurer` 来进行配置。

3.3 WebAsyncManagerIntegrationFilter

`WebAsyncManagerIntegrationFilter`用于集成SecurityContext到Spring异步执行机制中的WebAsyncManager。用来处理异步请求的安全上下文。具体逻辑为：

1. 从请求属性上获取所绑定的`WebAsyncManager`，如果尚未绑定，先做绑定。
2. 从`asyncManager`中获取`key`为`CALLABLE_INTERCEPTOR_KEY`的安全上下文多线程处理器`SecurityContextCallableProcessingInterceptor`，如果获取到的为`null`，新建一个`SecurityContextCallableProcessingInterceptor`并绑定`CALLABLE_INTERCEPTOR_KEY`注册到`asyncManager`中。

这里简单说一下`SecurityContextCallableProcessingInterceptor`。它实现了接口`CallableProcessingInterceptor`，
当它被应用于一次异步执行时，`beforeConcurrentHandling()`方法会在调用者线程执行，该方法会相应地从当前线程获取`SecurityContext`,然后被调用者线程中执行逻辑时，会使用这个`SecurityContext`，从而实现安全上下文从调用者线程到被调用者线程的传输。

`WebAsyncManagerIntegrationFilter`通过`WebSecurityConfigurerAdapter#getHttp()`方法添加到`HttpSecurity`中成为`DefaultSecurityFilterChain`的一个链节。

3.4 SecurityContextPersistenceFilter

`SecurityContextPersistenceFilter`主要控制`SecurityContext`的在一次请求中的生命周期。请求来临时，创建`SecurityContext`安全上下文信息，请求结束时清空`SecurityContextHolder`。

`SecurityContextPersistenceFilter`通过`HttpScurity#securityContext()`及相关方法引入其配置对象`SecurityContextConfigurer`来进行配置。

3.5 HeaderWriterFilter

`HeaderWriterFilter`用来给`http`响应添加一些`Header`,比如`X-Frame-Options`，`X-XSS-Protection`，`X-Content-Type-Options`。

你可以通过`HttpScurity#headers()`来定制请求`Header`。

3.6 CorsFilter

跨域相关的过滤器。这是`Spring MVC Java`配置和`XML`命名空间`CORS`配置的替代方法，仅对依赖于`spring-web`的应用程序有用（不适用于`spring-webmvc`）或要求在`javax.servlet.Filter`级别进行CORS检查的安全约束链接。这个是目前官方的一些解读，但是我还是不太清楚实际机制。

你可以通过`HttpSecurity#cors()`来定制。

3.7 CsrfFilter

`CsrfFilter`用于防止`csrf`攻击，前后端使用json交互需要注意的一个问题。

你可以通过`HttpSecurity.csrf()`来开启或者关闭它。在你使用`jwt`等`token`技术时，是不需要这个的。

3.8 LogoutFilter

`LogoutFilter` 很明显这是处理注销的过滤器。

你可以通过 `HttpSecurity.logout()` 来定制注销逻辑，非常有用。

3.9 OAuth2AuthorizationRequestRedirectFilter

和上面的有所不同，这个需要依赖 `spring-scurity-oauth2` 相关的模块。该过滤器是处理 `OAuth2` 请求首选重定向相关逻辑的。以后会我会带你们认识它，请多多关注公众号： `Felordcn` 。

3.10 Saml2WebSsoAuthenticationRequestFilter

这个需要用到 `Spring Security SAML` 模块，这是一个基于 `SMAL` 的 `SSO` 单点登录请求认证过滤器。

关于SAML

`SAML` 即安全断言标记语言，英文全称是 `Security Assertion Markup Language` 。它是一个基于 `XML` 的标准，用于在不同的安全域（ `security domain` ）之间交换认证和授权数据。在 `SAML` 标准定义了身份提供者（ `identity provider` ）和服务提供者（ `service provider` ），这两者构成了前面所说的不同的安全域。 `SAML` 是 `OASIS` 组织安全服务技术委员会(Security Services Technical Committee) 的产品。

`SAML` （Security Assertion Markup Language）是一个 `XML` 框架，也就是一组协议，可以用来传输安全声明。比如，两台远程机器之间要通讯，为了保证安全，我们可以采用加密等措施，也可以采用 `SAML` 来传输，传输的数据以 `XML` 形式，符合 `SAML` 规范，这样我们就可以不要求两台机器采用什么样的系统，只要求能理解 `SAML` 规范即可，显然比传统的方式更好。 `SAML` 规范是一组 `Schema` 定义。

可以这么说，在 `Web Service` 领域， `schema` 就是规范，在 `Java` 领域， `API` 就是规范

3.11 X509AuthenticationFilter

`X509` 认证过滤器。你可以通过 `HttpSecurity#X509()` 来启用和配置相关功能。

3.12 AbstractPreAuthenticatedProcessingFilter

`AbstractPreAuthenticatedProcessingFilter` 处理处理经过预先认证的身份验证请求的过滤器的基类，其中认证主体已经由外部系统进行了身份验证。 目的只是从传入请求中提取主体上的必要信息，而不是对它们进行身份验证。

你可以继承该类进行具体实现并通过 `HttpSecurity#addFilter` 方法来添加个性化的 `AbstractPreAuthenticatedProcessingFilter` 。

3.13 CasAuthenticationFilter

`CAS` 单点登录认证过滤器 。依赖 Spring Security CAS 模块

3.14 OAuth2LoginAuthenticationFilter

这个需要依赖 `spring-scurity-oauth2` 相关的模块。 `OAuth2` 登录认证过滤器。处理通过 `OAuth2` 进行认证登录的逻辑。

3.15 Saml2WebSsoAuthenticationFilter

这个需要用到 `Spring Security SAML` 模块，这是一个基于 `SMAL` 的 `SSO` 单点登录认证过滤器。**关于SAML**

3.16 UsernamePasswordAuthenticationFilter

这个看过我相关文章的应该不陌生了。处理用户以及密码认证的核心过滤器。认证请求提交的 `username` 和 `password`，被封装成 `token` 进行一系列的认证，便是主要通过这个过滤器完成的，在表单认证的方法中，这是最最关键的过滤器。

你可以通过 `HttpSecurity#formLogin()` 及相关方法引入其配置对象 `FormLoginConfigurer` 来进行配置。 我们在 Spring Security **实战干货： 玩转自定义登录** 已经对其进行过个性化的配置和魔改。

3.17 ConcurrentSessionFilter

参见 3.2 ConcurrentSessionFilter。 该过滤器可能会被多次执行。

3.18 OpenIDAuthenticationFilter

基于 `OpenID` 认证协议的认证过滤器。 你需要在依赖中依赖额外的相关模块才能启用它。

3.19 DefaultLoginPageGeneratingFilter

生成默认的登录页。默认 `/login` 。

3.20 DefaultLogoutPageGeneratingFilter

生成默认的退出页。 默认 `/logout` 。

3.21 ConcurrentSessionFilter

参见 3.2 ConcurrentSessionFilter 。 该过滤器可能会被多次执行。

3.23 DigestAuthenticationFilter

`Digest` 身份验证是 `Web` 应用程序中流行的可选的身份验证机制 。 `DigestAuthenticationFilter` 能够处理 `HTTP` 头中显示的摘要式身份验证凭据。你可以通过 `HttpSecurity#addFilter()` 来启用和配置相关功能。

3.24 BasicAuthenticationFilter

和 `Digest` 身份验证一样都是 `Web` 应用程序中流行的可选的身份验证机制 。 `BasicAuthenticationFilter` 负责处理 `HTTP` 头中显示的基本身份验证凭据。这个 Spring Security 的 Spring Boot 自动配置默认是启用的 。

`BasicAuthenticationFilter` 通过 `HttpSecurity#httpBasic()` 及相关方法引入其配置对象 `HttpBasicConfigurer` 来进行配置。

3.25 RequestCacheAwareFilter

用于用户认证成功后，重新恢复因为登录被打断的请求。当匿名访问一个需要授权的资源时。会跳转到认证处理逻辑，此时请求被缓存。在认证逻辑处理完毕后，从缓存中获取最开始的资源请求进行再次请求。

`RequestCacheAwareFilter` 通过 `HttpScurity#requestCache()` 及相关方法引入其配置对象 `RequestCacheConfigurer` 来进行配置。

3.26 SecurityContextHolderAwareRequestFilter

用来 实现 `j2ee` 中 `Servlet Api` 一些接口方法, 比如 `getRemoteUser` 方法、 `isUserInRole` 方法，在使用 Spring Security 时其实就是通过这个过滤器来实现的。

`SecurityContextHolderAwareRequestFilter` 通过 `HttpSecurity.servletApi()` 及相关方法引入其配置对象 `ServletApiConfigurer` 来进行配置。

3.27 JaasApiIntegrationFilter

适用于 `JAAS` （`Java` 认证授权服务）。 如果 `SecurityContextHolder` 中拥有的 `Authentication` 是一个 `JaasAuthenticationToken` ，那么该 `JaasApiIntegrationFilter` 将使用包含在 `JaasAuthenticationToken` 中的 `Subject` 继续执行 `FilterChain` 。

3.28 RememberMeAuthenticationFilter

处理 `记住我` 功能的过滤器。

`RememberMeAuthenticationFilter` 通过 `HttpSecurity.rememberMe()` 及相关方法引入其配置对象 `RememberMeConfigurer` 来进行配置。

3.29 AnonymousAuthenticationFilter

匿名认证过滤器。**对于 `Spring Security` 来说，所有对资源的访问都是有 `Authentication` 的。对于无需登录（`UsernamePasswordAuthenticationFilter` ）直接可以访问的资源，会授予其匿名用户身份。**

`AnonymousAuthenticationFilter` 通过 `HttpSecurity.anonymous()` 及相关方法引入其配置对象 `AnonymousConfigurer` 来进行配置。

3.30 SessionManagementFilter

`Session` 管理器过滤器，内部维护了一个 `SessionAuthenticationStrategy` 用于管理 `Session` 。

`SessionManagementFilter` 通过 `HttpScurity#sessionManagement()` 及相关方法引入其配置对象 `SessionManagementConfigurer` 来进行配置。

3.31 ExceptionTranslationFilter

主要来传输异常事件，还记得之前我们见过的 `DefaultAuthenticationEventPublisher` 吗?

3.32 FilterSecurityInterceptor

这个过滤器决定了访问特定路径应该具备的权限，访问的用户的角色，权限是什么？访问的路径需要什么样的角色和权限？这些判断和处理都是由该类进行的。**如果你要实现动态权限控制就必须研究该类**。

3.33 SwitchUserFilter

`SwitchUserFilter` 是用来做账户切换的。默认的切换账号的 `url` 为 `/login/impersonate`，默认注销切换账号的 `url` 为 `/logout/impersonate`，默认的账号参数为 `username`。

你可以通过此类实现自定义的账户切换。

4. 总结

所有内置的 31个过滤器作用都讲解完了，**有些默认已经启用。有一些需要引入特定的包并且对 `HttpSecurity` 进行配置才会生效。而且它们的顺序是既定的。**只有你了解这些过滤器你才能基于业务深度定制 Spring Security。

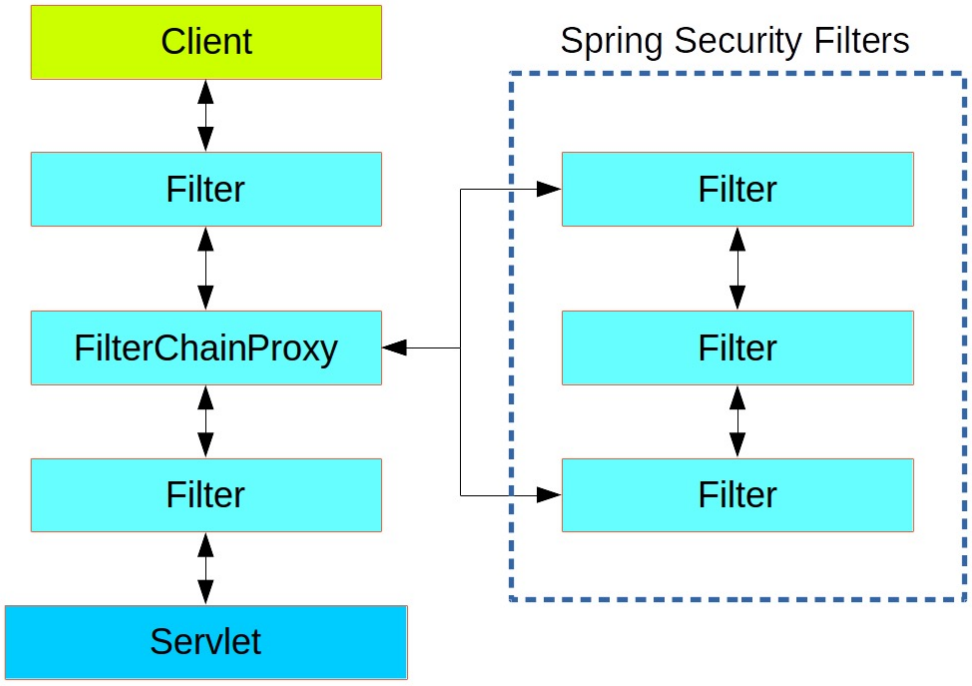
Spring Security中的过滤器链及其机制

1. 前言

过滤器作为 Spring Security 的重中之重，我们需要了解其中的机制。这样我们才能根据业务需求的变化进行定制。今天来探讨一下 Spring Security 中的过滤器链机制

2. Spring Security 过滤器链

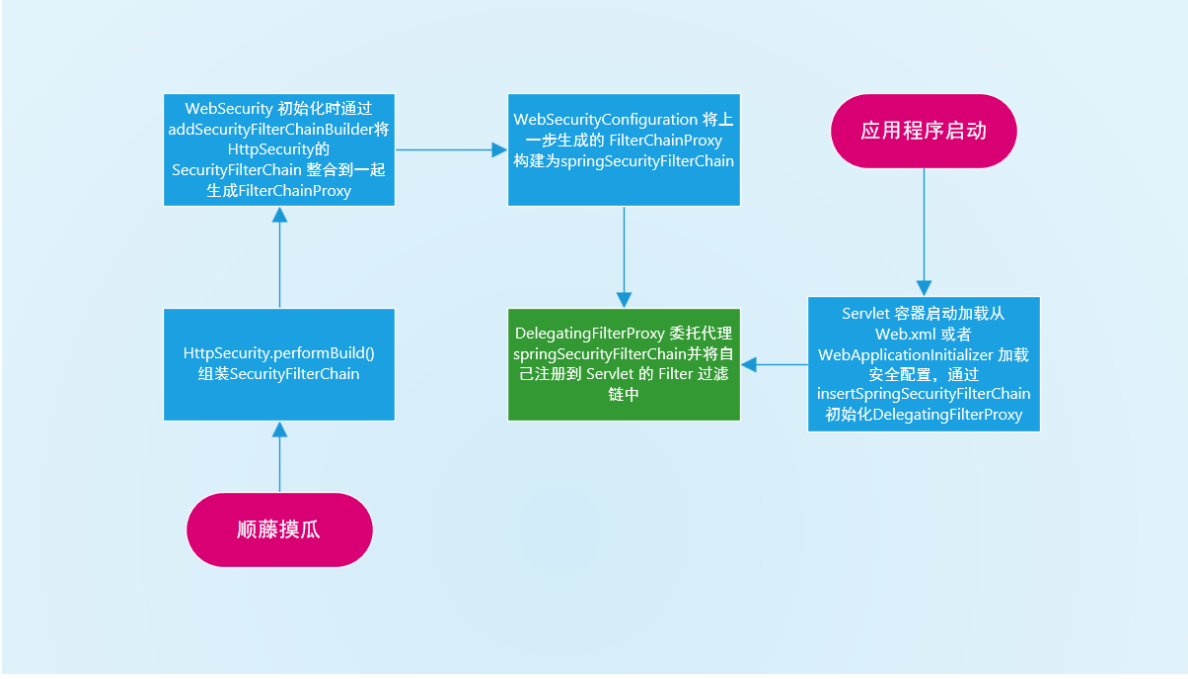
客户端（APP 和后台管理客户端）向应用程序发送请求，然后应用根据请求的 URI 的路径来确定该请求的过滤器链（Filter）以及最终的具体 Servlet 控制器（Controller）。



从上图我们可以看出 Spring Security 以一个单 Filter（FilterChainProxy） 存在于整个过滤器链中，而这个 `FilterChainProxy` 实际内部代理着众多的 Spring Security Filter 。这简直就是套娃啊！

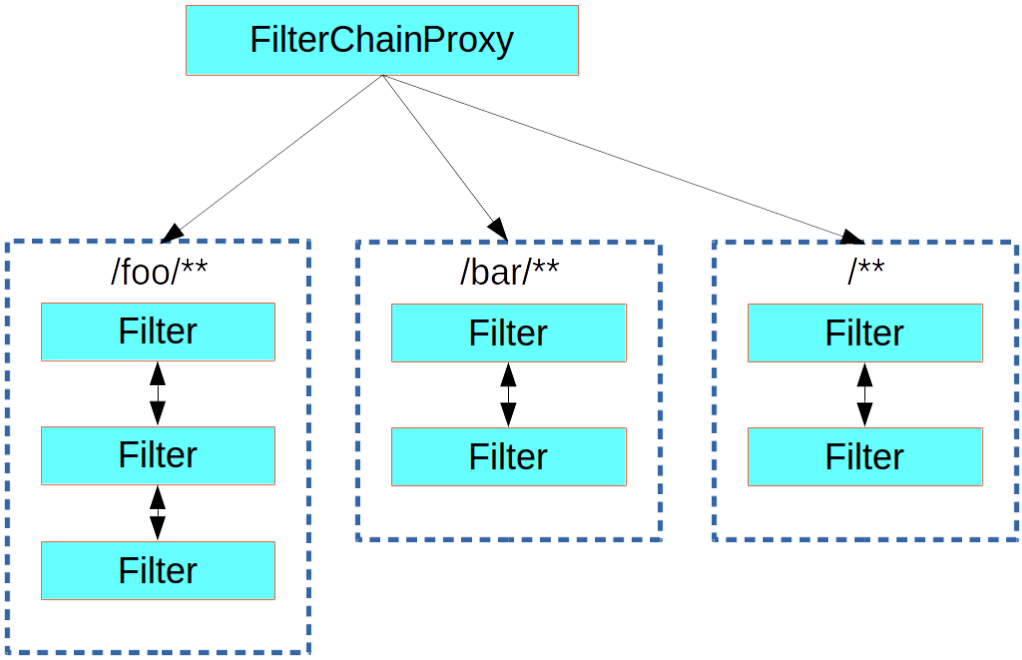
2.1 过滤器链的形成过程

再多说一点 Filter 们的初始化过程，首先 Filter 们按照一定的顺序被 `SecurityBuilder` 的实现来组装为 `SecurityFilterChain` ，然后通过 `WebSecurity` 注入到 `FilterChainProxy` 中去，接着 `FilterChainProxy` 又在 `WebSecurityConfiguration` 中以 `springSecurityFilterChain` 的名称注册为 Spring Bean 。实际上还有一个隐藏层 `DelegatingFilterProxy` 代理了 `springSecurityFilterChain` 注入到最后整个 Servlet 过滤器链中。 简单画了个图;



> 事实上 Spring Security 的内置 Filter 对于 Spring IoC 容器来说都是不可见的。

Spring Security 允许有多 条过滤器链并行， Spring Security 的 `FilterChainProxy` 可以代理多条过滤器链并根据不同的 URI 匹配策略进行分发。但是每个请求每次只能被分发到一条过滤器链。如下图所示：



关于 Filter 的其它细节可以通过 **相关文章** 了解。

> 实际每条过滤链 就是一个 `SecurityFilterChain`

4. 总结

今天我们通过对 Spring Security 中 过滤器链机制，对于深入学习 Spring Security 有着至关重要的意义。 有什么问题和心得请到微信公众号： **码农小胖哥** 留言反馈。

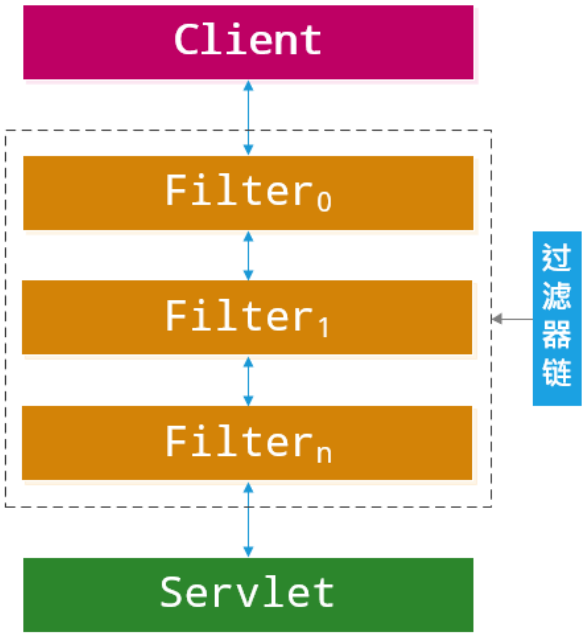
图解Spring Security的过滤器体系

1. 前言

我在 Spring Security **实战干货：内置 Filter 全解析** 对Spring Security的内置过滤器进行了罗列，但是Spring Security真正的过滤器体系才是我们了解它是如何进行"认证"、“授权”、“防止利用漏洞”的关键。

2. Servlet Filter体系

这里我们以Servlet Web为讨论目标，Reactive Web暂不讨论。我们先来看下最基础的Servlet体系，在Servlet体系中客户端发起一个请求过程是经过0到N个`Filter`然后交给`Servlet`处理。



`Filter`不但可以修改`HttpServletRequest`和`HttpServletResponse`，可以让我们在请求响应的前后做一些事情，甚至可以终止过滤器链`FilterChain`的传递。

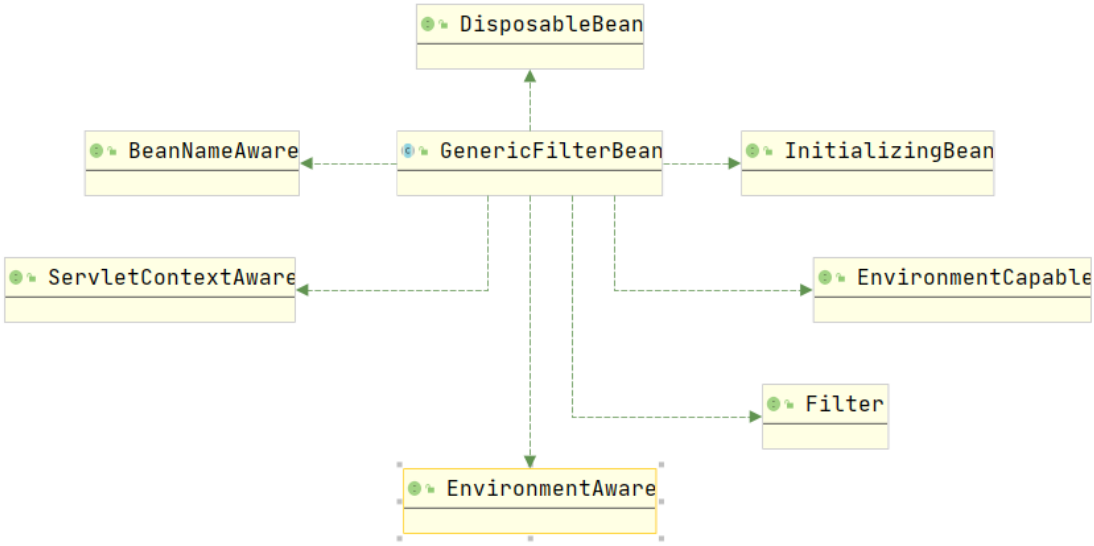
```
 public void doFilter(ServletRequest request, ServletResponse response,
FilterChain chain) {
    //  请求被servlet 处理前
     if(condition){
       // 根据条件来进入下一个过滤器
         chain.doFilter(request, response);
     }
    // 请求被执行完毕后处理一些事情
}
```

由于`Filter`仅影响下游Filters和Servlet，因此每个`Filter`调用的顺序非常重要。Spring Security正是根据这个个特性来实现一系列的安全功能。接下来我们来看看它们是如何结合的。

3. GenericFilterBean

在该系列的文章开篇我对**Spring Security和Shiro进行了简单的对比**。Spring Security利用了Spring IOC和AOP的特性而无法脱离Spring独立存在，而Apache Shiro可以独立存在。所以今天我们要一探究竟，看看他们是如何结合的。

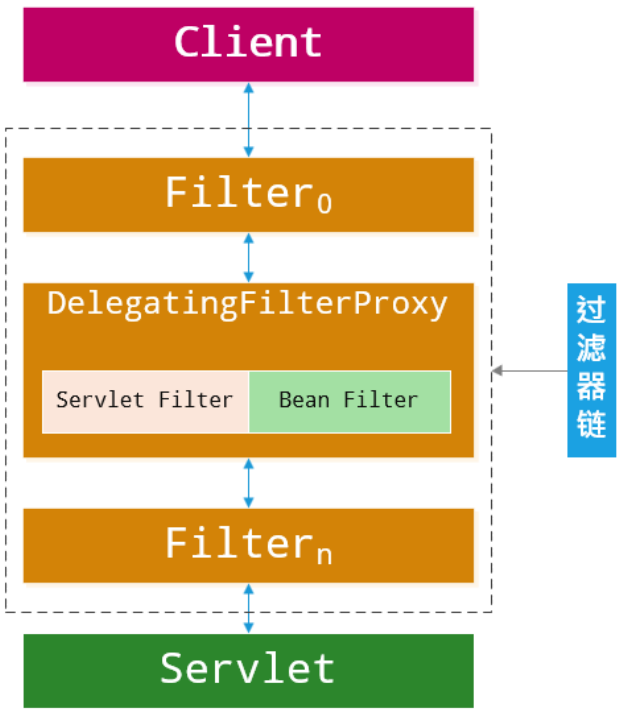
Spring结合Servlet Filter自然是要为Servlet Filter注入Spring Bean的特性，所以就搞出了一个抽象Filter Bean，这个抽象过滤器`GenericFilterBean`并不是在Spring Security下，而是Spring Web体系中，类图如下：



从类图上看 `Filter` 接口已经被注入了多个Spring Bean的特性，纳入了Spring Bean生命周期，使得Spring IoC容器能够充分的管理 `Filter` 。

4. DelegatingFilterProxy

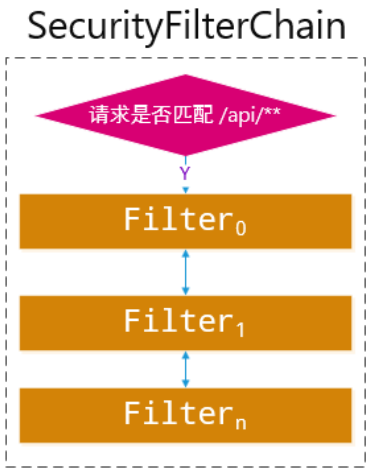
我们希望Servlet能够按照它自己的标准来注册到过滤器链中工作，但是同时也希望它能够被Spring IoC管理，所以Spring提供了一个 `GenericFilterBean` 的实现 `DelegatingFilterProxy` 。我们可以将原生的Servlet Filter或者Spring Bean Filter委托给 `DelegatingFilterProxy` ，然后在结合到Servlet FilterChain中。



5. SecurityFilterChain

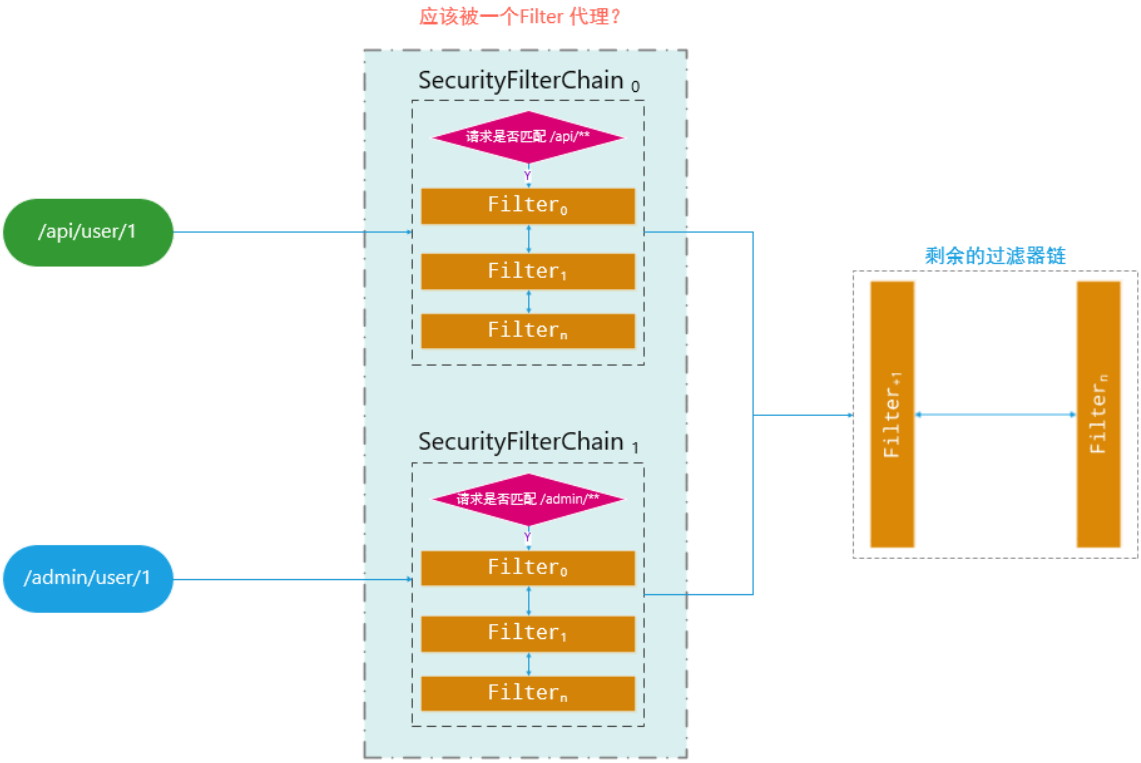
针对不同符合 Ant Pattern 的请求可能会走不同的过滤器链，比如登录会去验证，然后返回登录结果；管理后台的接口走后台的安全逻辑，应用客户端的接口走客户端的安全逻辑。 Spring Security提供了一个 `SecurityFilterChain` 接口来满足被匹配 `HttpServletRequest` 走特定的过滤器链的需求。

```
public interface SecurityFilterChain {
    // 判断请求 是否符合该过滤器链的要求
   boolean matches(HttpServletRequest request);
    // 对应的过滤器链
   List<Filter> getFilters();
}
```



6. FilterChainProxy

不同的 `SecurityFilterChain` 应该是互斥而且平等的，它们之间不应该是上下游关系。

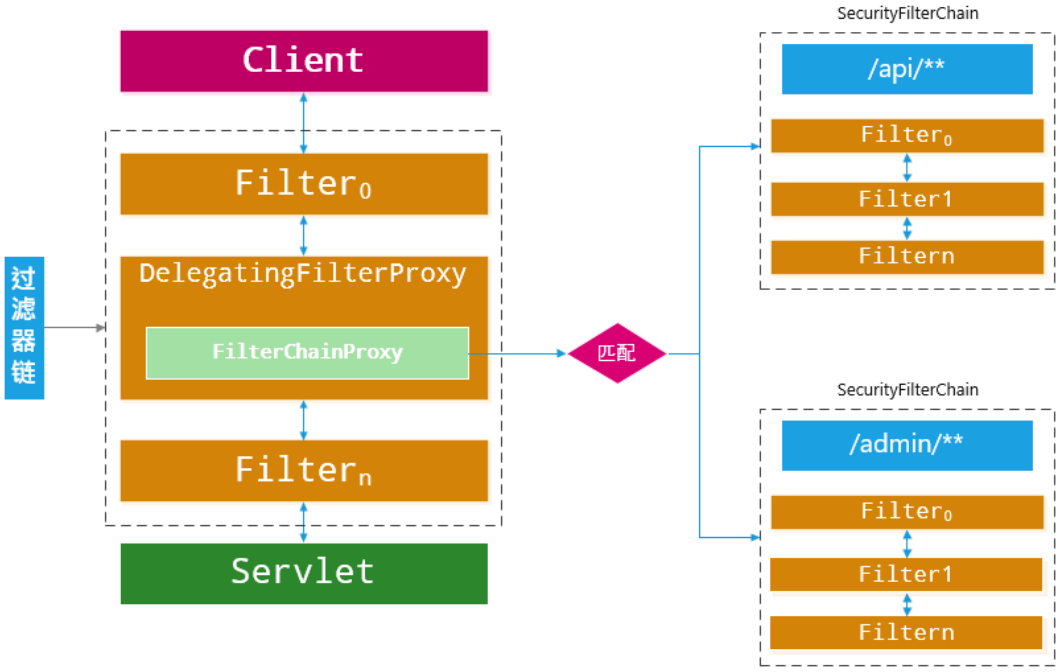


如上图请求被匹配到不同的 `SecurityFilterChain` 然后在执行剩余的过滤器链。它们经过 `SecurityFilterChain` 的总流程是相似的，而且有些时候特定的一些 `SecurityFilterChain` 也需要被集中管理来实现特定一揽子的请求的过滤逻辑。所以就有了另外一个 `GenericFilterBean` 实现来做这个事情，它就是 `FilterChainProxy` 。它的作用就是拦截符合条件的请求，然后根据请求筛选出符合要求的 `SecurityFilterChain` ，然后链式的执行这些**Filter**，最后继续执行剩下的 `FilterChain` 。

> 扩展阅读： **Spring Security 过滤器链**

7. 总结

结合上面，最终上述这些概念的关系彻底搞清楚了，搞清楚过滤器的运作模式对于学习和使用**Spring Security**至关重要。



多多关注微信公众号：**码农小胖哥** 获取更多的技术干货。

实现自定义退出登录

1. 前言

前面对 **Spring Security** 所有内置的 **Filter** 进行了介绍。今天我们来实战如何安全退出应用程序。

2. 我们使用 Spring Security 登录后都做了什么

这个问题我们必须搞清楚！一般登录后，服务端会给用户发一个凭证。常见有以下的两种：

- 基于 `Session` 客户端会存 `cookie` 来保存一个 `sessionId` ，服务端存一个 `Session` 。
- 基于 `token` 客户端存一个 `token` 串，服务端会在缓存中存一个用来校验此 `token` 的信息。

2. 退出登录需要我们做什么

1. 当前的用户登录状态失效。这就需要我们清除服务端的用户状态。
2. 退出登录接口并不是 `permitAll` ， 只有携带对应用户的凭证才退出。
3. 将退出结果返回给请求方。
4. 退出登录后用户可以通过重新登录来认证该用户。

3. Spring Security 中的退出登录

接下来我们来分析并实战 **如何定制退出登录逻辑**。首先我们要了解 `LogoutFilter` 。

3.1 LogoutFilter

通过 Spring Security 实战干货：内置 Filter 全解析 我们知道退出登录逻辑是由过滤器 `LogoutFilter` 来执行的。 它持有三个接口类型的属性：

1. `RequestMatcher logoutRequestMatcher` 这个用来拦截退出请求的 `URL`
2. `LogoutHandler handler` 用来处理退出的具体逻辑
3. `LogoutSuccessHandler logoutSuccessHandler` 退出成功后执行的逻辑

我们通过对以上三个接口的实现就能实现我们自定义的退出逻辑。

3.2 LogoutConfigurer

我们一般不会直接操作 `LogoutFilter` ，而是通过 `LogoutConfigurer` 来配置 `LogoutFilter` 。 你可以通过 `HttpSecurity#logout()` 方法来初始化一个 `LogoutConfigurer` 。 接下来我们来实战操作一下。

3.2.1 实现自定义退出登录请求URL

`LogoutConfigurer` 提供了 `logoutRequestMatcher(RequestMatcher logoutRequestMatcher)` 、 `logoutUrl(Sring logoutUrl)` 两种方式来定义退出登录请求的 `URL` 。它们作用是相同的，你选择其中一种方式即可。

3.2.2 处理具体的逻辑

默认情况下 Spring Security 是基于 `Session` 的。 `LogoutConfigurer` 提供了一些直接配置来满足你的需要。如下：

- `clearAuthentication(boolean clearAuthentication)` 是否在退出时清除当前用户的认证信息
- `deleteCookies(String... cookieNamesToClear)` 删除指定的 `cookies`
- `invalidateHttpSession(boolean invalidateHttpSession)` 是否移除 `HttpSession`

如果上面满足不了你的需要就需要你来定制 `LogoutHandler` 了。

3.2.3 退出成功逻辑

- `logoutSuccessUrl(String logoutSuccessUrl)` 退出成功后会被重定向到此 `URL` ，**你可以写一个Controller来完成最终返回，但是需要支持 `GET` 请求和 匿名访问** 。 通过 `setDefaultTargetUrl` 方法注入到 `LogoutSuccessHandler`
- `defaultLogoutSuccessHandlerFor(LogoutSuccessHandler handler, RequestMatcher preferredMatcher)` 用来构造默认的 `LogoutSuccessHandler` 我们可以通过添加多个来实现从不同 `URL` 退出执行不同的逻辑。
- `LogoutSuccessHandler logoutSuccessHandler` 退出成功后执行的逻辑的抽象根本接口。

3.3 Spring Security 退出登录实战

现在前后端分离比较多，退出后返回json。 而且只有用户在线才能退出登录。否则不能进行退出操作。我们采用实现 `LogoutHandler` 和 `LogoutSuccessHandler` 接口这种编程的方式来配置 。退出请求的 `url` 依然通过 `LogoutConfigurer#logoutUrl(String logoutUrl)` 来定义。

3.3.1 自定义 LogoutHandler

默认情况下清除认证信息 （`invalidateHttpSession`），和Session 失效 （`invalidateHttpSession`） 已经由内置的 `SecurityContextLogoutHandler` 来完成。我们自定义的 `LogoutHandler` 会在 `SecurityContextLogoutHandler` 来执行。

```
1     @Slf4j
2     public class CustomLogoutHandler implements LogoutHandler {
3         @Override
4         public void logout(HttpServletRequest request, HttpServletResponse response,
    Authentication authentication) {
5             User user = (User) authentication.getPrincipal();
6             String username = user.getUsername();
7             log.info("username: {}  is offline now", username);
8         }
9     }
```

以上是我们实现的 `LogoutHandler` 。 我们可以从 `logout` 方法的 `authentication` 变量中 获取当前用户信息。你可以通过这个来实现你具体想要的业务。比如记录用户下线退出时间、IP 等等。

3.3.2 自定义 LogoutSuccessHandler

如果我们实现了自定义的 `LogoutSuccessHandler` 就不必要设置 `LogoutConfigurer#logoutSuccessUrl(String logoutSuccessUrl)` 了。该处理器处理后会响应给前端。你可以转发到其它控制器。重定向到登录页面，也可以自行实现其它 `MediaType` ,可以是 `json` 或者页面

```
1      @Slf4j
2      public class CustomLogoutSuccessHandler implements LogoutSuccessHandler {
3          @Override
4          public void onLogoutSuccess(HttpServletRequest request,
    HttpServletResponse response, Authentication authentication) throws IOException,
    ServletException {
5              User user = (User) authentication.getPrincipal();
6              String username = user.getUsername();
7              log.info("username: {}  is offline now", username);
8
9
```

```
            responseJsonWriter(response, RestBody.ok("退出成功"));
        }

        private static void responseJsonWriter(HttpServletResponse response, Rest
rest) throws IOException {
            response.setStatus(HttpServletResponse.SC_OK);
            response.setCharacterEncoding("utf-8");
            response.setContentType(MediaType.APPLICATION_JSON_VALUE);
            ObjectMapper objectMapper = new ObjectMapper();
            String resBody = objectMapper.writeValueAsString(rest);
            PrintWriter printWriter = response.getWriter();
            printWriter.print(resBody);
            printWriter.flush();
            printWriter.close();
        }
    }
```

3.3.4 自定义退出的 Spring Security 配置

为了方便调试我 注释掉了我们 **实现的自定义登录**，你可以通过 `http:localhost:8080/login` 来登录，然后通过 `http:localhost:8080/logout` 测试退出。

```
        @Override
           protected void configure(HttpSecurity http) throws Exception {
               http.csrf().disable()
                       .cors()
                       .and()
                       .authorizeRequests().anyRequest().authenticated()
                       .and()
  //                     .addFilterBefore(preLoginFilter,
UsernamePasswordAuthenticationFilter.class)
                       // 登录

.formLogin().loginProcessingUrl(LOGIN_PROCESSING_URL).successForwardUrl("/login/s
uccess").failureForwardUrl("/login/failure")
                       .and().logout().addLogoutHandler(new
CustomLogoutHandler()).logoutSuccessHandler(new CustomLogoutSuccessHandler());

           }
```

4. 总结

本篇 我们实现了 在 **Spring Security** 下的自定义退出逻辑。相对比较简单，你可以根据你的业务需要来实现你的退出逻辑。有什么疑问可以通过 关注公众号：**码农小胖哥** 来私信提问 。相关DEMO代码也可以通过关注后回复 `ss04` 获取。

Web无状态会话Token技术JWT

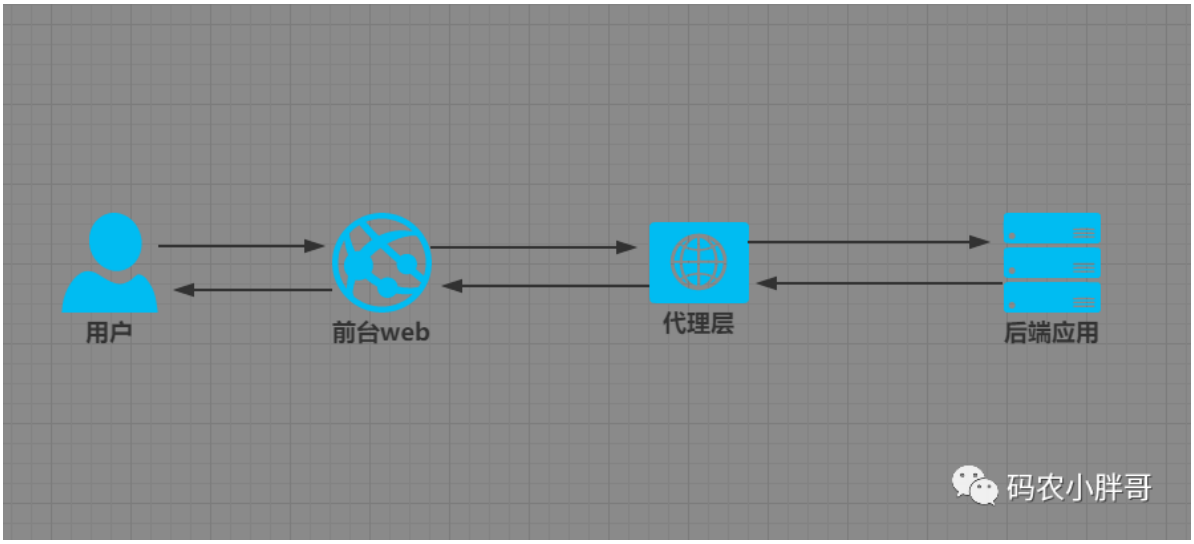
1. 前言

目前web开发前后端已经算非常的普及了。前后端分离要求我们对用户会话状态要进行一个无状态处理。我们都知道通常管理用户会话是session。用户每次从服务器认证成功后，服务器会发送一个sessionid给用户，session是保存在服务端 的，服务器通过session辨别用户，然后做权限认证等。那如何才知道用户的session是哪个？这时候cookie就出场了，浏览器第一次与服务器建立连接的时候，服务器会生成一个sessionid返回浏览器，浏览器把这个sessionid存储到cookie当中，以后每次发起请求都会在请求头cookie中带上这个sessionid信息，所以服务器就是根据这个sessionid作为索引获取到具体session。

2. 痛点

上面的场景会有一个痛点。对于前后端分离来说。比如前端都是部署在一台服务器的nginx上，后端部署在另一台服务器的web容器上。甚至 前端不能直接访问后端，中间还加了一层代理层。

大概如下所示：



也就是说前后端分离在应用解耦后增加了部署的复杂性。通常用户一次请求就要转发多次。如果用session 每次携带sessionid 到服务器，服务器还要查询用户信息。同时如果用户很多。这些信息存储在服务器内存中，给服务器增加负担。还有就是CSRF（跨站伪造请求攻击）攻击，session是基于cookie进行用户识别的, cookie如果被截获，用户就会很容易受到跨站请求伪造的攻击。还有就是sessionid就是一个特征值，表达的信息不够丰富。不容易扩展。而且如果你后端应用是多节点部署。那么就需要实现session共享机制。不方便集群应用。

3. 什么是JWT

所以JSON WEB TOKEN（以下称JWT）可以解决上面的问题。JWT还是一种token。token 是服务器颁发给客户端的。就像户籍管理部门给你发的身份证一样。你拿着这个证件就能去其他部门办事。其他部门验证你这个身份证是否过期，是否真假。不用每次都让户籍来认可。同时token 天然防止CSRF攻击。而且JWT可以携带一些不敏感的用户信息。这样服务器不用每次都去查询用户信息。开箱即用，方便服务器处理鉴权逻辑。

是为了在**网络应用环境间传递声明**而执行的一种基于JSON的开放标准（(RFC 7519)。该token被设计为紧凑且安全的，**特别适用于分布式站点的单点登录（SSO）场景**。

JWT的声明一般**被用来在身份提供者和服务提供者间传递被认证的用户身份信息**，以便于从资源服务器获取资源，也可以增加一些额外的其它业务逻辑所必须的声明信息，该token也可直接被用于认证，也可被加密。

JWT的特点：

- 简洁(Compact): 可以通过URL，POST参数或者在HTTP header发送，因为数据量小，传输速度也很快
- 自包含(Self-contained)：负载中包含了所有用户所需要的信息，避免了多次查询数据库或 缓存

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

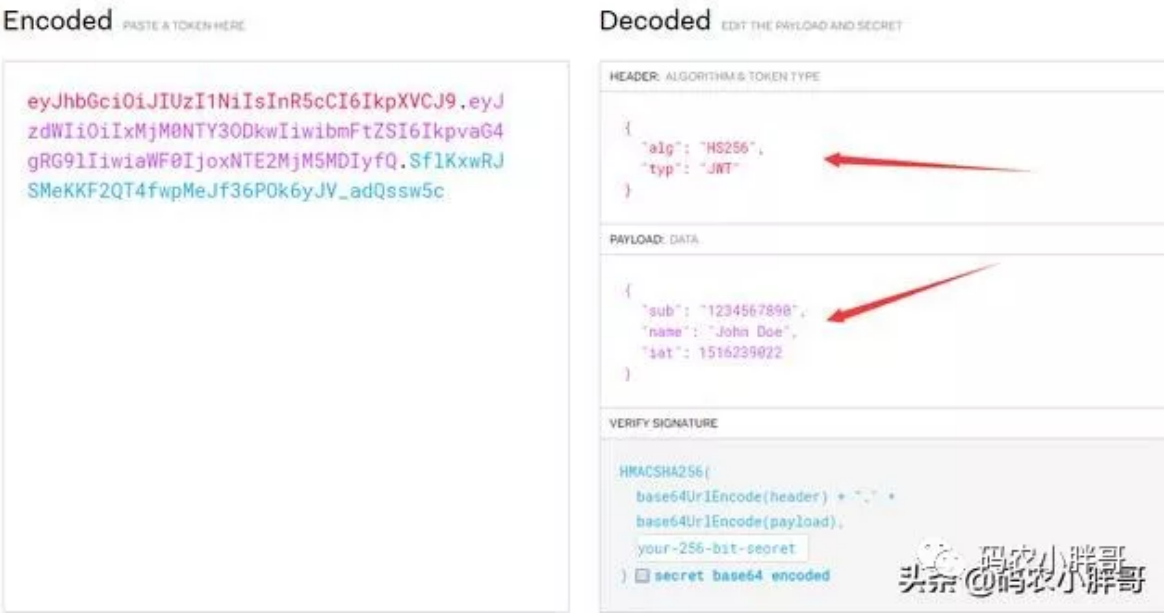
JWT消息结构：

JWT有3个组成部分，分别是:

- 头部（header) 声明类型以及加密算法 如 {"alg":"HS256","typ":"JWT"} 用Base64进行了处理
- 载荷（payload) 携带一些用户身份信息，用户id，颁发机构，颁发时间，过期时间等。用Base64进行了处理。这一段其实是明文，所以一定不要放敏感信息。
- 签证（signature) 签名信息，使用了自定义的一个密钥然后加密后的结果，目的就是为了保证签名的信息没有被别人改过，这个一般是让服务器验证的。

从上面的例子可以看出来JWT的规则是这样的

..三部分通过"."进行拼接 。jwt.io提供解析的方法 我们可以拿上面那个token去玩一玩



所以JWT不是简单的token，比session+cookie机制更加丰富。应用场景更加丰富。

JWT并不是完美的。JWT不足之处：

1. 比如说有可能一个用户同时出现两个可用的token情况。
2. 还有如果失效过期了如何进行续期的问题。
3. 同样会出现token被盗用的问题。
4. 注销如何让token失效的问题。
5. 用户信息修改让token同步的问题。

JWT知识扩展JOSE

1.前言

JSON Web Token (JWT) 其实目前已经广为软件开发者所熟知了，但是 JOSE (Javascript Object Signing and Encryption) 却鲜有人知道，我第一次知道它是在 Spring Security 的官方文档中，它改变了我对 JWT 的一些认识。目前国内能找到相关中文资料不是太多。所以我觉得有必要归纳一下。

2. JOSE 概述

JOSE 是一种旨在提供在各方之间安全传递声明（claims）的方法的规范集。我们常用的 JWT 就包含了允许客户端访问特定应用下特定资源的声明。JOSE 制定了一系列的规范来达到此目的。目前该规范还在不断的发展，我们常用的包含以下几个 RFC：

- **JWS（RFC 7515）** -JSON Web签名，描述生成和处理签名消息
- **JWE（RFC 7516）** -JSON Web加密，描述了保护和处理加密 消息
- **JWK（RFC 7517）** -JSON Web密钥，描述 Javascript 对象签名和加密中加密密钥的 格式和处理
- **JWA（RFC 7518）** -JSON Web算法，描述了 Javascript 对象签名和加密中使用的 加密 算法
- **JWT（RFC 7519）** -JSON Web令牌，描述以 JSON 编码并由 JWS 或 JWE 保护的声明的表示形式

3. 我们都看错了 JWT

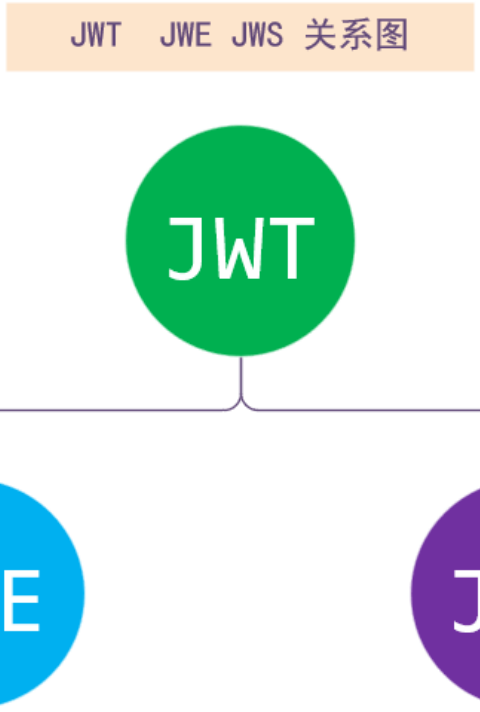
看了对 JWT 的描述中提到 “令牌以 JWS 或者 JWE 声明表示”。莫非我之前的认知是错误的吗？ 找了一些官方的资料研究了一番后，确实我之前的认知是不够全面的。

官方定义：

> JSON Web Token (JWT) is a compact URL-safe means of representing claims to be transferred between two parties

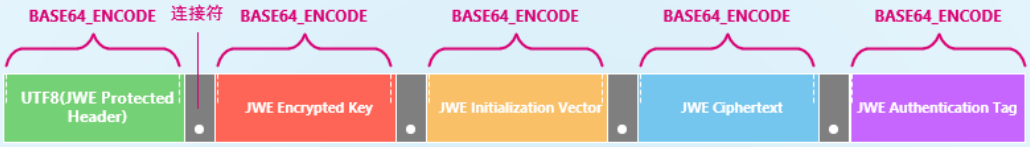
直译过来：JSON Web令牌（JWT）是一种紧凑的URL安全方法，用于表示要在两方之间转移的声明。

也就是说我们通常说的 JWT 实际上是一个对声明进行 JOSE 处理方式的统称。我们之前用的应该叫 JWS(JSON Web Signature)，是 JWT 的一种实现，除了 JWS，JWT 还有另一种实现 JWE(JSON Web Encryption) 。它们之间的关系应该是这样的：



4. 什么是 JWE

JWS 我们就不说了，就是通常我们所说的 JWT。包括之前我在 Spring Security 实战干货 中所涉及到的 JWT 都是 JWS。我们来说一下 JWE 。JWS 仅仅是对声明(claims)作了签名，保证了其不被篡改，但是其 payload(**中段负载**) 信息是暴露的。也就是 JWS 仅仅能保证数据的完整性而不能保证数据不被泄露。所以我以前也说过它不适合传递敏感数据。
JWE 的出现就是为了解决这个问题的。具体的可以看下图：



JWE 生成步骤：

- 第一部分与JWS Header 相同 标识加密算法和类型
- 生成一个随机的256位的 Content Encryption Key
- 使用RSAES-PKCS1-v1_5算法使用Token接收者的公钥对CEK进行加密，以生成JWE加密密钥
- 生成随机的128位JWE初始化向量
- 将附加身份凭据参数放入Header
- 使用CEK作为加密密钥，JWE初始化向量和上面的附加身份凭据参数值，使用AES_128_CBC_HMAC_SHA_256算法对纯文本执行身份验证加密。生成密文Ciphertext,同时会随之生成一个128位的认证标记Authentication Tag
- 将生成的五段加密串使用BASE64进行编码然后用 “.” 按上图顺序连起来组成Token

从上面可以看出 JWE 的生成非常繁琐，作为 Token 可能比较消耗资源和耗时。用作安全的数据传输途径应该不错。

5. Spring Security jose 相关

这里需要简单提一下 Spring Security 提供了 JOSE 有关的类库 `spring-security-oauth2-jose` ，你可以使用该类库来使用 JOSE 。如果 Java 开发者要在 Spring Security 安全框架中使用 OAuth2.0 ，这个类库也是需要研究一下的。

6. 总结

今天我们对 JOSE 这个相对陌生的概念进行了认识，对 JOSE 规范集中的几个重要的 RFC 进行了列举。对之前的局限性认识也进行了纠正。为我们后续的 OAuth2.0 相关学习进行了铺垫。

构造JWT Token生成服务

1. 前言

Json Web Token （`JWT`） 近几年是前后端分离常用的 Token 技术，是目前最流行的跨域身份验证解决方案。你可以通过文章 **一文了解web无状态会话token技术JWT** 来了解 `JWT` 。今天我们来手写一个通用的 `JWT` 服务。DEMO 获取方式在文末，实现在 `jwt` 相关包下

> 目前 spring-security-jwt 项目已经过期，但是不影响使用，如果有兴趣可以去Spring相关项目中去扒最新的依赖。

2. spring-security-jwt

`spring-security-jwt` 是 Spring Security Crypto 提供的 `JWT` 工具包 。

```
    <dependency>
          <groupId>org.springframework.security</groupId>
          <artifactId>spring-security-jwt</artifactId>
          <version>${spring-security-jwt.version}</version>
    </dependency>
```

核心类只有一个: `org.springframework.security.jwt.JwtHelper` 。它提供了两个非常有用的静态方法。

3. JWT 编码

`JwtHelper` 提供的第一个静态方法就是 `encode(CharSequence content, Signer signer)` 这个是用来生成jwt的方法 需要指定 payload 跟 signer 签名算法。 payload 存放了一些可用的**不敏感**信息:

- `iss` jwt签发者

- `sub` jwt所面向的用户
- `aud` 接收jwt的一方
- `iat` jwt的签发时间
- `exp` jwt的过期时间，这个过期时间必须要大于签发时间 `iat`
- `jti` jwt的唯一身份标识，主要用来作为一次性token,从而回避重放攻击

除了以上提供的基本信息外，我们可以定义一些我们需要传递的信息，比如目标用户的权限集 等等。**切记不要传递密码等敏感信息** ，因为 `JWT` 的前两段都是用了 `BASE64` 编码，几乎算是明文了。

3.1 构建 JWT 中的 payload

我们先来构建 **payload**：

```
/**
 * 构建 jwt payload
 *
 * @author Felordcn
 * @since 11:27 2019/10/25
 **/
public class JwtPayloadBuilder {

    private Map<String, String> payload = new HashMap<>();
    /**
     * 附加的属性
     */
    private Map<String, String> additional;
    /**
     * jwt签发者
     **/
    private String iss;
    /**
     * jwt所面向的用户
     **/
    private String sub;
    /**
     * 接收jwt的一方
     **/
    private String aud;
    /**
     * jwt的过期时间，这个过期时间必须要大于签发时间
     **/
    private LocalDateTime exp;
    /**
     * jwt的签发时间
     **/
    private LocalDateTime iat = LocalDateTime.now();
    /**
     * 权限集
     */
    private Set<String> roles = new HashSet<>();
    /**
     * jwt的唯一身份标识，主要用来作为一次性token,从而回避重放攻击
     **/
    private String jti = IdUtil.simpleUUID();

    public JwtPayloadBuilder iss(String iss) {
        this.iss = iss;
```

```
45              return this;
46          }
47
48
49          public JwtPayloadBuilder sub(String sub) {
50              this.sub = sub;
51              return this;
52          }
53
54          public JwtPayloadBuilder aud(String aud) {
55              this.aud = aud;
56              return this;
57          }
58
59
60          public JwtPayloadBuilder roles(Set<String> roles) {
61              this.roles = roles;
62              return this;
63          }
64
65          public JwtPayloadBuilder expDays(int days) {
66              Assert.isTrue(days > 0, "jwt expireDate must after now");
67              this.exp = this.iat.plusDays(days);
68              return this;
69          }
70
71          public JwtPayloadBuilder additional(Map<String, String> additional) {
72              this.additional = additional;
73              return this;
74          }
75
76          public String builder() {
77              payload.put("iss", this.iss);
78              payload.put("sub", this.sub);
79              payload.put("aud", this.aud);
80              payload.put("exp", this.exp.format(DateTimeFormatter.ofPattern("yyyy-MM-
    dd HH:mm:ss")));
81              payload.put("iat", this.iat.format(DateTimeFormatter.ofPattern("yyyy-MM-
    dd HH:mm:ss")));
82              payload.put("jti", this.jti);
83
84              if (!CollectionUtils.isEmpty(additional)) {
85                  payload.putAll(additional);
86              }
87              payload.put("roles", JSONUtil.toJsonStr(this.roles));
88              return JSONUtil.toJsonStr(JSONUtil.parse(payload));
89
90          }
91
92      }
```

通过建造类 `JwtClaimsBuilder` 我们可以很方便来构建 `JWT` 所需要的 **payload** `json` 字符串传递给 `encode(CharSequence content, Signer signer)` 中的 `content` 。

3.2 生成 RSA 密钥并进行签名

为了生成 JWT Token 我们还需要使用 RSA 算法来进行签名。 这里我们使用 JDK 提供的证书管理工具 Keytool 来生成 RSA **证书** ， 格式为 `jks` 格式。

生成证书命令参考：

```
 keytool -genkey -alias felordcn -keypass felordcn -keyalg RSA -storetype PKCS12 -keysize 1024 -validity 365 -keystore d:/keystores/felordcn.jks -storepass 123456 -dname "CN=(Felord), OU=(felordcn), O=(felordcn), L=(zz), ST=(hn), C=(cn)"

```

其中 `-alias felordcn -storepass 123456` 我们要作为配置使用要记下来。我们要使用下面定义的这个类来读取证书

```
package cn.felord.spring.security.jwt;

import org.springframework.core.io.ClassPathResource;

import java.security.KeyFactory;
import java.security.KeyPair;
import java.security.KeyStore;
import java.security.PublicKey;
import java.security.interfaces.RSAPrivateCrtKey;
import java.security.spec.RSAPublicKeySpec;

/**
 * KeyPairFactory
 *
 * @author Felordcn
 * @since 13:41 2019/10/25
 **/
class KeyPairFactory {

    private KeyStore store;

    private final Object lock = new Object();

    /**
     * 获取公私钥.
     *
     * @param keyPath  jks 文件在 resources 下的classpath
     * @param keyAlias  keytool 生成的 -alias 值  felordcn
     * @param keyPass  keytool 生成的  -storepass 值  123456
     * @return the key pair 公私钥对
     */
   KeyPair create(String keyPath, String keyAlias, String keyPass) {
        ClassPathResource resource = new ClassPathResource(keyPath);
        char[] pem = keyPass.toCharArray();
        try {
            synchronized (lock) {
                if (store == null) {
                    synchronized (lock) {
                        store = KeyStore.getInstance("jks");
                        store.load(resource.getInputStream(), pem);
                    }
                }
            }
```

```
            RSAPrivateCrtKey key = (RSAPrivateCrtKey) store.getKey(keyAlias,
pem);
            RSAPublicKeySpec spec = new RSAPublicKeySpec(key.getModulus(),
key.getPublicExponent());
            PublicKey publicKey =
KeyFactory.getInstance("RSA").generatePublic(spec);
            return new KeyPair(publicKey, key);
        } catch (Exception e) {
            throw new IllegalStateException("Cannot load keys from store: " +
resource, e);
        }

    }
 }
```

获取了 `KeyPair` 就能获取公私钥 生成 `Jwt` 的两个要素就完成了。我们可以和之前定义的 `JwtPayloadBuilder` 一起封装出生成 **Jwt Token** 的方法：

```
    private String jwtToken(String aud, int exp, Set<String> roles, Map<String,
String> additional) {
        String payload = jwtPayloadBuilder
                .iss(jwtProperties.getIss())
                .sub(jwtProperties.getSub())
                .aud(aud)
                .additional(additional)
                .roles(roles)
                .expDays(exp)
                .builder();
        RSAPrivateKey privateKey = (RSAPrivateKey) keyPair.getPrivate();

        RsaSigner signer = new RsaSigner(privateKey);
        return JwtHelper.encode(payload, signer).getEncoded();
    }
```

通常情况下 Jwt Token 都是成对出现的，一个为平常请求携带的 `accessToken`， 另一个只作为刷新 `accessToken` 之用的 `refreshToken` 。而且 `refreshToken` 的过期时间要相对长一些。当 `accessToken` 失效而 `refreshToken` 有效时，我们可以通过 `refreshToken` 来获取新的 **Jwt Token对** ； 当两个都失效就用户就必须重新登录了。

生成 **Jwt Token对** 的方法如下：

```
    public JwtTokenPair jwtTokenPair(String aud, Set<String> roles, Map<String,
String> additional) {
        String accessToken = jwtToken(aud, jwtProperties.getAccessExpDays(),
roles, additional);
        String refreshToken = jwtToken(aud, jwtProperties.getRefreshExpDays(),
roles, additional);

        JwtTokenPair jwtTokenPair = new JwtTokenPair();
        jwtTokenPair.setAccessToken(accessToken);
        jwtTokenPair.setRefreshToken(refreshToken);
        // 放入缓存
        jwtTokenStorage.put(jwtTokenPair, aud);
        return jwtTokenPair;
    }
```

通常 **Jwt Token对** 会在返回给前台的同时放入缓存中。过期策略你可以选择分开处理，也可以选择以 `refreshToken` 的过期时间为准。

4. JWT 解码以及验证

`JwtHelper` 提供的第二个静态方法是 `Jwt decodeAndVerify(String token, SignatureVerifier verifier)` 用来 验证和解码 **Jwt Token** 。我们获取到请求中的token后会解析出用户的一些信息。通过这些信息去缓存中对应的token， 然后比对并验证是否有效（包括是否过期）。

```
        /**
         * 解码 并校验签名 过期不予解析
         *
         * @param jwtToken the jwt token
         * @return the jwt claims
         */
        public JSONObject decodeAndVerify(String jwtToken) {
            Assert.hasText(jwtToken, "jwt token must not be bank");
            RSAPublicKey rsaPublicKey = (RSAPublicKey) this.keyPair.getPublic();
            SignatureVerifier rsaVerifier = new RsaVerifier(rsaPublicKey);
            Jwt jwt = JwtHelper.decodeAndVerify(jwtToken, rsaVerifier);
            String claims = jwt.getClaims();
            JSONObject jsonObject = JSONUtil.parseObj(claims);
            String exp = jsonObject.getStr(JWT_EXP_KEY);
           // 是否过期
            if (isExpired(exp)) {
                throw new IllegalStateException("jwt token is expired");
            }
            return jsonObject;
        }
```

上面我们将有效的 **Jwt Token** 中的 `payload` 解析为 **JSON对象** ， 方便后续的操作。

5. 配置

我们将 **JWT** 的可配置项抽出来放入 `JwtProperties` 如下：

```
    /**
     * Jwt 在 springboot application.yml 中的配置文件
     *
     * @author Felordcn
     * @since 15 :06 2019/10/25
     */
    @Data
    @ConfigurationProperties(prefix=JWT_PREFIX)
    public class JwtProperties {
        static final String JWT_PREFIX= "jwt.config";
        /**
         * 是否可用
         */
        private boolean enabled;
        /**
         * jks 路径
         */
```

```
18        private String keyLocation;
19        /**
20         * key alias
21         */
22        private String keyAlias;
23        /**
24         * key store pass
25         */
26        private String keyPass;
27        /**
28         * jwt签发者
29         **/
30        private String iss;
31        /**
32         * jwt所面向的用户
33         **/
34        private String sub;
35        /**
36         * access jwt token 有效天数
37         */
38        private int accessExpDays;
39        /**
40         * refresh jwt token 有效天数
41         */
42        private int refreshExpDays;
43    }
```

然后我们就可以配置 **JWT** 的 `javaConfig` 如下:

```
1     /**
2      * JwtConfiguration
3      *
4      * @author Felordcn
5      * @since 16 :54 2019/10/25
6      */
7     @EnableConfigurationProperties(JwtProperties.class)
8     @ConditionalOnProperty(prefix = "jwt.config",name = "enabled")
9     @Configuration
10    public class JwtConfiguration {
11
12
13        /**
14         * Jwt token storage .
15         *
16         * @return the jwt token storage
17         */
18        @Bean
19        public JwtTokenStorage jwtTokenStorage() {
20            return new JwtTokenCacheStorage();
21        }
22
23
24        /**
25         * Jwt token generator.
26         *
27         * @param jwtTokenStorage the jwt token storage
28         * @param jwtProperties   the jwt properties
29         * @return the jwt token generator
```

```
30          */
31         @Bean
32         public JwtTokenGenerator jwtTokenGenerator(JwtTokenStorage jwtTokenStorage,
    JwtProperties jwtProperties) {
33             return new JwtTokenGenerator(jwtTokenStorage, jwtProperties);
34         }
35 
36     }
```

然后你就可以通过 `JwtTokenGenerator` 编码/解码验证 **Jwt Token 对** ，通过 `JwtTokenStorage` 来处理 Jwt Token 缓存。缓存这里我用了Spring Cache Ehcache 来实现,你也可以切换到 Redis 。相关单元测试参见 DEMO

6. 总结

今天我们利用 `spring-security-jwt` 手写了一套 JWT 逻辑。无论对你后续结合 Spring Security 还是 Shiro 都十分有借鉴意义。下一篇我们会讲解 JWT 结合Spring Security ,敬请关注公众号：**码农小胖哥** 来及时获取资料。

本次 DEMO 也可通过关注公众号回复 day05 获取。

实现Spring Security登录后返回JWT

1. 前言

欢迎阅读 Spring Security **实战干货 系列文章** ， **上一文** 我们实现了 JWT 工具。本篇我们将一起探讨如何将 JWT 与 Spring Security 结合起来，在认证成功后不再跳转到指定页面而是直接返回 JWT Token 。本文的 `DEMO` **可通过文末的方式获取**

2. 流程

JWT 适用于前后端分离。我们在登录成功后不在跳转到首页，将会直接返回 JWT Token 对（DEMO中为 `JwtTokenPair` ），登录失败后返回认证失败相关的信息。

3. 实现登录成功/失败返回逻辑

如果你看过 Spring Security **实战干货： 玩转自定义登录** 将非常容易理解下面的做法。

3.1 AuthenticationSuccessHandler 返回 JWT Token

`AuthenticationSuccessHandler` 用于处理登录成功后的逻辑，我们编写实现并注入 Spring IoC 容器：

```
1         /**
2          * 处理登录成功后返回 JWT Token 对.
3          *
4          * @param jwtTokenGenerator the jwt token generator
```

```
         * @return the authentication success handler
         */
        @Bean
        public AuthenticationSuccessHandler
    authenticationSuccessHandler(JwtTokenGenerator jwtTokenGenerator) {
            return (request, response, authentication) -> {
                if (response.isCommitted()) {
                    log.debug("Response has already been committed");
                    return;
                }
                Map<String, Object> map = new HashMap<>(5);
                map.put("time",
    LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss")));
                map.put("flag", "success_login");
                User principal = (User) authentication.getPrincipal();

                String username = principal.getUsername();
                Collection<GrantedAuthority> authorities =
    principal.getAuthorities();
                Set<String> roles = new HashSet<>();
                if (CollectionUtil.isNotEmpty(authorities)) {
                    for (GrantedAuthority authority : authorities) {
                        String roleName = authority.getAuthority();
                        roles.add(roleName);
                    }
                }

                JwtTokenPair jwtTokenPair = jwtTokenGenerator.jwtTokenPair(username,
    roles, null);

                map.put("access_token", jwtTokenPair.getAccessToken());
                map.put("refresh_token", jwtTokenPair.getRefreshToken());

                ResponseUtil.responseJsonWriter(response, RestBody.okData(map, "登录
    成功"));
            };
        }
```

3.2 AuthenticationFailureHandler 返回认证失败信息

`AuthenticationFailureHandler` 处理认证失败后的逻辑，前端根据此返回进行跳转处理逻辑，我们也实现它并注入 Spring IoC 容器：

```
        /**
         * 失败登录处理器 处理登录失败后的逻辑 登录失败返回信息 以此为依据跳转
         *
         * @return the authentication failure handler
         */
        @Bean
        public AuthenticationFailureHandler authenticationFailureHandler() {
            return (request, response, exception) -> {
                if (response.isCommitted()) {
                    log.debug("Response has already been committed");
                    return;
                }
                Map<String, Object> map = new HashMap<>(2);

```

```
                map.put("time",
    LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss")));
                map.put("flag", "failure_login");
                ResponseUtil.responseJsonWriter(response,
    RestBody.build(HttpStatus.UNAUTHORIZED.value(), map, "认证失败","-9999"));
            };
        }
```

4. 配置

把上面写好的两个 Handler Bean 写入 登录配置，相关片断如下，详情参见文末 DEMO：

```
    httpSecurity.formLogin().loginProcessingUrl(LOGIN_PROCESSING_URL).successHandler(
    authenticationSuccessHandler).failureHandler(authenticationFailureHandler)
```

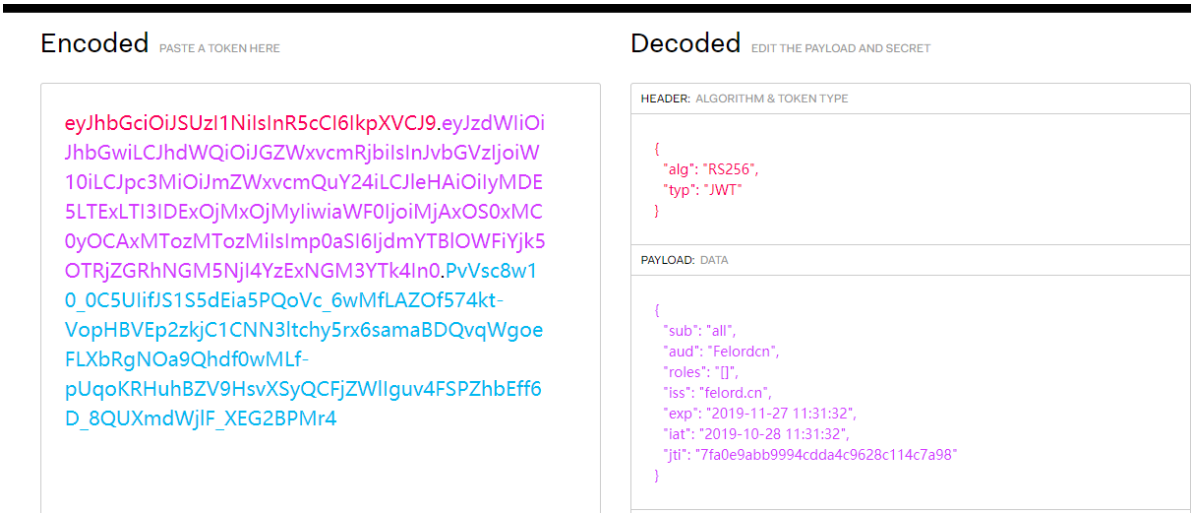
5. 验证

我们依然通过 **Spring Security 实战干货： 玩转自定义登录** 一文中章节 **6.4 测试** 来运行。结果如下：

5.1 登录成功结果

```
{
    "httpStatus": 200,
    "data": {
        "access_token":
"eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhbGwiLCJhdWQiOiJGZWxvcmRjbiIsInJ
vbGVzIjoiW10iLCJpc3MiOiJmZWxvcmQuY24iLCJleHAiOiIyMDE5LTExLTI3IDExOjMxOjMyIiwiaWF0
IjoiMjAxOS0xMC0yOCAxMTozMTozMiIsImp0aSI6IjdmYTBlOWFiYjk5OTRjZGRhNGM5NjI4YzExNGM3Y
Tk4In0.PvVsc8w10_0C5UIifJS1S5dEia5PQoVc_6wMfLAZOf574kt-
VopHBVEp2zkjC1CNN3ltchy5rx6samaBDQvqWgoeFLXbRgNOa9Qhdf0wMLf-
pUqoKRHuhBZV9HsvXSyQCFjZWlIguv4FSPZhbEff6D_8QUXmdWjlF_XEG2BPMr4",
        "refresh_token":
"eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhbGwiLCJhdWQiOiJGZWxvcmRjbiIsInJ
vbGVzIjoiW10iLCJpc3MiOiJmZWxvcmQuY24iLCJleHAiOiIyMDIwLTAxLTI2IDExOjMxOjMyIiwiaWF0
IjoiMjAxOS0xMC0yOCAxMTozMTozMiIsImp0aSI6IjdmYTBlOWFiYjk5OTRjZGRhNGM5NjI4YzExNGM3Y
Tk4In0.Caj4AAothdUwZAFl8IjcAZmmXHgTt76z8trVG1sf_WHZucFVcHR8FWjShhITpArsQpmokP6GBT
MsCvWDl08fUVZBpOWc1CdPUAIIEdArHCFzO64HXc_DLSyg9v0C-
qYfxaTlf0npL5QxpBBr9sJcyzxZF3CnpfZpAxm8WZzXG6o",
        "time": "2019-10-28 11:32:11",
        "flag": "success_login"
    },
    "msg": "登录成功",
    "identifier": ""
}
```

我们取 `access_token` 使用官网**jwt.io** 提供的解码功能进行解码如下：



5.2 登录失败结果

```
{
    "httpStatus": 401,
    "data": {
        "time": "2019-10-28 12:54:10",
        "flag": "failure_login"
    },
    "msg": "认证失败",
    "identifier": "-9999"
}
```

6. 总结

今天我们将 JWT 和 Spring Security 联系了起来，实现了 登录成功后返回 JWT Token 。 这仅仅是一个开始，在下一篇我们将介绍 客户端如何使用 JWT Token 、服务端如何验证 JWT Token ， 敬请关注。文章相关的 DEMO 可通过关注公众号：**码农小胖哥**并回复 `ss06` 获取。

Spring Security中自定义异常处理

1. 前言

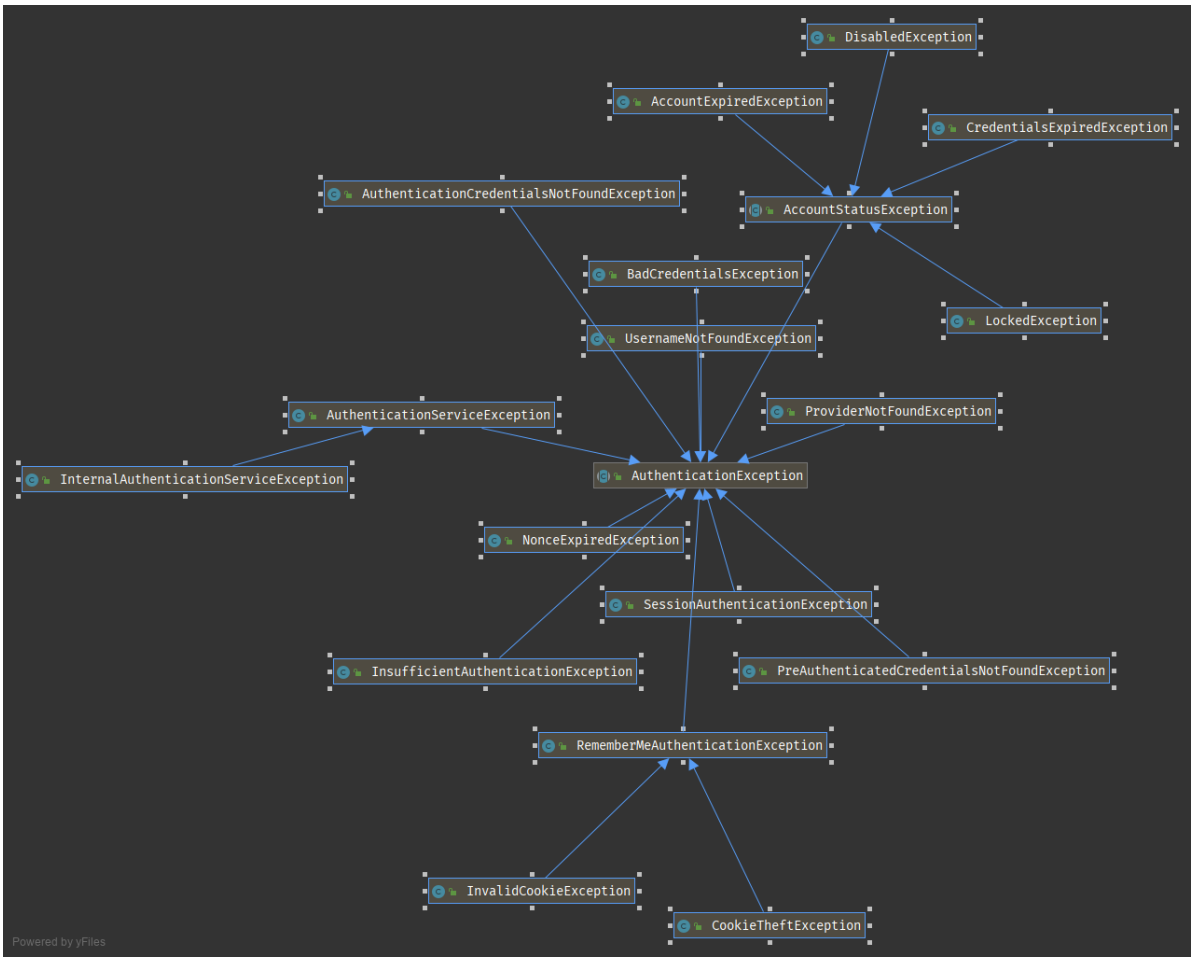
最近实在比较忙，很难抽出时间来继续更 Spring Security **实战干货系列** 。今天正好项目中 Spring Security 需要对认证授权异常的处理，就分享出来吧 。

2. Spring Security 中的异常

Spring Security 中的异常主要分为两大类：一类是认证异常，另一类是授权相关的异常。

2.1 AuthenticationException

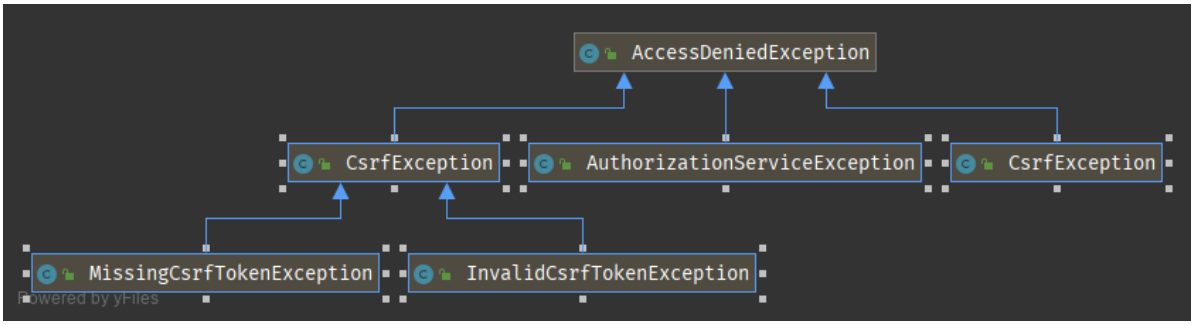
`AuthenticationException` 是在用户认证的时候出现错误时抛出的异常。主要的子类如图：



根据该图的信息，系统用户不存在，被锁定，凭证失效，密码错误等认证过程中出现的异常都由 `AuthenticationException` 处理。

2.2 AccessDeniedException

`AccessDeniedException` 主要是在用户在访问受保护资源时被拒绝而抛出的异常。同 `AuthenticationException` 一样它也提供了一些具体的子类。如下图：



`AccessDeniedException` 的子类比较少，主要是 `CSRF` 相关的异常和授权服务异常。

3. Http 状态对认证授权的规定

Http 协议对认证授权的响应结果也有规定。

3.1 401 未授权状态

HTTP 401 错误 - 未授权(Unauthorized) 一般来说该错误消息表明您首先需要登录（输入有效的用户名和密码）。 如果你刚刚输入这些信息，立刻就看到一个 `401` 错误，就意味着，无论出于何种原因您的用户名和密码其中之一或两者都无效（输入有误，用户名暂时停用，账户被锁定，凭证失效等）。总之就是认证失败了。其实正好对应我们上面的 `AuthenticationException` 。

3.2 403 被拒绝状态

HTTP 403 错误 - 被禁止(Forbidden) 出现该错误表明您在访问受限资源时没有得到许可。服务器理解了本次请求但是拒绝执行该任务，该请求不该重发给服务器。并且服务器想让客户端知道为什么没有权限访问特定的资源，服务器应该在返回的信息中描述拒绝的理由。一般实践中我们会比较模糊的表明原因。 该错误对应了我们上面的 `AccessDeniedException` 。

4. Spring Security 中的异常处理

我们在 Spring Security 实战干货系列文章中的 **自定义配置类入口 WebSecurityConfigurerAdapter** 一文中提到 `HttpSecurity` 提供的 `exceptionHandling()` 方法用来提供异常处理。该方法构造出 `ExceptionHandlingConfigurer` 异常处理配置类。该配置类提供了两个实用接口：

- **AuthenticationEntryPoint** 该类用来统一处理 `AuthenticationException` 异常
- **AccessDeniedHandler** 该类用来统一处理 `AccessDeniedException` 异常

我们只要实现并配置这两个异常处理类即可实现对 Spring Security 认证授权相关的异常进行统一的自定义处理。

4.1 实现 AuthenticationEntryPoint

以 `json` 信息响应。

```
1   import com.fasterxml.jackson.databind.ObjectMapper;
2   import org.springframework.http.MediaType;
3   import org.springframework.security.core.AuthenticationException;
4   import org.springframework.security.web.AuthenticationEntryPoint;
5
6   import javax.servlet.ServletException;
7   import javax.servlet.http.HttpServletRequest;
8   import javax.servlet.http.HttpServletResponse;
9   import java.io.IOException;
10  import java.io.PrintWriter;
11  import java.util.HashMap;
12
13  /**
14   * @author dax
15   * @since 2019/11/6 22:11
16   */
17  public class SimpleAuthenticationEntryPoint implements AuthenticationEntryPoint
    {
18      @Override
19      public void commence(HttpServletRequest request, HttpServletResponse
    response, AuthenticationException authException) throws IOException,
    ServletException {
20
21          //todo your business
```

```
            HashMap<String, String> map = new HashMap<>(2);
            map.put("uri", request.getRequestURI());
            map.put("msg", "认证失败");
            response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
            response.setCharacterEncoding("utf-8");
            response.setContentType(MediaType.APPLICATION_JSON_VALUE);
            ObjectMapper objectMapper = new ObjectMapper();
            String resBody = objectMapper.writeValueAsString(map);
            PrintWriter printWriter = response.getWriter();
            printWriter.print(resBody);
            printWriter.flush();
            printWriter.close();
        }
    }
```

4.2 实现 AccessDeniedHandler

同样以 `json` 信息响应。

```
    import com.fasterxml.jackson.databind.ObjectMapper;
    import org.springframework.http.MediaType;
    import org.springframework.security.access.AccessDeniedException;
    import org.springframework.security.web.access.AccessDeniedHandler;

    import javax.servlet.ServletException;
    import javax.servlet.http.HttpServletRequest;
    import javax.servlet.http.HttpServletResponse;
    import java.io.IOException;
    import java.io.PrintWriter;
    import java.util.HashMap;

    /**
     * @author dax
     * @since 2019/11/6 22:19
     */
    public class SimpleAccessDeniedHandler implements AccessDeniedHandler {
        @Override
        public void handle(HttpServletRequest request, HttpServletResponse response,
    AccessDeniedException accessDeniedException) throws IOException, ServletException
    {
            //todo your business
            HashMap<String, String> map = new HashMap<>(2);
            map.put("uri", request.getRequestURI());
            map.put("msg", "认证失败");
            response.setStatus(HttpServletResponse.SC_FORBIDDEN);
            response.setCharacterEncoding("utf-8");
            response.setContentType(MediaType.APPLICATION_JSON_VALUE);
            ObjectMapper objectMapper = new ObjectMapper();
            String resBody = objectMapper.writeValueAsString(map);
            PrintWriter printWriter = response.getWriter();
            printWriter.print(resBody);
            printWriter.flush();
            printWriter.close();
        }
    }
```

4.3 个人实践建议

其实我个人建议 Http 状态码 都返回 `200` 而将 `401` 状态在 元信息 `Map` 中返回。因为异常状态码在浏览器端会以 error 显示。我们只要能捕捉到 `401` 和 `403` 就能认定是认证问题还是授权问题。

4.4 配置

实现了上述两个接口后，我们只需要在 `WebSecurityConfigurerAdapter` 的 `configure(HttpSecurity http)` 方法中配置即可。相关的配置片段如下：

```
1    http.exceptionHandling()
2            .accessDeniedHandler(new SimpleAccessDeniedHandler())
3            .authenticationEntryPoint(new SimpleAuthenticationEntryPoint())
```

5. 总结

今天我们对 Spring Security 中的异常处理进行了讲解。分别实现了自定义的认证异常处理和自定义的授权异常处理。相关的 DEMO 可关注微信公众号： **码农小胖哥** 回复 ss07 获取。

Spring Security中使用 JWT 认证访问接口

1. 前言

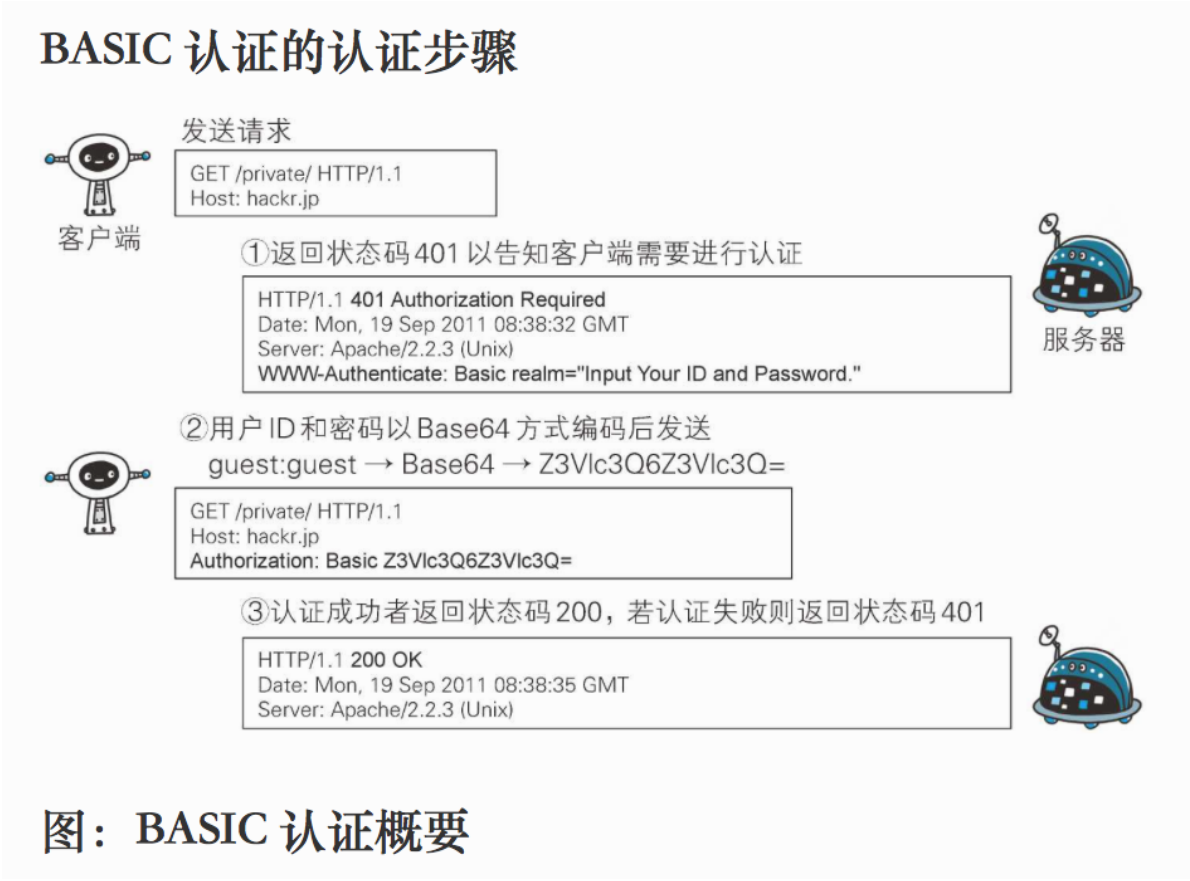
欢迎阅读 Spring Security **实战干货系列** 。之前我讲解了如何编写一个自己的 Jwt 生成器以及如何在用户认证通过后返回 Json Web Token 。今天我们来看看如何在请求中使用 Jwt 访问鉴权。 DEMO 获取方法在文末。

2. 常用的 Http 认证方式

我们要在 Http 请求中使用 Jwt 我们就必须了解 常见的 Http 认证方式。

2.1 HTTP Basic Authentication

HTTP Basic Authentication 又叫基础认证，它简单地使用 `Base64` 算法对用户名、密码进行加密，并将加密后的信息放在请求头 Header 中，本质上还是明文传输用户名、密码，并不安全，所以最好在 Https 环境下使用。其认证流程如下：



图：BASIC 认证概要

客户端发起 **GET** 请求 服务端响应返回 **401 Unauthorized**， `www-Authenticate` 指定认证算法，`realm` 指定安全域。然后客户端一般会弹窗提示输入用户名称和密码，输入用户名密码后放入 **Header** 再次请求，服务端认证成功后以 **200** 状态码响应客户端。

2.2 HTTP Digest Authentication

为弥补 BASIC 认证存在的弱点就有了 **HTTP Digest Authentication** 。它又叫摘要认证。它使用随机数加上 **MD5** 算法来对用户名、密码进行摘要编码，流程类似 **Http Basic Authentication** ，但是更加复杂一些:



图：DIGEST 认证概要

步骤1：跟基础认证一样，只不过返回带 `WWW-Authenticate` 首部字段的响应。该字段内包含质问响应方式认证所需要的临时咨询码（随机数，`nonce`）。首部字段 `WWW-Authenticate` 内必须包含 `realm` 和 `nonce` 这两个字段的信息。客户端就是依靠向服务器回送这两个值进行认证的。`nonce` 是一种每次随返回的 401 响应生成的任意随机字符串。该字符串通常推荐由 Base64 编码的十六进制数的组成形式，但实际内容依赖服务器的具体实现

步骤2：接收到 401 状态码的客户端，返回的响应中包含 DIGEST 认证必须的首部字段 `Authorization` 信息。首部字段 `Authorization` 内必须包含 `username`、`realm`、`nonce`、`uri` 和 `response` 的字段信息，其中，`realm` 和 `nonce` 就是之前从服务器接收到的响应中的字段。

步骤3：接收到包含首部字段 Authorization 请求的服务器，会确认认证信息的正确性。认证通过后则会返回包含 Request-URI 资源的响应。

并且这时会在首部字段 Authorization-Info 写入一些认证成功的相关信息。

2.3 SSL 客户端认证

SSL 客户端认证就是通常我们说的 HTTPS 。安全级别较高，但需要承担 CA 证书费用。SSL 认证过程中涉及到一些重要的概念，数字证书机构的公钥、证书的私钥和公钥、非对称算法（配合证书的私钥和公钥使用）、对称密钥、对称算法（配合对称密钥使用）。相对复杂一些这里不过多讲述。

2.4 Form 表单认证

Form 表单的认证方式并不是HTTP规范。所以实现方式也呈现多样化，其实我们平常的扫码登录，手机验证码登录都属于表单登录的范畴。表单认证一般都会配合 Cookie，Session 的使用，现在很多 Web 站点都使用此认证方式。用户在登录页中填写用户名和密码，服务端认证通过后会将 `sessionId` 返回给浏览器端，浏览器会保存 `sessionId` 到浏览器的 Cookie 中。因为 HTTP 是无状态的，所以浏览器使用 Cookie 来保存 sessionId。下次客户端会在发送的请求中会携带 `sessionId` 值，服务端发现

`sessionId` 存在并以此为索引获取用户存在服务端的认证信息进行认证操作。认证过则会提供资源访问。

我们在 Spring Security **实战干货：登录后返回** JWT Token 一文其实也是通过 Form 提交来获取 Jwt 其实 Jwt 跟 sessionId 同样的作用，只不过 Jwt 天然携带了用户的一些信息，而 `sessionId` 需要去进一步获取用户信息。

2.5 Json Web Token 的认证方式 Bearer Authentication

我们通过表单认证获取 Json Web Token ，那么如何使用它呢? 通常我们会把 Jwt 作为令牌使用 Bearer Authentication 方式使用。Bearer Authentication 是一种基于令牌的 HTTP 身份验证方案，用户向服务器请求访问受限资源时，会携带一个 Token 作为凭证，检验通过则可以访问特定的资源。最初是在 RFC 6750 中作为 OAuth 2.0 的一部分，但有时也可以单独使用。
我们在使用 Bear Token 的方法是在请求头的 `Authorization` 字段中放入 `Bearer <token>` 的格式的加密串(Json Web Token)。**请注意 Bearer 前缀与 Token 之间有一个空字符位，与基本身份验证类似，Bearer Authentication 只能在HTTPS（SSL）上使用。**

3. Spring Security 中实现接口 Jwt 认证

接下来我们是我们该系列的重头戏 ———— 接口的 Jwt 认证。

3.1 定义 Json Web Token 过滤器

无论上面提到的哪种认证方式，我们都可以使用 Spring Security 中的 Filter 来处理。 Spring Security 默认的基础配置没有提供对 Bearer Authentication 处理的过滤器， 但是提供了处理 Basic Authentication 的过滤器：

> org.springframework.security.web.authentication.www.BasicAuthenticationFilter

`BasicAuthenticationFilter` 继承了 `OncePerRequestFilter` 。所以我们也模仿 `BasicAuthenticationFilter` 来实现自己的 `JwtAuthenticationFilter` 。 完整代码如下：

```
1   package cn.felord.spring.security.filter;
2
3   import cn.felord.spring.security.exception.SimpleAuthenticationEntryPoint;
4   import cn.felord.spring.security.jwt.JwtTokenGenerator;
5   import cn.felord.spring.security.jwt.JwtTokenPair;
6   import cn.felord.spring.security.jwt.JwtTokenStorage;
7   import cn.hutool.json.JSONArray;
8   import cn.hutool.json.JSONObject;
9   import lombok.extern.slf4j.Slf4j;
10  import org.springframework.http.HttpHeaders;
11  import
    org.springframework.security.authentication.AuthenticationCredentialsNotFoundExc
    eption;
12  import org.springframework.security.authentication.BadCredentialsException;
13  import org.springframework.security.authentication.CredentialsExpiredException;
14  import
    org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
15  import org.springframework.security.core.AuthenticationException;
16  import org.springframework.security.core.GrantedAuthority;
17  import org.springframework.security.core.authority.AuthorityUtils;
18  import org.springframework.security.core.context.SecurityContextHolder;
```

```
19  import org.springframework.security.core.userdetails.User;
20  import org.springframework.security.web.AuthenticationEntryPoint;
21  import
    org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
22  import org.springframework.util.StringUtils;
23  import org.springframework.web.filter.OncePerRequestFilter;
24
25  import javax.servlet.FilterChain;
26  import javax.servlet.ServletException;
27  import javax.servlet.http.HttpServletRequest;
28  import javax.servlet.http.HttpServletResponse;
29  import java.io.IOException;
30  import java.util.List;
31  import java.util.Objects;
32
33  /**
34   * jwt 认证拦截器 用于拦截 请求 提取jwt 认证
35   *
36   * @author dax
37   * @since 2019/11/7 23:02
38   */
39  @Slf4j
40  public class JwtAuthenticationFilter extends OncePerRequestFilter {
41      private static final String AUTHENTICATION_PREFIX = "Bearer ";
42      /**
43       * 认证如果失败由该端点进行响应
44       */
45      private AuthenticationEntryPoint authenticationEntryPoint = new
    SimpleAuthenticationEntryPoint();
46      private JwtTokenGenerator jwtTokenGenerator;
47      private JwtTokenStorage jwtTokenStorage;
48
49
50      public JwtAuthenticationFilter(JwtTokenGenerator jwtTokenGenerator,
    JwtTokenStorage jwtTokenStorage) {
51          this.jwtTokenGenerator = jwtTokenGenerator;
52          this.jwtTokenStorage = jwtTokenStorage;
53      }
54
55
56      @Override
57      protected void doFilterInternal(HttpServletRequest request,
    HttpServletResponse response, FilterChain chain) throws IOException,
    ServletException {
58          // 如果已经通过认证
59          if (SecurityContextHolder.getContext().getAuthentication() != null) {
60              chain.doFilter(request, response);
61              return;
62          }
63          // 获取 header 解析出 jwt 并进行认证 无token 直接进入下一个过滤器  因为
    SecurityContext 的缘故 如果无权限并不会放行
64          String header = request.getHeader(HttpHeaders.AUTHORIZATION);
65          if (StringUtils.hasText(header) &&
    header.startsWith(AUTHENTICATION_PREFIX)) {
66              String jwtToken = header.replace(AUTHENTICATION_PREFIX, "");
67
68
69              if (StringUtils.hasText(jwtToken)) {
```

```
70                  try {
71                      authenticationTokenHandle(jwtToken, request);
72                  } catch (AuthenticationException e) {
73                      authenticationEntryPoint.commence(request, response, e);
74                  }
75              } else {
76                  // 带安全头 没有带token
77                  authenticationEntryPoint.commence(request, response, new
    AuthenticationCredentialsNotFoundException("token is not found"));
78              }
79
80          }
81          chain.doFilter(request, response);
82      }
83
84      /**
85       * 具体的认证方法  匿名访问不要携带token
86       * 有些逻辑自己补充 这里只做基本功能的实现
87       *
88       * @param jwtToken jwt token
89       * @param request  request
90       */
91      private void authenticationTokenHandle(String jwtToken, HttpServletRequest
    request) throws AuthenticationException {
92
93          // 根据我的实现 有效token才会被解析出来
94          JSONObject jsonObject = jwtTokenGenerator.decodeAndVerify(jwtToken);
95
96          if (Objects.nonNull(jsonObject)) {
97              String username = jsonObject.getStr("aud");
98
99              // 从缓存获取 token
100             JwtTokenPair jwtTokenPair = jwtTokenStorage.get(username);
101             if (Objects.isNull(jwtTokenPair)) {
102                 if (log.isDebugEnabled()) {
103                     log.debug("token : {}  is  not in cache", jwtToken);
104                 }
105                 // 缓存中不存在就算 失败了
106                 throw new CredentialsExpiredException("token is not in cache");
107             }
108             String accessToken = jwtTokenPair.getAccessToken();
109
110             if (jwtToken.equals(accessToken)) {
111                   // 解析 权限集合  这里
112                 JSONArray jsonArray = jsonObject.getJSONArray("roles");
113
114                 String roles = jsonArray.toString();
115
116                 List<GrantedAuthority> authorities =
    AuthorityUtils.commaSeparatedStringToAuthorityList(roles);
117                 User user = new User(username, "[PROTECTED]", authorities);
118                 // 构建用户认证token
119                 UsernamePasswordAuthenticationToken
    usernamePasswordAuthenticationToken = new
    UsernamePasswordAuthenticationToken(user, null, authorities);
120                 usernamePasswordAuthenticationToken.setDetails(new
    WebAuthenticationDetailsSource().buildDetails(request));
121                 // 放入安全上下文中
```

```
122
     SecurityContextHolder.getContext().setAuthentication(usernamePasswordAuthenticat
     ionToken);
123                 } else {
124                     // token 不匹配
125                     if (log.isDebugEnabled()){
126                         log.debug("token : {}  is  not in matched", jwtToken);
127                     }
128
129                     throw new BadCredentialsException("token is not matched");
130                 }
131             } else {
132                 if (log.isDebugEnabled()) {
133                     log.debug("token : {}  is  invalid", jwtToken);
134                 }
135                 throw new BadCredentialsException("token is invalid");
136             }
137         }
138     }
139
```

具体看代码注释部分，逻辑有些地方根据你业务进行调整。**匿名访问必然是不能带 Token 的！**

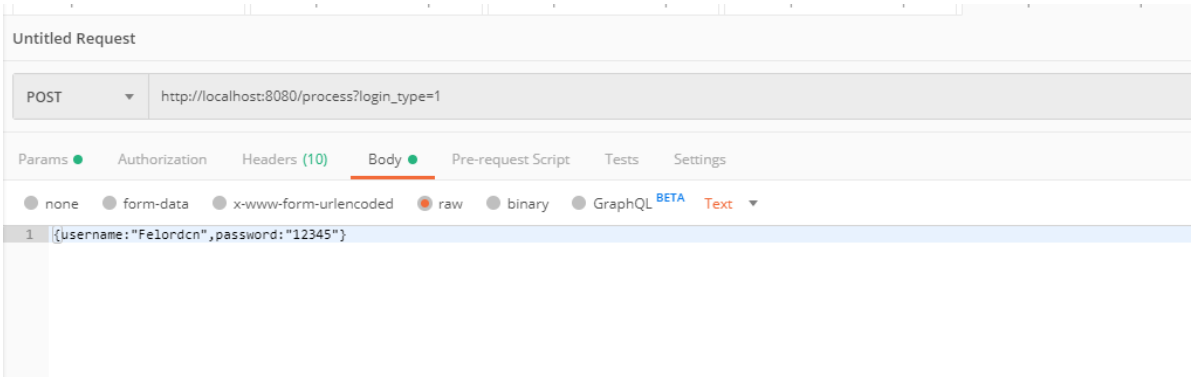
3.2 配置 JwtAuthenticationFilter

首先将过滤器 `JwtAuthenticationFilter` 注入 Spring IoC 容器，然后一定要将 `JwtAuthenticationFilter` 顺序置于 `UsernamePasswordAuthenticationFilter` 之前：

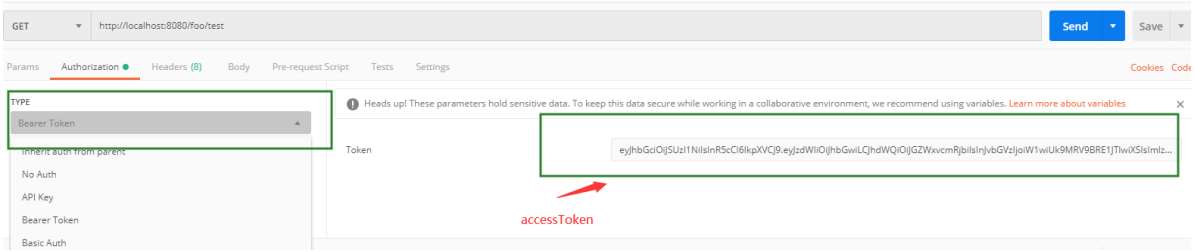
```
1           @Override
2            protected void configure(HttpSecurity http) throws Exception {
3                http.csrf().disable()
4                        .cors()
5                        .and()
6                        // session 生成策略用无状态策略
7
    .sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS)
8                        .and()
9                        .exceptionHandling().accessDeniedHandler(new
    SimpleAccessDeniedHandler()).authenticationEntryPoint(new
    SimpleAuthenticationEntryPoint())
10                       .and()
11                       .authorizeRequests().anyRequest().authenticated()
12                       .and()
13                       .addFilterBefore(preLoginFilter,
    UsernamePasswordAuthenticationFilter.class)
14                       // jwt 必须配置于 UsernamePasswordAuthenticationFilter 之前
15                       .addFilterBefore(jwtAuthenticationFilter,
    UsernamePasswordAuthenticationFilter.class)
16                       // 登录  成功后返回jwt token  失败后返回 错误信息
17
    .formLogin().loginProcessingUrl(LOGIN_PROCESSING_URL).successHandler(authenticati
    onSuccessHandler).failureHandler(authenticationFailureHandler)
18                       .and().logout().addLogoutHandler(new
    CustomLogoutHandler()).logoutSuccessHandler(new CustomLogoutSuccessHandler());
19
20          }
```

4. 使用 Jwt 进行请求验证

编写一个受限接口，我们这里是 `http://localhost:8080/foo/test` 。直接请求会被 401 。 我们通过下图方式获取 Token：



然后在 Postman 中使用 Jwt：



最终会认证成功并访问到资源。

5. 刷新 Jwt Token

我们在 Spring Security 实战干货：手把手教你实现JWT Token 中已经实现了 Json Web Token 都是成对出现的逻辑。 `accessToken` 用来接口请求， `refreshToken` 用来刷新 `accessToken` 。我们可以同样定义一个 Filter 可参照 上面的 `JwtAuthenticationFilter` 。只不过 这次请求携带的是 `refreshToken` ，我们在过滤器中拦截 `URI` 跟我们定义的刷新端点进行匹配。同样验证 `Token` ，通过后像登录成功一样返回 Token 对即可。这里不再进行代码演示。

6. 总结

这是系列原创文章，总有不仔细看的同学抓不着头脑颇有微词。饭需要一口一口的吃，没有现成的可以吃，都是这么过来的，急什么。原创不易，关注才是动力。 Spring Security 实战干货系列 每一篇都有不同的知识点，而且它们都是相互有联系的。有不懂得地方多回头看。Spring Security 并不难学，关键是你找对思路了没有。本次 DEMO 可通过关注公众号：**码农小胖哥** 回复 ss08 获取。

我对RBAC权限控制的理解

1. 前言

欢迎阅读 Spring Security **实战干货系列** 文章 。截止到上一篇我们已经能够简单做到用户主体认证到接口的访问控制了，但是依然满足不了实际生产的需要。 如果我们需要一个完整的权限管理系统就必须了解一下 RBAC (`Role-Based Access Control` 基于角色的访问控制) 的权限控制模型。

2. 为什么需要 RBAC?

在正式讨论 RBAC 模型之前，我们要思考一个问题，为什么我们要做角色权限系统？ 答案很明显，一个系统肯定具有不同访问权限的用户。比如付费用户和非付费用户的权限，如果你是 QQ音乐的会员那么你能听高音质的歌曲，如果不是就不能享受某些便利的、优质的服务。那么这是一成不变的吗？又时候为了流量增长或者拉新的需要，我们又可能把一些原来充钱才能享受的服务下放给免费用户。如果你有了会员等级那就更加复杂了，VIP1 跟 VIP2 具有的功能肯定又有所差别了。主流的权限管理系统都是 RBAC 模型的变形和运用，只是根据不同的业务和设计方案，呈现不同的显示效果。
下图展示了用户和角色以及资源的简单关系：



那为什么不直接给用户分配权限，还多此一举的增加角色这一环节呢？当然直接给用户具体的资源访问控制权限也不是不可以。只是这样做的话就少了一层关系，扩展性弱了许多。如果你的系统足够简单就不要折腾 RBAC 了，怎么简单就怎么玩。如果你的系统需要考虑扩展性和权限控制的多样性就必须考虑 RBAC 。
如果你有多个具有相同权限的用户，再分配权限的时候你就需要重复为用户去 Query (查询) 和 Add (赋予) 权限，如果你要修改，比如上面的 VIP1 增加一个很 Cool 的功能,你就要遍历 VIP1 用户进行修改。有了角色后，我们只需要为该角色制定好权限后，将相同权限的用户都指定为同一个角色即可，便于权限管理。
对于批量的用户权限调整，只需调整该用户关联的角色权限，无需遍历，既大幅提升权限调整的效率，又降低了漏调权限的概率。这样用户和资源权限解除了耦合性，这就是 RBAC 模型的优势所在。

3. RBAC 模型的分类

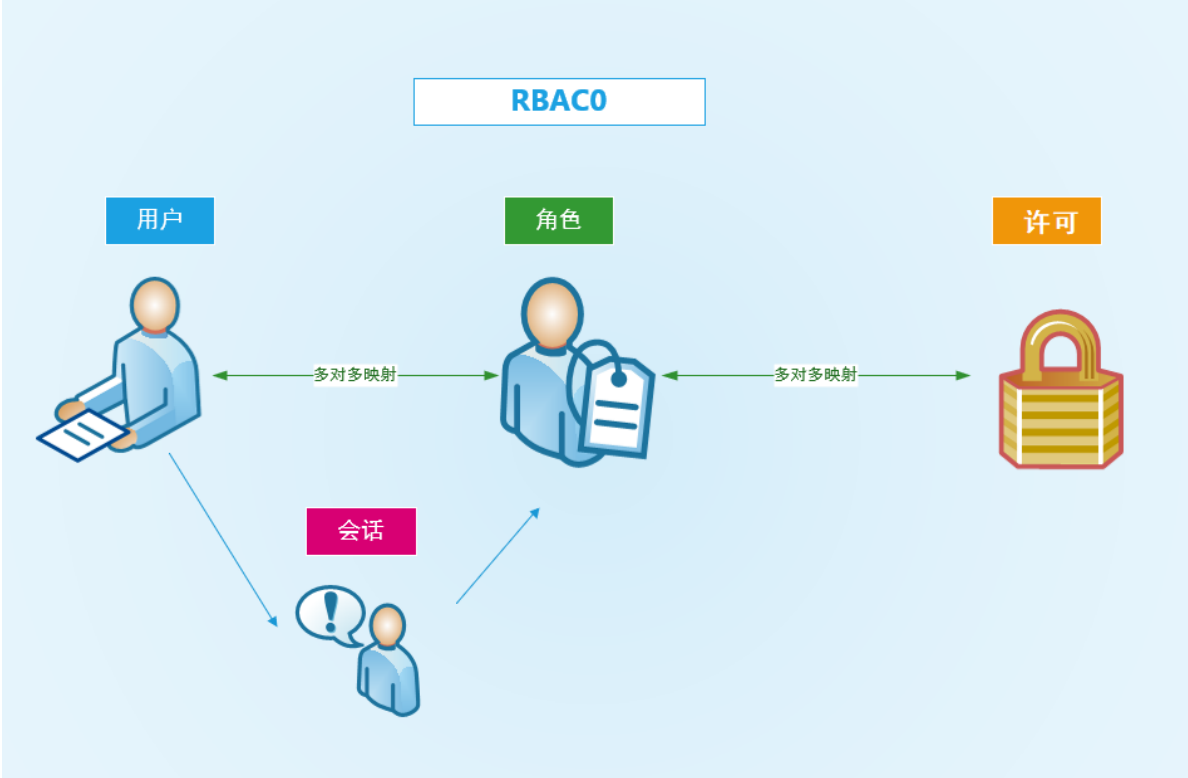
RBAC 模型可以分为： RBAC0、 RBAC1、 RBAC2、 RBAC3 四种。其中 RBAC0 是基础，其它三种都是在 RBAC0 基础上的变种。大部分情况下，使用 RBAC0 模型就可以满足常规的权限管理系统设计了。不过一定不要拘泥于模型，要以业务需要为先导。接下来简单对四种模型进行简单的介绍一下。

3.1 RBAC0

RBAC0 是基础，定义了能构成 RBAC 权限控制系统的最小的集合， RBAC0 由四部分构成：

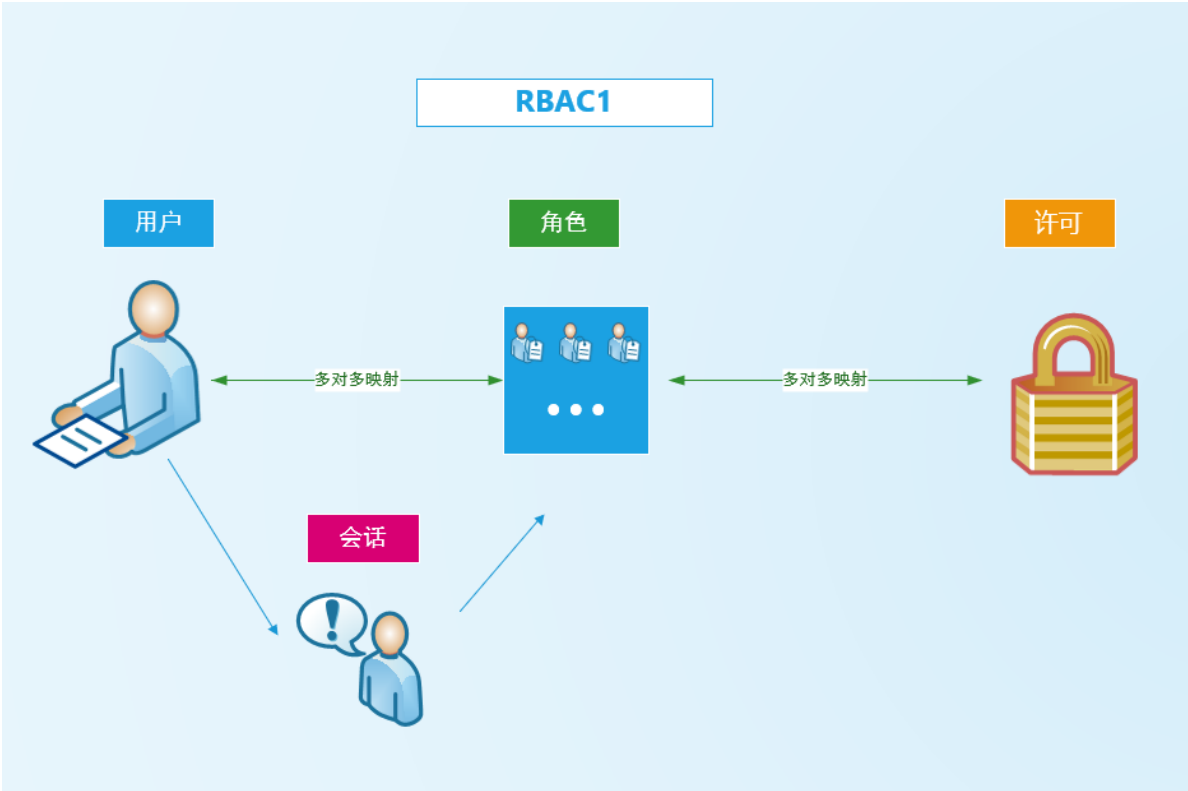
- 用户（User） 权限的使用主体
- 角色（Role） 包含许可的集合

- 会话（Session）绑定用户和角色关系映射的中间通道。而且用户必须通过会话才能给用户设置角色。
- 许可（Pemission） 对特定资源的特定的访问许可。



3.2 RBAC1

RBAC1 在 RBAC0 的基础之上引入了角色继承的概念，有了继承那么角色就有了上下级或者等级关系。父角色拥有其子角色所有的许可。通俗讲就是来说： **你能干的，你的领导一定能干，反过来就不一定能行。**



3.3 RBAC2

> 在体育比赛中，你不可能既是运动员又是裁判员！

这是很有名的一句话。反应了我们经常出现的一种职务（其实也就是角色）冲突。有些角色产生的历史原因就是为了制约另一个角色，裁判员就是为了制约运动员从而让运动员按照规范去比赛。如果一个人兼任这两个角色，比赛必然容易出现不公正的情况从而违背竞技公平性准则。还有就是我们每个人在不同的场景都会充当不同的角色，在公司你就是特定岗位的员工，在家庭中你就是一名家庭成员。随着场景的切换，我们的角色也在随之变化。
所以 **RBAC2** 在 **RBAC0** 的基础上引入了静态职责分离（Static Separation of Duty，简称SSD）和动态职责分离（Dynamic Separation of Duty，简称DSD）两个约束概念。他们两个作用的生命周期是不同的;

- **SSD** 作用于约束用户和角色绑定时。 1.互斥角色：就像上面的例子你不能既是A又是B，互斥的角色只能二选一；2. 数量约束：用户的角色数量是有限的不能多于某个基数； 3. 条件约束：只能达到某个条件才能拥有某个角色。经常用于用户等级体系，只有你充钱成为VIP才能一刀999。
- **DSD** 作用于会话和角色交互时。当用户持有多个角色，在用户通过会话激活角色时加以条件约束，根据不同的条件执行不同的策略。

图就不画了就是在 **RBAC0** 加了上述两个约束。

3.4 RBAC3

> 我全都要!

RBAC1 和 **RBAC2** 各有神通。当你拿着这两个方案给产品经理看时，他给了你一个坚定的眼神：**我全都要！** 于是 RBAC3 就出现了。也就是说 **RBAC3 = RBAC1 + RBAC2** 。

4. RBAC 中一些概念的理解

四个模型说完，我们来简单对其中的一些概念进行进一步的了解。

4.1 用户（User）

对用户的理解不应该被局限于单个用户，也可以是用户组（类似 `linux` 的 User Group），或许还有其它的名字比如部门或者公司；也可以是虚拟的账户，客户，甚至说第三方应用也可以算用户 。所以对用户的理解要宽泛一些，只要是有访问资源需求的主体都可以纳入用户范畴。

4.2 角色（Role）

角色是特定许可的集合以及载体。一个角色可以包含多个用户，一个用户同样的也可以属于多个角色；同样的一个角色可以包含多个用户组，一个用户组也可以具有多个角色，所以角色和用户是多对多的关系。角色是可以细分的，也就是可以继承、可以分组的。

4.3 许可（Permission）

许可一般称它为权限。通常我们将访问的目标统称为资源，不管是数据还是静态资源都是资源。我们访问资源基本上又通过 **api** 接口来访问。所以一般权限都体现在对接口的控制上。再细分的话我将其划分为菜单控制，具体数据增删改查功能控制（前台体现为按钮）。另外许可具有原子性，不可再分。我们将许可授予角色时就是粒度最小的单元。

5. 总结

基于角色的访问控制（RBAC）已成为高级访问控制的主要方法之一。通过RBAC，您可以控制最终用户在广义和精细级别上可以做什么。您可以指定用户是管理员，专家用户还是最终用户，并使角色和访问权限与组织中员工的职位保持一致。仅根据需要为员工完成工作的足够访问权限来分配权限。通过上面的介绍相信一定会让你有所收获。对我接下来的 Spring Security **实战干货** 集成 **RBAC** 也是提前预一下热。其实不管你使用什么安全框架， **RBAC** 都是必须掌握的。如果你有什么问题可以留言讨论。也可以通过关注公众号 **码农小胖哥** 与我联系。

基于配置的接口访问控制

1. 前言

欢迎阅读 Spring Security **实战干货** 系列文章 。对于受限的访问资源，并不是对所有认证通过的用户开放的。比如 **A** 用户的角色是会计，那么他就可以访问财务相关的资源。**B** 用户是人事，那么他只能访问人事相关的资源。我们在 一文中也对基于角色的访问控制的相关概念进行了探讨。在实际开发中我们如何对资源进行角色粒度的管控呢？今天我来告诉你 **Spring Security** 是如何来解决这个问题的。

2. 将角色写入 UserDetails

我们使用 `UserDetailsService` 加载 `UserDetails` 时也会把用户的 `GrantedAuthority` 权限集写入其中。你可以将角色持久化并在这个点进行注入然后配置访问策略，后续的问题交给 **Spring Security** 。

3. 在 HttpSecurity 中进行配置角色访问控制

我们可以通过配置 `WebSecurityConfigurerAdapter` 中的 `HttpSecurity` 来控制接口的角色访问。

3.1 通过判断用户是否持有角色来进行访问控制

> httpSecurity.authorizeRequests().antMatchers("/foo/test").hasRole("ADMIN")

表示 持有 `ROLE_ADMIN` 角色的用户才能访问 `/foo/test` 接口。注意： `hasRole(String role)` 方法入参不能携带前缀 `ROLE_` 。我们来查看 `SecurityExpressionRoot` 中相关源码:

```
1
2       public final boolean hasRole(String role) {
3           return hasAnyRole(role);
4       }
5
```

很明显 `hasRole` 方法源于 `hasAnyRole` (持有任何其中角色之一，就能满足访问条件，用于一个接口开放给多个角色访问时) :

```
    public final boolean hasAnyRole(String... roles) {
        return hasAnyAuthorityName(defaultRolePrefix, roles);
    }
```

如果一个接口开放给多个角色，比如 `/foo/test` 开放给了 `ROLE_APP` 和 `ROLE_ADMIN` 可以这么写：

> httpSecurity.authorizeRequests().antMatchers("/foo/test").hasAnyRole("APP","ADMIN")

`hasAnyRole` 方法最终的实现为 `hasAnyAuthorityName(String prefix, String... roles)`：

```
private boolean hasAnyAuthorityName(String prefix, String... roles) {
        Set<String> roleSet = getAuthoritySet();

        for (String role : roles) {
            String defaultedRole = getRoleWithDefaultPrefix(prefix, role);
            if (roleSet.contains(defaultedRole)) {
                return true;
            }
        }

        return false;
    }
```

上面才是根本的实现，需要一个 `prefix` 和每一个 `role` 进行拼接，然后用户的角色集合 `roleSet` 中包含了就返回 `true` 放行，否则就 `false` 拒绝。默认的 `prefix` 为 `defaultRolePrefix= ROLE_` 。

3.2 通过判断用户的 GrantedAuthority 来进行访问控制

我们也可以通过 `hasAuthority` 和 `hasAnyAuthority` 来判定。 其实底层实现和 `hasAnyRole` 方法一样，只不过 `prefix` 为 `null` 。也就是你写入的 `GrantedAuthority` 是什么样子的，这里传入参数的就是什么样子的，不再受 `ROLE_` 前缀的制约。

2.1 章节的写法等同如下的写法：

> httpSecurity.authorizeRequests().antMatchers("/foo/test").hasAuthority("ROLE_ADMIN")

> httpSecurity.authorizeRequests().antMatchers("/foo/test").hasAnyAuthority("ROLE_APP","ROLE_ADMIN")

4. 匿名访问

匿名身份验证的用户和未经身份验证的用户之间没有真正的概念差异。 Spring Security 的匿名身份验证只是为您提供了一种更方便的方式来配置访问控制属性。所有的匿名用户都持有角色 `ROLE_ANONYMOUS` 。所以你可以使用 2.1 和 2.2 章节的方法来配置匿名访问:

> httpSecurity.authorizeRequests().antMatchers("/foo/test").hasAuthority("ROLE_ANONYMOUS")

你也可以通过以下方式进行配置：

> httpSecurity.authorizeRequests().antMatchers("/foo/test").anonymous()

5. 开放请求

开放请求可以这么配置：

```
httpSecurity.authorizeRequests().antMatchers("/foo/test").permitAll()
```

6. permitAll 与 anonymous 的一些探讨

开放请求 其实通常情况下跟 **匿名请求** 有交叉。它们的主要区别在于： 当前的 `Authentication` 为 `null` 时 `permitAll` 是放行的，而 `anonymous` 需要 `Authentication` 为 `AnonymousAuthenticationToken` 。这里是比较难以理解的，下面是来自 Spring 文档中的一些信息：

> 通常，采用“默认拒绝”的做法被认为是一种良好的安全做法，在该方法中，您明确指定允许的内容，并禁止其他所有内容。定义未经身份验证的用户可以访问的内容的情况与此类似，尤其是对于Web应用程序。许多站点要求用户必须通过身份验证才能使用少数几个URL（例如，主页和登录页面）。在这种情况下，最简单的是为这些特定的URL定义访问配置属性，而不是为每个受保护的资源定义访问配置属性。换句话说，有时很高兴地说默认情况下需要ROLE_SOMETHING，并且只允许该规则的某些例外，例如应用程序的登录，注销和主页。您还可以从过滤器链中完全忽略这些页面，从而绕过访问控制检查，
> 这就是我们所说的匿名身份验证。

使用 `permitAll()` 将配置授权，以便在该特定路径上允许所有请求（来自匿名用户和已登录用户），`anonymous()` 主要是指用户的状态（是否登录）。基本上，直到用户被“认证”为止，它就是“匿名用户”。就像每个人都有“默认角色”一样。

7. 总结

基于配置来解决基于角色的访问控制是常用的方案之一。也是最容易入门的 **Spring Security** 访问控制技术。下一期我们将介绍基于注解的接口访问控制。

基于注解的接口访问控制

1. 前言

欢迎阅读 **Spring Security 实战干货** 系列文章 。在上一篇 **基于配置的接口角色访问控制** 我们讲解了如何通过 **javaConfig** 的方式配置接口的角色访问控制。其实还有一种更加灵活的配置方式 **基于注解** 。今天我们就来探讨一下。DEMO 获取方式在文末。

2. Spring Security 方法安全

Spring Security 基于注解的安全认证是通过在相关的方法上进行安全注解标记来实现的。

2.1 开启全局方法安全

我们可以在任何 `@Configuration` 实例上使用 `@EnableGlobalMethodSecurity` 注解来启用全局方法安全注解功能。该注解提供了三种不同的机制来实现同一种功能，所以我们单独开一章进行探讨。

3. @EnableGlobalMethodSecurity 注解

```
@Retention(value = java.lang.annotation.RetentionPolicy.RUNTIME)
@Target(value = { java.lang.annotation.ElementType.TYPE })
@Documented
@Import({ GlobalMethodSecuritySelector.class })
@EnableGlobalAuthentication
@Configuration
public @interface EnableGlobalMethodSecurity {

   /**
    * 基于表达式进行方法访问控制
    */
   boolean prePostEnabled() default false;

   /**
    * 基于 @Secured 注解
    */
   boolean securedEnabled() default false;

   /**
    * 基于 JSR-250 注解
    */
   boolean jsr250Enabled() default false;

   boolean proxyTargetClass() default false;

   int order() default Ordered.LOWEST_PRECEDENCE;
}
```

`@EnableGlobalMethodSecurity` 源码中提供了 `prePostEnabled` 、 `securedEnabled` 和 `jsr250Enabled` 三种方式。当你开启全局基于注解的方法安全功能时，也就是使用 `@EnableGlobalMethodSecurity` 注解时我们需要选择使用这三种的一种或者其中几种。我们接下来将分别介绍它们。

4. 使用 prePostEnabled

如果你在 `@EnableGlobalMethodSecurity` 设置 `prePostEnabled` 为 `true` ，则开启了基于表达式的方法安全控制。通过表达式运算结果的布尔值来决定是否可以访问（ `true` 开放， `false` 拒绝）。有时您可能需要执行开启 `prePostEnabled` 复杂的操作。对于这些实例，您可以扩展 `GlobalMethodSecurityConfiguration` ，确保子类上存在 `@EnableGlobalMethodSecurity(prePostEnabled = true)` 。例如，如果要提供自定义 `MethodSecurityExpressionHandler` ：

```java
@EnableGlobalMethodSecurity(prePostEnabled = true)
public class MethodSecurityConfig extends GlobalMethodSecurityConfiguration {
    @Override
    protected MethodSecurityExpressionHandler createExpressionHandler() {
        // ... create and return custom MethodSecurityExpressionHandler ...
        return expressionHandler;
    }
}
```

上面示例属于高级操作，一般没有必要。无论是否继承 `GlobalMethodSecurityConfiguration` 都将会开启四个注解。 `@PreAuthorize` 和 `@PostAuthorize` 侧重于方法调用的控制；而 `@PreFilter` 和 `@PostFilter` 侧重于数据的控制。

4.1 @PreAuthorize

在标记的方法调用之前，通过表达式来计算是否可以授权访问。接下来我来总结以下常用的表达式。

- 基于 `SecurityExpressionOperations` 接口的表达式，也就是我们在**上一文**的 `javaConfig` 配置。示例： `@PreAuthorize("hasRole('ADMIN')")` 必须拥有 `ROLE_ADMIN` 角色。
- 基于 `UserDetails` 的表达式，此表达式用以对当前用户的一些额外的限定操作。示例： `@PreAuthorize("principal.username.startsWith('Felordcn')")` 用户名开头为 `Felordcn` 的用户才能访问。
- 基于对入参的 `SpEL` 表达式处理。关于 `SpEL` 表达式可参考官方文档。或者通过关注公众号：Felordcn 来获取相关资料。 示例： `@PreAuthorize("#id.equals(principal.username)")` 入参 `id` 必须同当前的用户名相同。

4.2 @PostAuthorize

在标记的方法调用之后，通过表达式来计算是否可以授权访问。该注解是针对 `@PreAuthorize` 。区别在于先执行方法。而后进行表达式判断。如果方法没有返回值实际上等于开放权限控制；如果有返回值实际的结果是用户操作成功但是得不到响应。

4.3 @PreFilter

基于方法入参相关的表达式，对入参进行过滤。分页慎用！**该过程发生在接口接收参数之前。** 入参必须为 `java.util.Collection` 且支持 `remove(Object)` 的参数。如果有多个集合需要通过 `filterTarget=<参数名>` 来指定过滤的集合。内置保留名称 `filterObject` 作为集合元素的操作名来进行评估过滤。

样例：

```java
// 入参为Collection<String> ids   测试数据 ["Felordcn","felord","jetty"]

// 过滤掉  felord jetty  为  Felordcn
@PreFilter(value = "filterObject.startsWith('F')",filterTarget = "ids")
// 如果 当前用户持有 ROLE_AD 角色  参数都符合  否则 过滤掉不是 f 开头的
// DEMO 用户不持有 ROLE_AD 角色  故而 集合只剩下 felord
@PreFilter("hasRole('AD') or filterObject.startsWith('f')")
```

4.4 @PostFilter

和 `@PreFilter` 不同的是， 基于返回值相关的表达式，对返回值进行过滤。分页慎用！**该过程发生接口进行数据返回之前。** 相关测试与 `@PreFilter` 相似，参见文末提供的 DEMO。

5. 使用 securedEnabled

如果你在 `@EnableGlobalMethodSecurity` 设置 `securedEnabled` 为 `true` ，就开启了角色注解 `@Secured` ，该注解功能要简单的多，默认情况下只能基于角色（默认需要带前缀 `ROLE_` ）集合来进行访问控制决策。

该注解的机制是只要其声明的角色集合（ `value` ）中包含当前用户持有的任一角色就可以访问。也就是用户的角色集合和 `@Secured` 注解的角色集合要存在非空的交集。 不支持使用 SpEL 表达式进行决策。

6. 使用 jsr250Enabled

启用 JSR-250 安全控制注解，这属于 JavaEE 的安全规范（现为 jakarta 项目）。一共有五个安全注解。如果你在 `@EnableGlobalMethodSecurity` 设置 `jsr250Enabled` 为 `true` ，就开启了 JavaEE 安全注解中的以下三个：

- @DenyAll 拒绝所有的访问
- @PermitAll 同意所有的访问
- @RolesAllowed 用法和 5. 中的 `@Secured` 一样。

7. 总结

今天讲解了 Spring Security 另一种基于注解的静态配置。相比较基于 `javaConfig` 的方式要灵活一些、粒度更细、基于 SpEL 表达式可以实现更加强大的功能。但是这两种的方式还是基于编程的静态方式，具有一定的局限性。更加灵活的方式应该是动态来处理用户的角色和资源的映射关系，这是以后我们将要解决的问题。
本次的 DEMO 可通过 关注微信公众号：**码农小胖哥** 回复 ss09 获取。

安全上下文SecurityContext

1. 前言

欢迎阅读 Spring Security **实战干货** 系列文章 。在前两篇我们讲解了 **基于配置** 和 **基于注解** 来配置访问控制。今天我们来讲一下如何在接口访问中检索当前认证用户信息。
我们先讲一下具体的场景。通常我们在认证后访问需要认证的资源时需要获取当前认证用户的信息。比如 “查询我的个人信息”。如果你直接在接口访问时显式的传入你的 UserID 肯定是不合适的。因为你认证通过后访问资源，系统是知道你是谁的。而且显式的暴露用户的检索接口也不安全。所以我们需要一个业务中可以检索当前认证用户的工具。 接下来我们来看看 Spring Security 是如何解决这个痛点的。

2. 安全上下文 SecurityContext

不知道你有没有留意 Spring Security 实战干货：使用 JWT 认证访问接口 中是如何实现 JWT 认证拦截器 `JwtAuthenticationFilter` 。当服务端对 JWT Token 认证通过后，会将认证用户的信息封装到 `UsernamePasswordAuthenticationToken` 中 并使用工具类放入安全上下文 `SecurityContext` 中，当服务端响应用户后又使用同一个工具类将 `UsernamePasswordAuthenticationToken` 从 `SecurityContext` 中 `clear` 掉。
我们来简单了解 `SecurityContext` 具体是个什么东西。

```
package org.springframework.security.core.context;

import java.io.Serializable;
import org.springframework.security.core.Authentication;

public interface SecurityContext extends Serializable {
    Authentication getAuthentication();

    void setAuthentication(Authentication var1);
}
```

从源码上来看很简单就是一个 存储 `Authentication` 的容器。而 `Authentication` 是一个用户凭证接口用来作为用户认证的凭证使用，通常常用的实现有 **认证用户** `UsernamePasswordAuthenticationToken` 和 **匿名用户** `AnonymousAuthenticationToken` 。其中 `UsernamePasswordAuthenticationToken` 包含了 `UserDetails` ，`AnonymousAuthenticationToken` 只包含了一个字符串 `anonymousUser` 作为匿名用户的标识。我们通过 `SecurityContext` 获取上下文时需要来进行类型判断。接下来我们来聊聊操作 `SecurityContext` 的工具类。

3. SecurityContextHolder

这个工具类就是 `SecurityContextHolder` 。 它提供了两个有用的方法：

- **setContext** 设置当前的 `SecurityContext`
- **getContext** 获取当前的 `SecurityContext` ,进而你可以获取到当前认证用户。
- **clearContext** 清除当前的 `SecurityContext`

平常我们通过这三个方法来操作安全上下文 `SecurityContext` 。你可以直接在代码中使用工具类 `SecurityContextHolder` 获取用户信息，像下面一样：

```
 public String getCurrentUser() {
    Authentication authentication =
SecurityContextHolder.getContext().getAuthentication();

     if (authentication instanceof AnonymousAuthenticationToken){
         return "anonymousUser";
     }
    UserDetails principal = (UserDetails) authentication.getPrincipal();
    return principal.getUsername();
}
```

通过上面的自定义方法就可以解析到 `UserDetails` 的用户信息,你可以扩展 `UserDetails` 使得信息符合你的业务需要。上面方法中的判断是必须的，如果是匿名用户（`AnonymousAuthenticationToken`）返回的 `Principal` 类型是一个字符串 `anonymousUser` 。

3.1 扩展知识：SecurityContextHolder 存储策略

这里也扩展一下知识面，简单讲一下 `SecurityContextHolder` 是如何存储 `SecurityContext` 的。`SecurityContextHolder` 默认有三种存储 `SecurityContext` 的策略：

- **MODE_THREADLOCAL** 利用 `ThreadLocal` 机制来保存每个使用者的 `SecurityContext`，**缺省策略**，平常我们使用这个就行了。
- **MODE_INHERITABLETHREADLOCAL** 利用 `InheritableThreadLocal` 机制来保存每个使用者的 `SecurityContext`。多用于多线程环境环境下。
- **MODE_GLOBAL** 静态机制，作用域为全局。目前不太常用。

4. 总结

`SecurityContext` 是 **Spring Security** 中的一个非常重要类，今天不但介绍 `SecurityContext` 是什么、有什么作用，也对以前讲过的一些知识进行回顾。
也对如何使用 `SecurityContextHolder` 操作 `SecurityContext` 进行了讲解。最后也简单讲述了 `SecurityContextHolder` 三种存储 `SecurityContext` 的策略和使用场景 。希望对你学习 **Spring Security** 有帮助。还请多多关注：**码农小胖哥**。

实现动态权限控制（上）

1. 前言

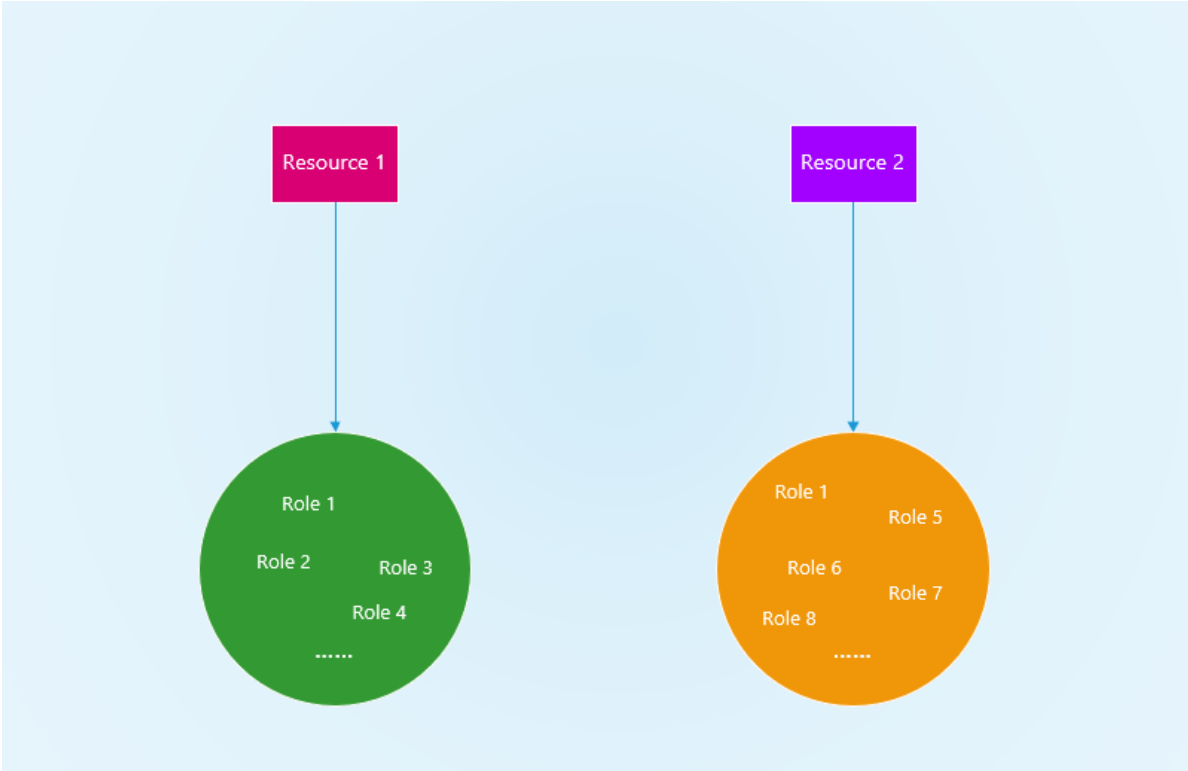
欢迎阅读 **Spring Security 实战干货系列** 文章 。截止目前已经对 **基于配置** 和 **基于注解** 的角色访问控制进行了讲解。对于一些小项目来说基本是够用的。然而如果希望运营管理人员能够动态的配置和分配权限，以上两种方式显然是满足不了需求的。接下来我们来一起探讨一下思路。

2. 动态的权限控制同样依赖 RBAC 模型

我们依然应该在 **RBAC** 及其变种的基础上构建动态的权限控制系统。所有被访问的目标，无论是 API、静态资源都应该是关联了角色的东西统称为 **资源（Resource）** 。我们需要建立起角色和资源之间的关系。

2.1 资源映射到角色

下面是一个资源到角色的映射关系图：



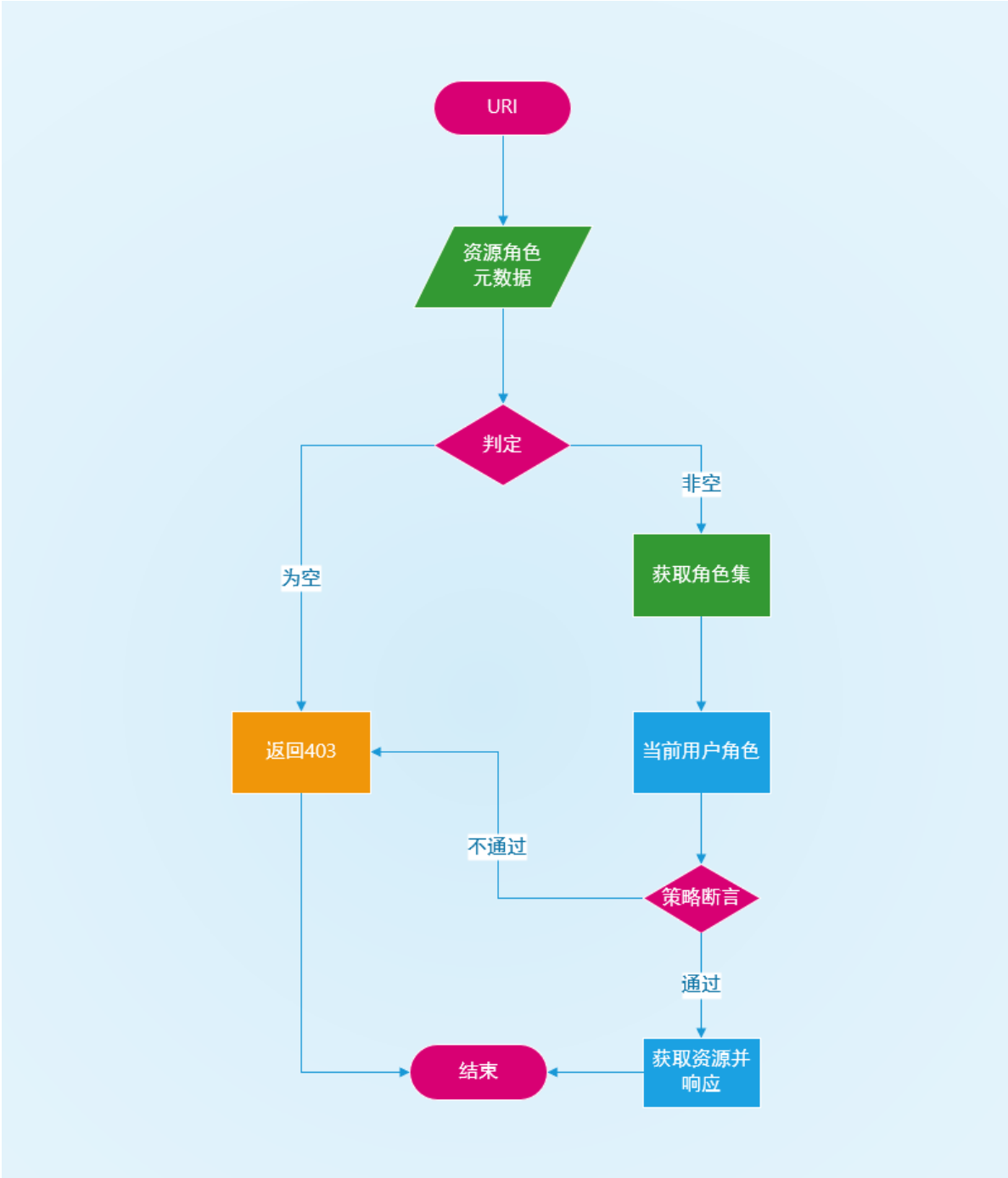
模型大致如上所示，每一个资源对应一个可能无重复的角色集（`Set` 集合）；你可以注意到一个细节 `Role 1` 既指向 `Resource 1` 又指向 `Resource 2` 中，这是可以理解的，毕竟有可能对同一资源的访问权可能分散到多个角色中去；当然也可以互斥这取决于你的业务。
我们选择资源映射到角色是因为当请求时，资源是唯一的而角色可能是多个，如果进行反转的话解析的效率低一些。

3. 请求认证过程

这里有很多搞法，但是总体的思路是我们的请求肯定是带下面两个东西（起码在走到进行访问决策这一步是必须有的）：

- URI 访问资源必然要用 URI 来定位，我们同样通过 URI 来和资源接口进行匹配；**最好是** Ant match，因为 `/user/1` 和 `/user/2` 有可能访问的是同一个资源接口。如果你想避免这种情况，要么在开发规约中禁止这种风格，这样的好处是配置人员可以不必熟悉 Ant 风格；要么必须让配置人员掌握 Ant 风格。
- Principal， Spring Security 中为 `Authentication` （认证主体），之前讲过的一个比较绕的概念，Spring Security 中的用户身份有两种 一种是 **认证用户** 另一种是 **匿名用户** ，它们都包含角色。拿到角色到角色集进行匹配。

然后我画了一个下面的图来更加清晰的展示一下流程：



4. 如何结合安全框架

虽然本文是 Spring Security 系列的，但是我们如果使用其它安全框架或者自己研发安全框架都可以依据上面的思路。如果需要用编程语言总结一下就是我们需要两个接口来协同：

- **获取资源角色关系这些元数据的接口** 这是我们动态权限控制的基石，只有将角色和资源的映射关系接口化才能动态的进行权限控制。 这里没有唯一标准，根据你的业务来设计。
- **对 Request 进行解析并和提取的元数据进行匹配的接口** 这是我们动态权限控制的最终逻辑实现。这里的规则同样也没有唯一标准

抓住了这两点之后我们就非常了然了，无非实现一个具有这两种功能的 `Filter` ，注入安全框架的过滤链中的合适位置中。要么你可以自己造个轮子，要么你使用现在有的轮子。那么有没有现成的轮子呢？我一般建议如果你在造轮子前先看看你选型的安全框架有没有现成的轮子可用。当现成有轮子可用并且能够满足你的需要时往往能够事半功倍。如果没有合适的就造一个！

5. 总结

本篇主要理清一下动态权限所需要的一些要点，并对请求认证的过程进行了分析。最后对结合安全框架定制也提供了一些个人的见解。实现也写了大部分，之所以拆分成上下篇，因为理论和实现放在一篇的话实在有点篇幅过长，分成上篇理论、下篇实践更加合适。

实现动态权限控制（下）

1. Spring Security 提供的轮子

Spring Security 实战干货：内置 Filter 全解析 中提到的第 32 个 `Filter` 不知道你是否有印象。它决定了访问特定路径应该具备的权限，访问的用户的角色，权限是什么？访问的路径需要什么样的角色和权限？ 它就是 `FilterSecurityInterceptor` ，正是我们需要的那个轮子。

2. FilterSecurityInterceptor

过滤器排行榜第 32 位！肩负对接口权限认证的重要职责。我们来看它的过滤逻辑：

```
    public void doFilter(ServletRequest request, ServletResponse response,
            FilterChain chain) throws IOException, ServletException {
        FilterInvocation fi = new FilterInvocation(request, response, chain);
        invoke(fi);
    }
```

初始化了一个 `FilterInvocation` 然后被 `invoke` 方法处理：

```
     public void invoke(FilterInvocation fi) throws IOException, ServletException
{
         if ((fi.getRequest() != null)
                 && (fi.getRequest().getAttribute(FILTER_APPLIED) != null)
                 && observeOncePerRequest) {
             // filter already applied to this request and user wants us to
observe
             // once-per-request handling, so don't re-do security checking
             fi.getChain().doFilter(fi.getRequest(), fi.getResponse());
         }
         else {
             // first time this request being called, so perform security checking
             if (fi.getRequest() != null && observeOncePerRequest) {
                 fi.getRequest().setAttribute(FILTER_APPLIED, Boolean.TRUE);
             }

             InterceptorStatusToken token = super.beforeInvocation(fi);

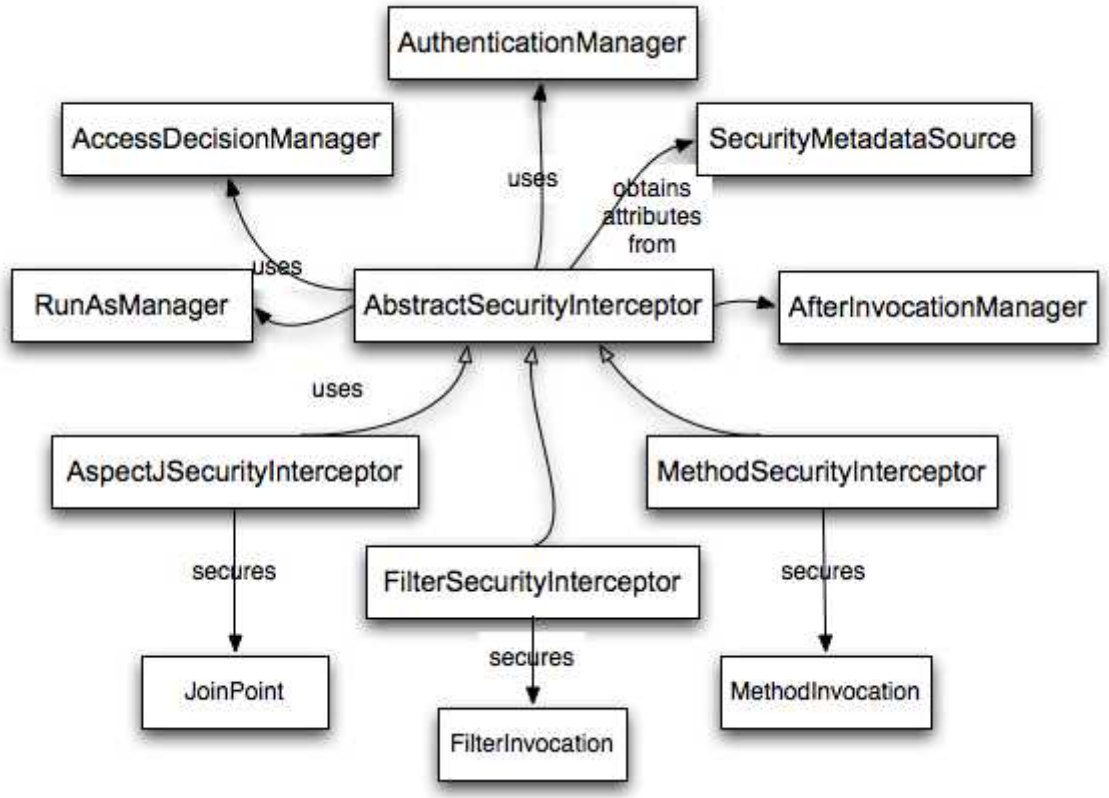
             try {
                 fi.getChain().doFilter(fi.getRequest(), fi.getResponse());
             }
             finally {
                 super.finallyInvocation(token);
             }

```

```
24                 super.afterInvocation(token, null);
25             }
26         }
```

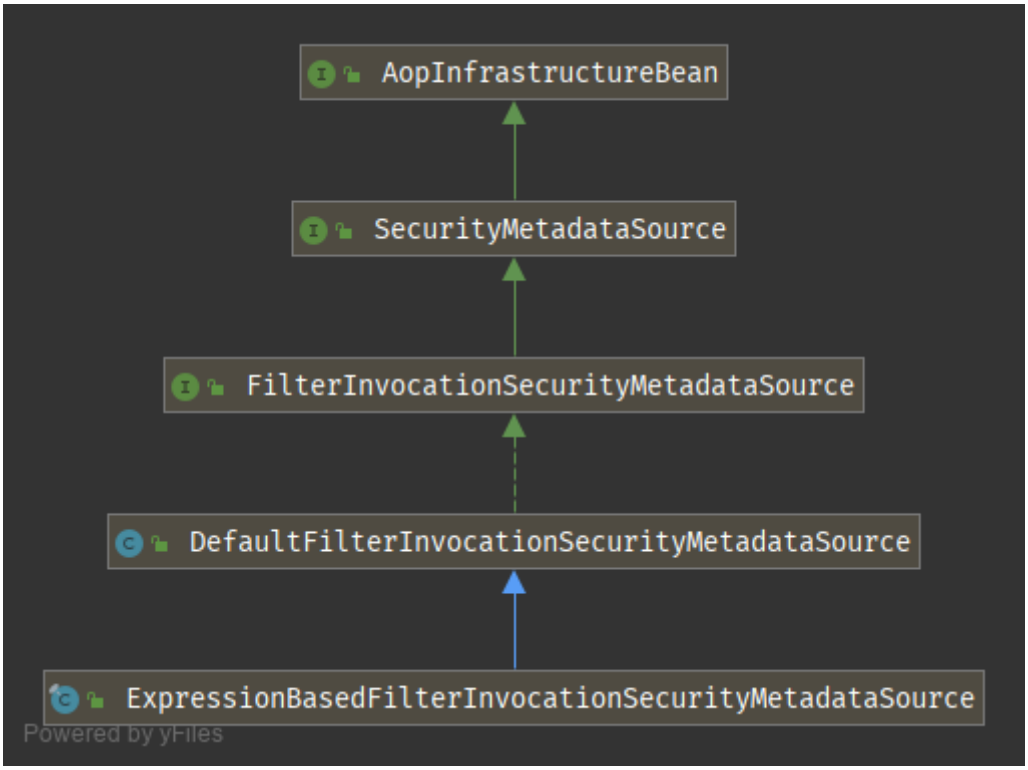
每一次请求被 `Filter` 过滤都会被打上标记 `FILTER_APPLIED`，没有被打上标记的 走了父类的 `beforeInvocation` 方法然后再进入过滤器链，看上去是走了一个前置的处理。那么前置处理了什么呢?
首先会通过 `this.obtainSecurityMetadataSource().getAttributes(Object object)` 拿受保护对象（就是当前请求的URI）所有的映射角色（`ConfigAttribute` 直接理解为角色的进一步抽象）。然后使用访问决策管理器 `AccessDecisionManager` 进行投票决策来确定是否放行。 我们来看一下这两个接口。

安全拦截器和“安全对象”模型参考：



3. FilterInvocationSecurityMetadataSource

这个接口是 `FilterSecurityInterceptor` 的属性，UML图如下：



`FilterInvocationSecurityMetadataSource` 是一个标记接口，其抽象方法继承自 `SecurityMetadataSource``AopInfrastructureBean` 。它的作用是来获取我们**上一篇**文章所描述的**资源角色元数据**。

- **Collection getAttributes(Object object)** 根据提供的受保护对象的信息，其实就是URI，获取该URI 配置的所有角色
- **Collection getAllConfigAttributes()** 这个就是获取全部角色
- **boolean supports(Class<?> clazz)** 对特定的安全对象是否提供 `ConfigAttribute` 支持

3.1 自定义实现 FilterInvocationSecurityMetadataSource 的思路分析

所有的思路仅供参考，实际以你的业务为准！

`Collection<ConfigAttribute> getAttributes(Object object)` 方法的实现：肯定是获取请求中的 `URI` 来和 所有的 资源配置中的 `Ant Pattern` 进行匹配以获取对应的资源配置, 这里需要将资源查询接口查询的资源配置封装为 `AntPathRequestMatcher` 以方便进行 `Ant Match` 。
这里需要特别提一下如果你使用 Restful 风格，这里 **增删改查** 将非常方便你来对资源的管控。参考的实现：

```
1    @Bean
2    public RequestMatcherCreator requestMatcherCreator() {
3      return metaResources -> metaResources.stream()
4              .map(metaResource -> new
     AntPathRequestMatcher(metaResource.getPattern(), metaResource.getMethod()))
5              .collect(Collectors.toSet());
6    }
```

`HttpRequest` 匹配到对应的资源配置后就能根据资源配置去取对应的角色集合。这些角色将交给访问决策管理器 `AccessDecisionManager` 进行投票表决以决定是否放行。

4. AccessDecisionManager

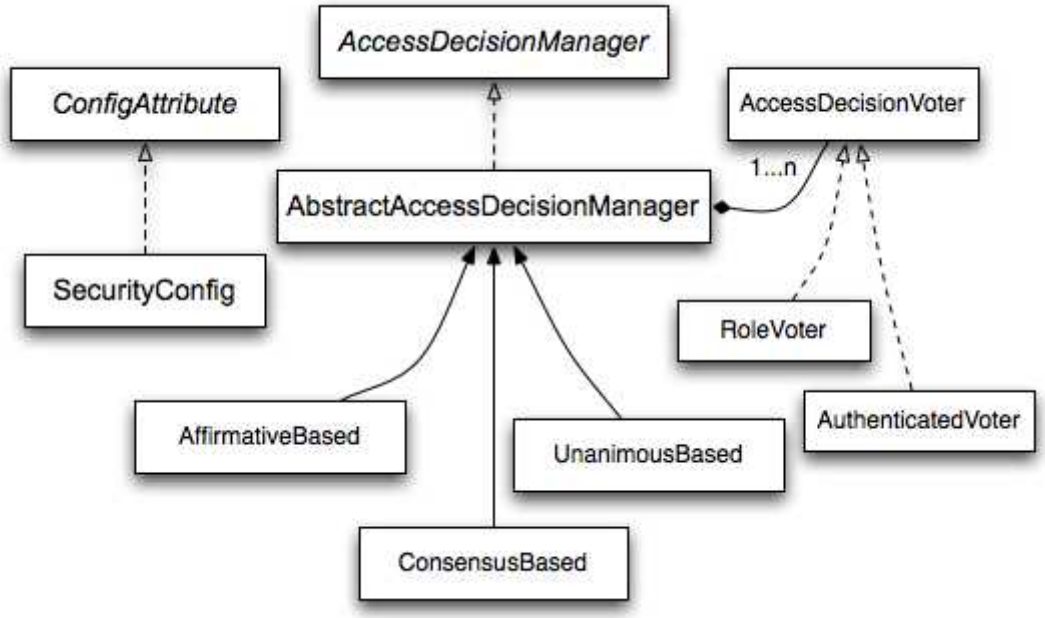
决策管理器，用来投票决定是否放行请求。

```java
public interface AccessDecisionManager {
  // 决策 主要通过其持有的 AccessDecisionVoter 来进行投票决策
  void decide(Authentication authentication, Object object,
          Collection<ConfigAttribute> configAttributes) throws
AccessDeniedException,
          InsufficientAuthenticationException;
 // 以确定AccessDecisionManager是否可以处理传递的ConfigAttribute
  boolean supports(ConfigAttribute attribute);
 //以确保配置的AccessDecisionManager支持安全拦截器将呈现的安全 object 类型。
  boolean supports(Class<?> clazz);
 }
```

`AccessDecisionManager` 有三个默认实现：

- **AffirmativeBased** 基于肯定的决策器。 用户持有一个同意访问的角色就能通过。
- **ConsensusBased** 基于共识的决策器。 用户持有同意的角色数量多于禁止的角色数。
- **UnanimousBased** 基于一致的决策器。 用户持有的所有角色都同意访问才能放行。

投票决策模型参考：



4.1 自定义 AccessDecisionManager

动态控制权限就需要我们实现自己的访问决策器。我们上面说了默认有三个实现，这里我选择基于肯定的决策器 `AffirmativeBased`，只要用户持有一个持有一个角色包含想要访问的资源就能访问该资源。接下来就是投票器 `AccessDecisionVoter` 的定义了，其实我们可以选择内置的

5. AccessDecisionVoter

`AccessDecisionVoter` 将安全配置属性 `ConfigAttribute` 以特定的逻辑进行解析并基于特定的策略来进行投票，投赞成票时总票数 `+1` ，反对票总票数 `-1` ，弃权时总票数 `+0` ,然后由 `AccessDecisionManager` 根据具体的计票策略来决定是否放行。

5.1 角色投票器 RoleVoter

Spring Security 提供的最常用的投票器是角色投票器 `RoleVoter`，它将安全配置属性 `ConfigAttribute` 视为简单的角色名称，并在用户被分配了该角色时授予访问权限。
如果任何 `ConfigAttribute` 以前缀 `ROLE_` 开头，它将投票。如果有一个 `GrantedAuthority` 返回一个字符串(通过 `getAuthority()` 方法)正好等于一个或多个从前缀 `ROLE_` 开始的 `ConfigAttributes`，它将投票授予访问权限。如果没有任何以 `ROLE_` 开头的 `ConfigAttributes` 匹配，则 `RoleVoter` 将投票拒绝访问。如果没有 `ConfigAttribute` 以ROLE_为前缀，将弃权。
这正是我们想要的投票器。

5.2 角色分层投票器 RoleHierarchyVoter

通常要求应用程序中的特定角色应自动“包含”其他角色。例如，在具有 `ROLE_ADMIN` 和 `ROLE_USER` 角色概念的应用中，您可能希望管理员能够执行普通用户可以执行的所有操作。你不得不进行各种复杂的逻辑嵌套来满足这一需求。现在幸好有了 `RoleHierarchyVoter` 可以帮你减少这种负担。
它由上面的 `RoleVoter` 派生,通过配置了一个 `RoleHierarchy` 就可以实现 `ROLE_ADMIN ⇒ ROLE_STAFF ⇒ ROLE_USER ⇒ ROLE_GUEST` 这种层次包含结构，**左边的一定能访问右边可以访问的资源**。具体的配置规则为：角色从左到右、从高到低以 `>` 相连（注意两个空格），以换行符 `\n` 为分割线。举个例子

```
    ROLE_ADMIN > ROLE_STAFF
    ROLE_STAFF > ROLE_USER
    ROLE_USER > ROLE_GUEST
```

请注意动态配置中你需要自行实现角色分层的逻辑。 DEMO 中并未对该风格进行实现。

6. 配置

配置需要两个方面。

6.1 自定义组件的配置

我们需要将元数据加载器 和 访问决策器注入 Spring IoC ：

```
/**
 * 动态权限组件配置
 *
 * @author Felordcn
 */
@Configuration
public class DynamicAccessControlConfiguration {
    /**
     *  RequestMatcher 生成器
     * @return RequestMatcher
     */
    @Bean
    public RequestMatcherCreator requestMatcherCreator() {
        return metaResources -> metaResources.stream()
                .map(metaResource -> new AntPathRequestMatcher(metaResource.getPattern(), metaResource.getMethod()))
                .collect(Collectors.toSet());
    }

```

```
19        /**
20         * 元数据加载器
21         *
22         * @return dynamicFilterInvocationSecurityMetadataSource
23         */
24        @Bean
25        public FilterInvocationSecurityMetadataSource
    dynamicFilterInvocationSecurityMetadataSource() {
26            return new DynamicFilterInvocationSecurityMetadataSource();
27        }
28
29        /**
30         *  角色投票器
31         * @return roleVoter
32         */
33        @Bean
34        public RoleVoter roleVoter() {
35            return new RoleVoter();
36        }
37
38        /**
39         *  基于肯定的访问决策器
40         *
41         * @param decisionVoters  AccessDecisionVoter类型的 Bean 会自动注入到
    decisionVoters
42         * @return affirmativeBased
43         */
44        @Bean
45        public AccessDecisionManager affirmativeBased(List<AccessDecisionVoter<?>>
    decisionVoters) {
46            return new AffirmativeBased(decisionVoters);
47        }
48
49    }
```

Spring Security 的 Java Configuration 不会公开它配置的每个 object 的每个 property。这简化了大多数用户的配置。
虽然有充分的理由不直接公开每个 property，但用户可能仍需要像本文一样的取实现个性化需求。为了解决这个问题，Spring Security 引入了 `ObjectPostProcessor` 的概念，它可用于修改或替换 Java Configuration 创建的许多 `Object` 实例。 `FilterSecurityInterceptor` 的替换配置正是通过这种方式来进行：

```
1   @Configuration
2   @ConditionalOnClass(WebSecurityConfigurerAdapter.class)
3   @ConditionalOnWebApplication(type = ConditionalOnWebApplication.Type.SERVLET)
4   public class CustomSpringBootWebSecurityConfiguration {
5       private static final String LOGIN_PROCESSING_URL = "/process";
6
7       /**
8        * Json login post processor json login post processor.
9        *
10       * @return the json login post processor
11       */
12      @Bean
13      public JsonLoginPostProcessor jsonLoginPostProcessor() {
14          return new JsonLoginPostProcessor();
15      }
```

```
16
17      /**
18       * Pre login filter pre login filter.
19       *
20       * @param loginPostProcessors the login post processors
21       * @return the pre login filter
22       */
23      @Bean
24      public PreLoginFilter preLoginFilter(Collection<LoginPostProcessor>
    loginPostProcessors) {
25          return new PreLoginFilter(LOGIN_PROCESSING_URL, loginPostProcessors);
26      }
27
28      /**
29       * Jwt 认证过滤器.
30       *
31       * @param jwtTokenGenerator jwt 工具类 负责 生成 验证 解析
32       * @param jwtTokenStorage   jwt 缓存存储接口
33       * @return the jwt authentication filter
34       */
35      @Bean
36      public JwtAuthenticationFilter jwtAuthenticationFilter(JwtTokenGenerator
    jwtTokenGenerator, JwtTokenStorage jwtTokenStorage) {
37          return new JwtAuthenticationFilter(jwtTokenGenerator, jwtTokenStorage);
38      }
39
40      /**
41       * The type Default configurer adapter.
42       */
43      @Configuration
44      @Order(SecurityProperties.BASIC_AUTH_ORDER)
45      static class DefaultConfigurerAdapter extends WebSecurityConfigurerAdapter {
46
47          @Autowired
48          private JwtAuthenticationFilter jwtAuthenticationFilter;
49          @Autowired
50          private PreLoginFilter preLoginFilter;
51          @Autowired
52          private AuthenticationSuccessHandler authenticationSuccessHandler;
53          @Autowired
54          private AuthenticationFailureHandler authenticationFailureHandler;
55          @Autowired
56          private FilterInvocationSecurityMetadataSource
    filterInvocationSecurityMetadataSource;
57          @Autowired
58          private AccessDecisionManager accessDecisionManager;
59
60          @Override
61          protected void configure(AuthenticationManagerBuilder auth) throws
    Exception {
62              super.configure(auth);
63          }
64
65          @Override
66          public void configure(WebSecurity web) {
67              super.configure(web);
68          }
69
```

```
70          @Override
71          protected void configure(HttpSecurity http) throws Exception {
72              http.csrf().disable()
73                      .cors()
74                      .and()
75                      // session 生成策略用无状态策略
76
    .sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS)
77                      .and()
78                      .exceptionHandling().accessDeniedHandler(new
    SimpleAccessDeniedHandler()).authenticationEntryPoint(new
    SimpleAuthenticationEntryPoint())
79                      .and()
80                      //   动态权限配置
81
    .authorizeRequests().anyRequest().authenticated().withObjectPostProcessor(filter
    SecurityInterceptorObjectPostProcessor())
82                      .and()
83                      .addFilterBefore(preLoginFilter,
    UsernamePasswordAuthenticationFilter.class)
84                      // jwt 必须配置于 UsernamePasswordAuthenticationFilter 之前
85                      .addFilterBefore(jwtAuthenticationFilter,
    UsernamePasswordAuthenticationFilter.class)
86                      // 登录  成功后返回jwt token  失败后返回 错误信息
87
    .formLogin().loginProcessingUrl(LOGIN_PROCESSING_URL).successHandler(authenticat
    ionSuccessHandler).failureHandler(authenticationFailureHandler)
88                      .and().logout().addLogoutHandler(new
    CustomLogoutHandler()).logoutSuccessHandler(new CustomLogoutSuccessHandler());
89
90          }
91
92          /**
93           * 自定义 FilterSecurityInterceptor  ObjectPostProcessor 以替换默认配置达到
    动态权限的目的
94           *
95           * @return ObjectPostProcessor
96           */
97          private ObjectPostProcessor<FilterSecurityInterceptor>
    filterSecurityInterceptorObjectPostProcessor() {
98              return new ObjectPostProcessor<FilterSecurityInterceptor>() {
99                  @Override
100                 public <O extends FilterSecurityInterceptor> O postProcess(O
    object) {
101                     object.setAccessDecisionManager(accessDecisionManager);
102
     object.setSecurityMetadataSource(filterInvocationSecurityMetadataSource);
103                     return object;
104                 }
105             };
106         }
107
108     }
109 }
```

然后你编写一个 `Controller` 方法就将其在数据库注册为一个资源进行动态的访问控制了。**无须注解或者更详细的 Java Config 配置。**

7. 总结

从最开始到现在一共10个 DEMO 。我们循序渐进地从如何学习 Spring Security 到目前实现了基于 RBAC、动态的权限资源访问控制。如果你能坚持到现在那么已经能满足了一些基本开发定制的需要。当然 Spring Security 还有很多局部的一些概念，我也会在以后抽时间进行讲解。 关注微信公众号 **码农小胖哥** 回复 day10 获取 DEMO 。

在一个应用中实现多个安全策略

1. 前言

欢迎阅读 Spring Security **实战干货** 系列文章 。最近有开发小伙伴提了一个有趣的问题。他正在做一个项目，涉及两种风格，一种是给小程序出接口，安全上使用无状态的JWT Token；另一种是管理后台使用的是Freemarker，也就是前后端不分离的Session机制。用Spring Security该怎么办?

2. 解决方案

我们可以通过多次继承WebSecurityConfigurerAdapter构建多个`HttpSecurity`。`HttpSecurity` 对象会告诉我们如何验证用户的身份，如何进行访问控制，采取的何种策略等等。

如果你看过之前的教程，我们是这么配置的:

```
/**
 * 单策略配置
 *
 * @author felord.cn
 * @see org.springframework.boot.autoconfigure.security.servlet.SpringBootWebSecurityConfiguration
 * @since 14 :58 2019/10/15
 */
@Configuration
@EnableGlobalMethodSecurity(prePostEnabled = true, jsr250Enabled = true, securedEnabled = true)
@EnableWebSecurity
@ConditionalOnClass(WebSecurityConfigurerAdapter.class)
@ConditionalOnWebApplication(type = ConditionalOnWebApplication.Type.SERVLET)
public class CustomSpringBootWebSecurityConfiguration {

    /**
     * The type Default configurer adapter.
     */
    @Configuration
    @Order(SecurityProperties.BASIC_AUTH_ORDER)
    static class DefaultConfigurerAdapter extends WebSecurityConfigurerAdapter {

        @Override
        protected void configure(AuthenticationManagerBuilder auth) throws Exception {
            super.configure(auth);
        }
```

```
26
27          @Override
28          public void configure(WebSecurity web) {
29              super.configure(web);
30          }
31
32          @Override
33          protected void configure(HttpSecurity http) throws Exception {
34              // 配置 httpSecurity
35
36          }
37      }
38  }
```

上面的配置了一个`HttpSecurity`，我们如法炮制再增加一个`WebSecurityConfigurerAdapter`的子类来配置另一个`HttpSecurity`。伴随而来的还有不少的问题要解决。

2.1 如何路由不同的安全配置

我们配置了两个`HttpSecurity`之后，程序如何让小程序接口和后台接口走对应的`HttpSecurity`？

`HttpSecurity.antMatcher(String antPattern)`可以提供过滤机制。比如我们配置：

```
1       @Override
2          protected void configure(HttpSecurity http) throws Exception {
3              // 配置 httpSecurity
4              http.antMatcher("/admin/v1");
5
6          }
```

那么该`HttpSecurity`将只提供给以`/admin/v1`开头的所有URL。这要求我们针对不同的客户端指定统一的URL前缀。

> 举一反三只要`HttpSecurity`提供的功能都可以进行个性化定制。比如登录方式，角色体系等。

2.2 如何指定默认的HttpSecurity

我们可以通过在`WebSecurityConfigurerAdapter`实现上使用`@Order`注解来指定优先级，数值越大优先级越低，没有`@Order`注解将优先级最低。

2.3 如何配置不同的UserDetailsService

很多情况下我们希望普通用户和管理用户完全隔离，我们就需要多个`UserDetailsService`，你可以在下面的方法中对`AuthenticationManagerBuilder`进行具体的设置来配置`UserDetailsService`，同时也可以配置不同的密码策略。

```
1   @Override
2   protected void configure(AuthenticationManagerBuilder auth) throws Exception {
3       DaoAuthenticationProvider daoAuthenticationProvider = new
    DaoAuthenticationProvider();
4       daoAuthenticationProvider.setUserDetailsService(new UserDetailsService() {
5           @Override
6           public UserDetails loadUserByUsername(String username) throws
    UsernameNotFoundException {
7               // 自行实现
8               return  null ;
```

```
        }
    });
    // 也可以设计特定的密码策略
    BCryptPasswordEncoder bCryptPasswordEncoder = new BCryptPasswordEncoder();
    daoAuthenticationProvider.setPasswordEncoder(bCryptPasswordEncoder);
    auth.authenticationProvider(daoAuthenticationProvider);
}
```

2.4 最终的配置模板

上面的几个问题解决之后，我们基本上掌握了在一个应用中执行多种安全策略。配置模板如下：

```
/**
 * 多个策略配置
 *
 * @author felord.cn
 * @see org.springframework.boot.autoconfigure.security.servlet.SpringBootWebSecurityConfiguration
 * @since 14 :58 2019/10/15
 */
@Configuration
@EnableGlobalMethodSecurity(prePostEnabled = true, jsr250Enabled = true, securedEnabled = true)
@EnableWebSecurity
@ConditionalOnClass(WebSecurityConfigurerAdapter.class)
@ConditionalOnWebApplication(type = ConditionalOnWebApplication.Type.SERVLET)
public class CustomSpringBootWebSecurityConfiguration {

    /**
     * 后台接口安全策略. 默认配置
     */
    @Configuration
    @Order(1)
    static class AdminConfigurerAdapter extends WebSecurityConfigurerAdapter {

        @Override
        protected void configure(AuthenticationManagerBuilder auth) throws Exception {
            DaoAuthenticationProvider daoAuthenticationProvider = new DaoAuthenticationProvider();
            //用户详情服务个性化
            daoAuthenticationProvider.setUserDetailsService(new UserDetailsService() {
                @Override
                public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
                    // 自行实现
                    return null;
                }
            });
            // 也可以设计特定的密码策略
            BCryptPasswordEncoder bCryptPasswordEncoder = new BCryptPasswordEncoder();
            daoAuthenticationProvider.setPasswordEncoder(bCryptPasswordEncoder);
            auth.authenticationProvider(daoAuthenticationProvider);
        }
```

```
38
39          @Override
40          public void configure(WebSecurity web) {
41              super.configure(web);
42          }
43
44          @Override
45          protected void configure(HttpSecurity http) throws Exception {
46              // 根据需求自行定制
47              http.antMatcher("/admin/v1")
48                      .sessionManagement(Customizer.withDefaults())
49                      .formLogin(Customizer.withDefaults());
50
51
52          }
53      }
54
55      /**
56       * app接口安全策略. 没有{@link Order}注解优先级比上面低
57       */
58      @Configuration
59      static class AppConfigurerAdapter extends WebSecurityConfigurerAdapter {
60
61          @Override
62          protected void configure(AuthenticationManagerBuilder auth) throws
    Exception {
63              DaoAuthenticationProvider daoAuthenticationProvider = new
    DaoAuthenticationProvider();
64              //用户详情服务个性化
65              daoAuthenticationProvider.setUserDetailsService(new
    UserDetailsService() {
66                  @Override
67                  public UserDetails loadUserByUsername(String username) throws
    UsernameNotFoundException {
68                      // 自行实现
69                      return null;
70                  }
71              });
72              // 也可以设计特定的密码策略
73              BCryptPasswordEncoder bCryptPasswordEncoder = new
    BCryptPasswordEncoder();
74              daoAuthenticationProvider.setPasswordEncoder(bCryptPasswordEncoder);
75              auth.authenticationProvider(daoAuthenticationProvider);
76          }
77
78          @Override
79          public void configure(WebSecurity web) {
80              super.configure(web);
81          }
82
83          @Override
84          protected void configure(HttpSecurity http) throws Exception {
85              // 根据需求自行定制
86              http.antMatcher("/app/v1")
87                      .sessionManagement(Customizer.withDefaults())
88                      .formLogin(Customizer.withDefaults());
89
90
```

```
91              }
92          }
93      }
```

3. 总结

今天我们解决了如何针对不同类型接口采取不同的安全策略的方法，希望对你有用，如果你有什么问题可以留言。多多关注：**码农小胖哥**,更多干货奉上。